ThermoGFN-IF: Multi-Fidelity GFlowNet Fine-Tuning of ADFLIP for Thermostable, Binding-Aware, and Catalysis-Aware Protein Design

A fully specified methodology using SPURS, BioEmu, UMA, KcatNet, GraphKcat, and MMKcat as complementary oracle families

Anonymous Author(s)

Abstract

We develop a complete methodology for fine-tuning the all-atom inverse protein folding model ADFLIP toward thermostable, binding-aware, and catalysis-aware protein design with Generative Flow Networks (GFlowNets). The proposal is motivated by seven complementary advances. First, ADFLIP is a discrete flow-matching inverse-folding model that natively conditions on all-atom structural context, progressively incorporates predicted sidechains during sequence generation, supports ensemble sampling across multiple structural states, and already provides a training-free guidance mechanism for plugging in arbitrary pretrained property predictors [**? ?**]. Second, SPURS rewires a protein language model and an inverse-folding model into a scalable stability predictor that can score all single mutations in one forward pass and support epistatic scoring with an explicit higher-order decoder [18]. Third, BioEmu provides a fast sequence-conditioned equilibrium-ensemble emulator that can generate large sets of statistically independent conformational samples efficiently and was trained using large structural corpora, more than 200 milliseconds of MD simulation, and experimental stability data [16]. Fourth, Meta FAIR's UMA provides a family of atomistic ML potentials that can be used as a higher-fidelity MLIP-MD oracle when the prepared system is in a calibrated paradigm [5, 15**?**]. Fifth, KcatNet provides a wide-turnover geometric deep learning oracle for enzyme sequence and substrate pairs [19]. Sixth, GraphKcat adds catalytic-pocket-aware structure conditioning and jointly predicts $\log_{10} k_{\mathrm{cat}}$, $\log_{10} K_{\mathrm{M}}$, and $\log_{10}(k_{\mathrm{cat}}/K_{\mathrm{M}})$ [17]. Seventh, MMKcat provides a multimodal missing-modality oracle that remains informative when product or structure channels are incomplete [29]. We therefore propose a multi-oracle design stack in which ADFLIP is the generator and edit-policy substrate, SPURS is a cheap stability oracle, BioEmu is a fast whole-chain dynamics oracle, UMA is the higher-fidelity MLIP-MD oracle for calibrated physical rescoring, and the catalytic branch is split between learned kinetic predictors and a new whole-enzyme UMA catalytic dynamics protocol that starts from docked reactant-bound and product-bound complexes.

The manuscript makes ten methodological contributions. (i) It provides a technical review of ADFLIP, SPURS, BioEmu, UMA, KcatNet, GraphKcat, MMKcat, and the core GFlowNet literature, emphasizing the exact capabilities relevant to thermostability, binding, and catalysis optimization. (ii) It formalizes a variable-order multi-mutant protein design problem in which an ADFLIP-designed full-redesign seed may be transformed into a high-order variant with no scientific Hamming-distance cap beyond sequence length and rollout termination. (iii) It specifies a simultaneous multi-oracle method in which ADFLIP is the sole proposal backbone in the primary methodology, while SPURS, BioEmu, UMA, and the catalytic oracles are external oracle branches rather than proposal priors. (iv) It specifies a sequential curriculum in which full-redesign and single-mutant learning are followed by an explicit epistatic double-mutant stage, after which BioEmu and UMA refine higher-order candidates; higher-order SPURS epistasis remains an ablation. (v) It extends the non-iterative training paradigm already present in the manuscript into a teacher-student active-learning workflow whose teacher can optimize fused stability,

binding, and catalytic targets, while the deployed ADFLIP student proposes full multi-mutant sequences in a single pass. (vi) It extends the generator, oracle stack, and reward construction to protein-protein interactions, multimers, protein-ligand systems, and reaction-conditioned enzyme design through explicit complex-versus-component and substrate-versus-product bookkeeping. (vii) It makes complex-aware training a first-class part of the standard curriculum through a mixed batch sampler over monomers, multimers, ligand complexes, and catalytic examples, with explicit warmup and delayed-introduction options. (viii) It treats the initial undesigned protein or complex as an explicitly scored baseline so that the first full-redesign step already receives absolute and relative oracle supervision. (ix) It adds a catalytic-oracle protocol with explicit uncertainty decomposition, metadata validation, reactant-to-product steered whole-enzyme UMA dynamics, near-transition-state harvesting, and optional PMF refinement, with GraphKcat used as the default learned catalytic refinement branch and KcatNet/MMKcat retained as auxiliary predictors rather than the primary implemented RL path. (x) It adds target-conditioned inference so that users may request design candidates satisfying target thermostability, binding, and kinetic properties within explicit tolerances, subject to bounded retries and transparent reporting of predicted properties in either success or failure.

A central design choice in the revised manuscript is how the stability and catalytic oracles are divided by scale and by metadata availability. When the prepared protein system fits within roughly 8000 atoms, whole-protein UMA-MD is the default high-fidelity stability branch. In the 8000–10000 atom range, downweighting UMA is a soft recommendation rather than a hard rule, and the training methodology does not include systems larger than roughly 10 000 atoms. A second central design choice is how catalytic prediction is routed. In the default implemented catalytic RL path, whole-enzyme UMA provides the broad conformational-gating screen, forward and reverse steered UMA dynamics provide a real pathway and barrier signal between reactant-bound and product-bound basins, optional path umbrella sampling refines that barrier into a PMF, and GraphKcat supplies an orthogonal pocket-aware regression signal. KcatNet and MMKcat remain valuable auxiliary catalytic predictors, especially when structure or product modalities are incomplete, but they are not the default control loop in the repository's current catalytic Method III specialization. A third central design choice is how the generator is positioned relative to complex and catalytic design. Under the present ADFLIP-centered formulation, mixed monomer, multimer, ligand, and reaction-conditioned training is a native operating mode rather than an architectural afterthought: ADFLIP already accepts all-atom context, including ligands, nucleotides, and ions, and already supports multi-state conditioning [**?**]. No empirical results are fabricated or implied. Every performance statement about prior models is attributed to the corresponding source, and every newly introduced hyperparameter is explicitly labeled as a proposed default, a tunable range, or a diagnostic threshold.

1 Introduction

Thermostability remains a central objective in protein engineering because it improves expression robustness, storage, manufacturability, and tolerance to mutational drift. In practical workflows, however, protein design is rarely a single-scalar maximization problem. One often wants *multiple* promising sequences rather than a single argmax sequence: sequences that are stable, diverse, structurally plausible, experimentally synthesizable, catalytically competent, and close enough to a trusted seed to make interpretation and wet-lab follow-up tractable. This is precisely the setting in which GFlowNets are attractive. Rather than learning a policy that greedily climbs a reward landscape, a GFlowNet learns to sample terminal objects with probability proportional to an unnormalized reward, thereby preserving diversity across high-reward modes [2, 21, 22]. In protein engineering, that matters because stability, affinity, and catalytic landscapes are strongly multimodal: many distinct combinations of compensatory mutations may preserve foldability while changing activity, and proxy oracles are noisy enough that one wants a diverse candidate set before

committing to expensive downstream evaluations.

This manuscript addresses the following design question. Suppose a fixed target backbone or bound enzyme-substrate context is given. ADFLIP can already generate a plausible initial sequence and corresponding sidechains for that structure by explicit full-atom inverse folding [**?**]. SPURS can score mutations for thermostability quickly and at scale [18]. BioEmu can provide fast sequence-conditioned equilibrium ensembles and stability-related conformational diagnostics [16]. UMA can provide higher-fidelity MLIP-MD rescoring when the prepared system is in a setting where that use is calibrated [5, 15**?**]. KcatNet, GraphKcat, and MMKcat can predict catalytic parameters from increasingly rich combinations of enzyme sequence, substrate identity, products, assay metadata, protein structure, and catalytic-pocket geometry [17, 19, 29]. Can one fine-tune ADFLIP with a GFlowNet objective so that it proposes *diverse, variable-order, thermostable, binding-aware, and catalytically competent* edit trajectories and terminal multi-mutant variants rather than only reconstructing natural sequence statistics? We argue that the answer is yes, provided the optimization is treated as a multi-oracle search problem rather than as a single-oracle RL problem.

The key practical observation is that the relevant oracles sit at different points on the cost-accuracy frontier and along different conditional axes. SPURS is cheap and broad: it produces backbone-conditioned mutation-effect estimates for all point mutations in one forward pass, and its higher-order decoder makes multi-mutant scoring possible [18]. BioEmu is cheap-to-middle-cost and dynamics-aware: it emulates protein equilibrium ensembles and has been trained on structural data, more than 200 milliseconds of MD, and experimental stabilities [16]. UMA is slower but physically richer: it is a universal MLIP family trained across molecules, materials, and catalysts and exposed through the FAIR Chemistry software stack for energy, force, and MD workflows [5**?**]. On the catalytic side, KcatNet is a wide-sequence-plus-substrate screen, MMKcat is robust to missing product or structure channels, and GraphKcat is narrower but pocket-aware and jointly predicts turnover, Michaelis constant, and catalytic efficiency [17, 19, 29]. Yet UMA also introduces a crucial caveat. OMol25, the large DFT corpus underlying Meta's molecular release, contains broad chemical diversity but systems up to ~ 350 atoms; direct, claim-level use of UMA for whole-protein explicit-solvent folding free energies therefore requires explicit calibration on proteins [15]. Likewise, catalytic predictors require explicit calibration against assay context, substrate identity, and metadata completeness. Rigorous methodology must therefore exploit each oracle's strengths without overclaiming its domain of validity.

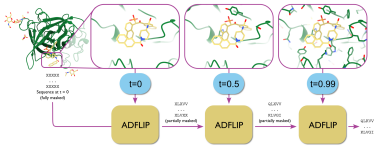
Our proposal is therefore a multi-oracle design system. ADFLIP is the generator. An ideal GFlowNet edit policy traverses a variable-order mutation graph rooted at an ADFLIP seed sequence. SPURS provides dense state-wise mutation landscapes and terminal multi-mutant stability scores. BioEmu provides a whole-chain equilibrium screen that is especially useful for proteins whose prepared all-atom systems exceed roughly 8000–10000 atoms or for classes where whole-protein UMA calibration is weaker. UMA is used as the high-fidelity MLIP-MD oracle whenever the prepared system fits within roughly 8000 atoms, and as a mild down-weighted or hybrid fallback above that size. On the catalytic side, KcatNet, MMKcat, and GraphKcat provide learned kinetic predictors, while the default implemented catalytic RL path adds a dynamics-based whole-enzyme UMA branch that starts from docked reactant-bound and product-bound complexes, estimates conformational gating by productive-pose occupancy, probes the pathway by forward and reverse steered MD, and optionally reconstructs a PMF along an sMD-derived path. This leads to three complementary training procedures. The first, Simul-MF, mixes stability, binding, and catalytic supervision during a single fine-tuning run using asynchronous multi-stage acquisition and uncertainty-aware reward fusion. The second, Seq-Curr, separates the problem into broad cheap-oracle exploration, dynamics

refinement, and targeted high-fidelity rescoring. The third, OneShot-AL, adapts the active-learning style of earlier biological sequence-design GFlowNets [8] into a round-based teacher-student workflow. In the ideal formulation, the teacher is a reward-proportional GFlowNet; in the current repository implementation, the teacher is a reward-weighted policy approximation distilled into a one-shot student. In the catalytic specialization already wired in this repository, OneShot-AL becomes a LigandMPNN-backed round-based loop whose primary catalytic feedback comes from packed-structure GraphKcat prefiltering followed by whole-enzyme UMA broad screening, forward/reverse sMD, optional PMF refinement, and catalytic reward fusion.

The document is written with three principles in mind. First, all sign conventions, symbols, and units are made explicit. Second, the roles of cheap mutation signals, fast whole-chain dynamics signals, high-fidelity MLIP-MD scores, and eventual experimental readouts are kept distinct throughout. Third, because the paper is at present a methodology paper rather than a completed empirical study, every training or evaluation choice is labeled as a proposal rather than as an already validated best practice until evals are presented in supplementary material. The aim is not to imply unearned performance; the aim is to leave as few implementation ambiguities as possible.

2 Technical review of the component models

2.1 Why ADFLIP is the right generator substrate



ADFLIP is an all-atom inverse protein folding model built around discrete flow matching rather than autoregressive decoding or masked sidechain diffusion [**?** **?**]. It models a categorical flow from a fully masked sequence toward the clean amino-acid sequence, while progressively reincorporating predicted sidechains as structural context during sampling. That combination makes it a particularly strong substrate for GFlowNet fine-tuning in the present setting.

First, ADFLIP natively conditions on all-atom structural context. The model was explicitly designed for complexes containing proteins together with ligands, nucleotides, metal ions, and other non-protein atoms [**?**]. This is an important practical simplification for the present manuscript: once ADFLIP becomes the generator substrate, the methodology no longer needs a separate ligand-conditioning retrofit of the core inverse-folding model. Monomers, multimers, protein-protein interfaces, and protein-ligand systems can be treated as native inputs to the same generator.

Second, ADFLIP already supports multi-state inverse folding. The source paper ensembles denoising predictions across multiple conformational states and reports improvements on NMR ensembles of dynamic complexes [**?**]. For thermostability and affinity design, this is valuable because a single rigid backbone is often an incomplete description of the design objective. The same sequence may need to remain compatible with multiple nearby states, hinge motions, or pocket geometries.

Third, ADFLIP exposes useful confidence surrogates even though it does not include a dedicated sidechain-error head like the one used in some other full-atom generators. In practice, the per-position denoiser purity, terminal categorical entropy, agreement across structural states, and post-packing clash or packing-consistency statistics provide cheap diagnostics about whether a

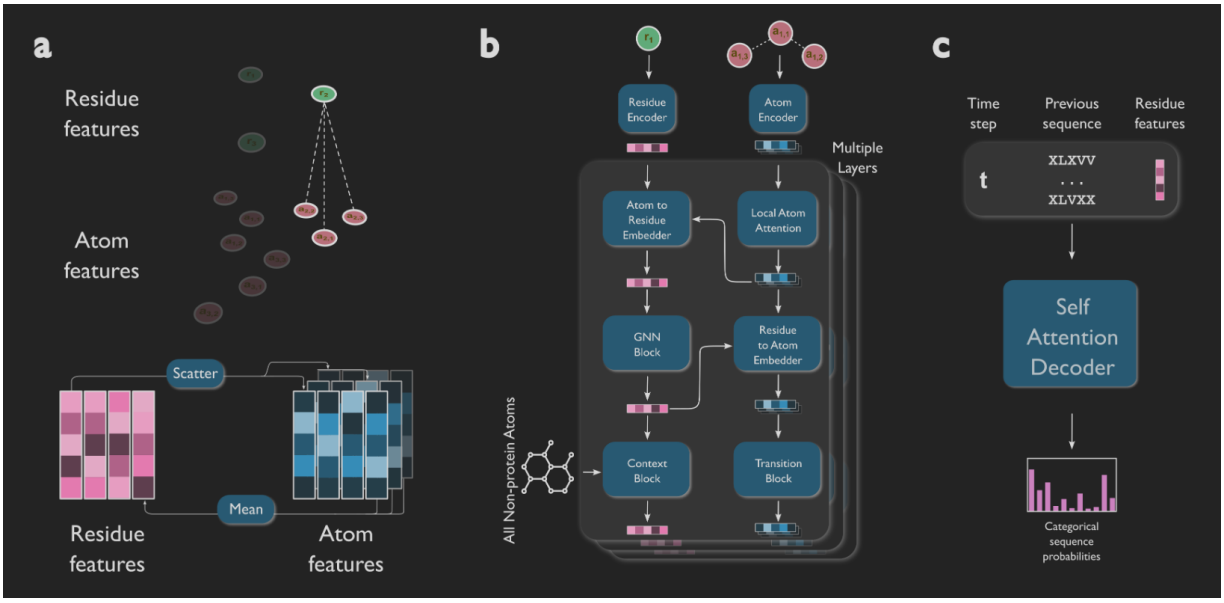


Figure 1: Architecture of the ADFLIP denoiser. a, The information flow between residue features and atom features. Updating residue features from atom features involves averaging over all atoms in a residue; updating atom features from residue features requires scattering the same residue feature over all atoms in that residue. b, Schematic showing the main blocks of the graph neural network (GNN) encoder. c, Schematic showing the inputs and output of the self-attention decoder.

proposed edit is well supported by the current context [**?** **?**]. In the present manuscript these quantities are combined into a packing-quality proxy, u_{pack}, which serves as the main replay-filtering and reward-regularization signal for local structural plausibility.

Fourth, ADFLIP already has a training-free guidance interface. The source paper shows how to approximate a property model on intermediate masked states by taking an expectation of a regressor over the denoiser's completion distribution [**?**]. This is closely aligned with the current tri-fidelity design stack: SPURS, BioEmu-derived surrogate heads, calibrated binding models, or other structure-function predictors can be plugged into the sequence generator without retraining the backbone model from scratch.

Architecturally, ADFLIP uses a multi-scale graph neural network with both residue nodes and atom nodes. An atom encoder handles element, chain, residue-type, and time-step information; a residue encoder summarizes protein and non-protein geometry; the processing trunk alternates local atom attention, atom-to-residue pooling, residue-level message passing, and residue-to-atom feedback; and a residue decoder outputs amino-acid logits [**?**]. Sidechains are packed concurrently during sampling with PIPPACK [**?**]. Because non-protein atoms are first-class citizens in the atom graph, and because progressive sidechain incorporation is part of the native sampling loop, ADFLIP offers a cleaner starting point for the present methodology than a protein-only inverse-folding backbone would.

For the rest of the paper we therefore treat ADFLIP as the sole generator substrate of interest.

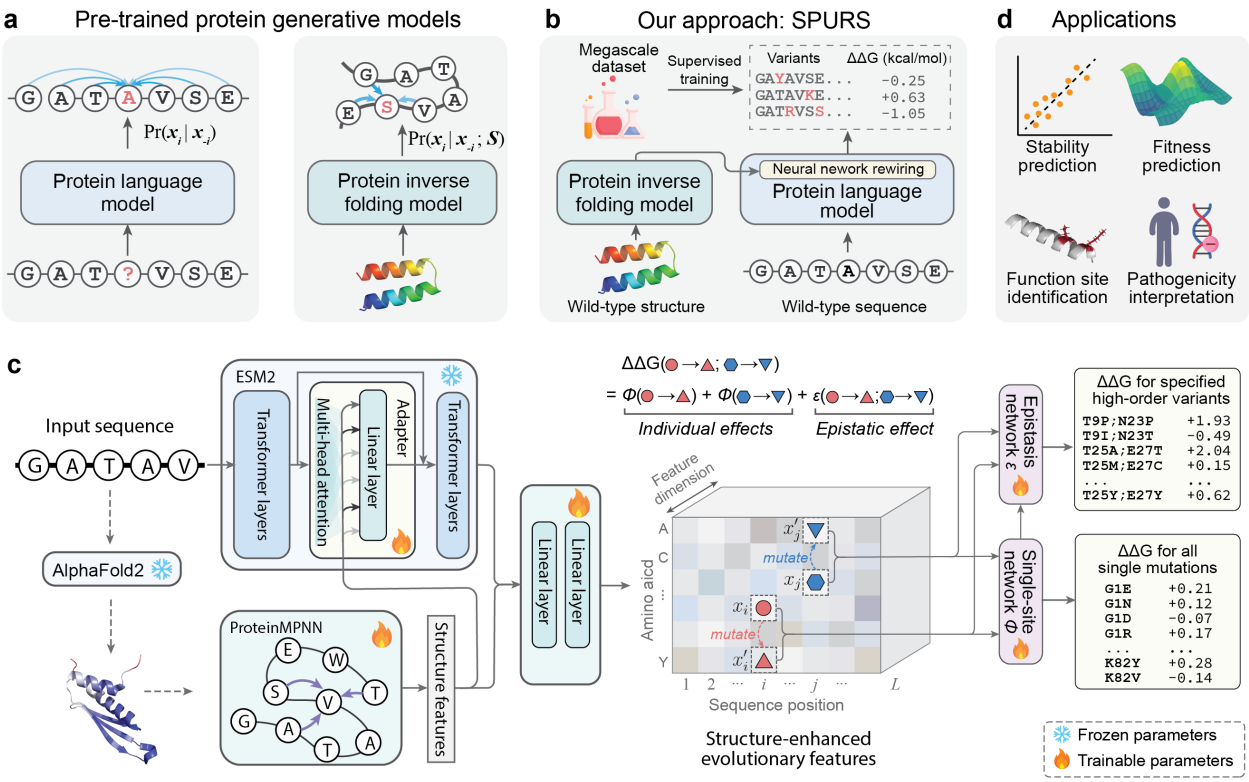


Figure 2: SPURS architecture. SPURS is a deep learning framework that rewires two pre-trained protein generative models (a)–a protein language model (ESM2) and an inverse folding model (ProteinMPNN)–to predict changes in protein thermo-stability ($\Delta\Delta G$) upon sequence mutations (b). c The model takes as input a protein's wild-type sequence and structure (predicted by AlphaFold2 if the experimental structure is unavailable). Through neural network rewiring, SPURS integrates evolutionary priors from ESM2 and structural features from ProteinMPNN to learn structure-enhanced evolutionary features. These representations are passed to a prediction module that efficiently decodes $\Delta\Delta G$ predictions for all single mutations. The same features are reused by another decoder to predict epistatic effects of multiple mutations, enabling $\Delta\Delta G$ prediction for higher-order mutants. d SPURS's performance is demonstrated in stability prediction and diverse applications in protein biology, including functional site identification, protein fitness prediction, and pathogenicity analysis. Abbreviations: Struct=Structure; Evo=Evolution.

2.2 Why use SPURS as the cheap mutation oracle?

SPURS remains attractive because it is inexpensive, global, and already calibrated as a mutation-effect predictor on large thermostability datasets [18]. In one forward pass it produces a per-position, per-amino-acid matrix $\phi(i, a)$ that approximates mutation-conditioned stability. That all-mutants-per-pass structure is extremely useful for screening, replay prioritization, and fast baselining of a candidate sequence against its reference. Just as importantly for the present paper, SPURS is not a diffusion model or a heavy structure predictor; it is cheap enough to evaluate densely during training if desired.

However, the primary methodology in this revised manuscript uses SPURS as an *external oracle*, not as the main proposal engine. ADFLIP is itself small enough for large-batch backpropagation and already has the right inductive bias for all-atom proposal generation, including ligands and multi-state contexts [**?**]. Pushing SPURS too far into the proposal path risks over-constraining the policy to a stability regressor that was not trained as a generative controller. Accordingly,

the primary ADFLIP methodology uses SPURS to provide terminal and intermediate property supervision, oracle calibration anchors, and acquisition scores for what should be sent onward to BioEmu and UMA. SPURS-guided action priors or proposal shortlists are treated as optional ablations rather than defaults.

A second methodological change is that we do *not* replace SPURS's full-sequence epistasis model by ad hoc local block factorization. SPURS was trained with a rewired architecture that includes global interactions through its sequence and structure branches, and its higher-order decoder is most defensible when it is applied to the full candidate sequence in the settings where it has support. In the primary methodology we therefore rely on SPURS directly for single-mutant supervision and for explicit *double-mutant* epistatic supervision, while higher-order SPURS epistasis is relegated to ablations and diagnostics. This choice keeps the cheap branch scientifically conservative without artificially discarding global dependencies or allosteric mutations learned by the model (see "functional site" identification functionality of SPURS).

Finally, SPURS itself admits two structured oracle-backbone ablations that are especially relevant for complex tasks. The first replaces the ESM-2 branch with MINT, a multimer-aware ESM-2 descendant trained on a large curated PPI corpus and shown to improve several interaction-prediction tasks [20]. The second replaces the ProteinMPNN branch with LigandMPNN, which is architecturally close to ProteinMPNN but trained with explicit atomic ligand context and is therefore a more natural drop-in candidate for protein-ligand and complex-heavy settings [**?**]. These are plausible ablations precisely because they are *structured* replacements—MINT is an ESM-2-family refinement and LigandMPNN is a ProteinMPNN-family refinement—rather than arbitrary backbone swaps. They remain ablations because any such substitution changes the oracle's original calibration settings.

2.3 Why GFlowNets instead of greedy search or reward-maximizing RL?

GFlowNets are designed to sample terminal objects with probability proportional to a nonnegative reward rather than to maximize expected terminal reward with a single optimal policy [2]. This distinction is especially relevant in scientific design. When the reward is an imperfect proxy, a diverse batch of high-reward candidates is usually preferable to a single greedy solution. In the original biological sequence-design work with GFlowNets, the method generated more diverse and novel high-scoring biological sequences than several alternative search strategies [8]. More recent work on biological sequence editing extended the idea to seed-centered edits, often with bounded mutation counts [6]. In the present manuscript we inherit the edit-trajectory formulation but deliberately remove any scientific hard cap on terminal Hamming distance, because thermostability optimization can require coordinated redesign of many residues. The GFlowNet literature also provides mature training objectives: detailed balance, trajectory balance, and subtrajectory balance, with the latter often improving optimization stability for longer horizons and delayed rewards [2, 21, 22].

The present design task has exactly the properties under which GFlowNets are attractive: a compositional state space, a structured but noisy reward, long-range credit assignment, and the need for diversity across multiple high-value modes. By contrast, greedy mutation accumulation or PPO-style reward maximization is more susceptible to mode collapse and to over-committing to oracle noise. We therefore treat the GFlowNet not as a generic RL embellishment but as a principled choice for a distributional design objective.

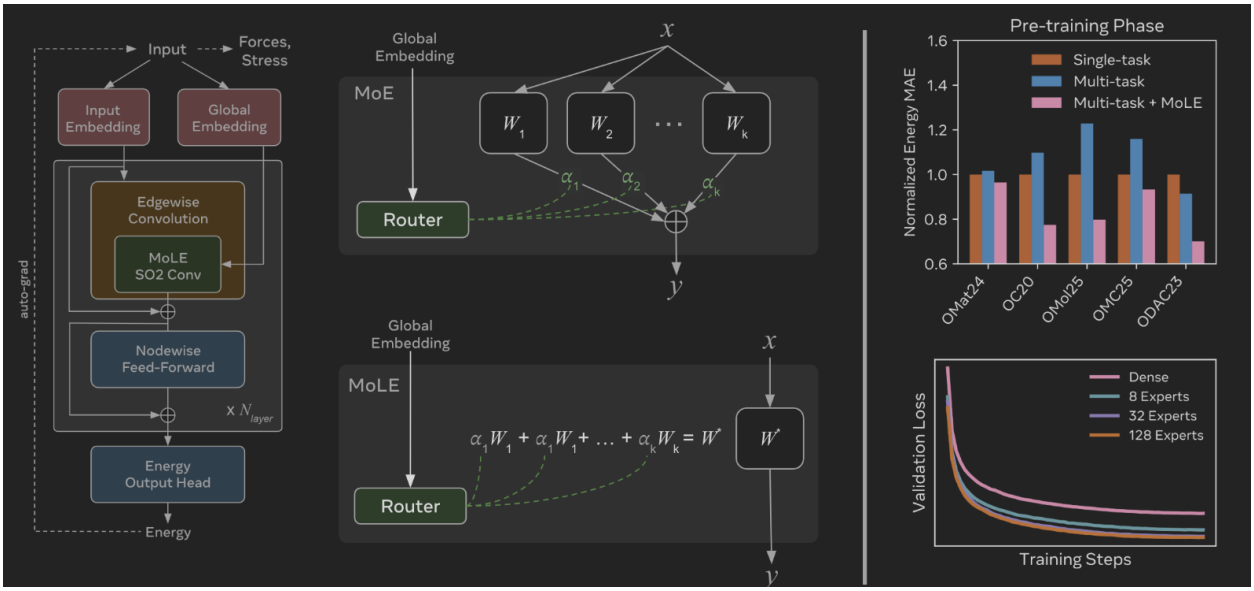


Figure 3: (left) Overview of UMA model architecture. The **SO**(2) convolution is made up of a set of linear operations and each one of these operations is replaced with MoLE. (middle) Illustration of MoE and MoLE. The embedding used for routing, which estimates the expert weights α, only depends on global information making it possible to merge before the model forward pass (middle, bottom), which has substantial benefits for applications that require long roll outs such as molecular dynamics. (right) Bar plot of UMA-S trained with MoLE for multi-task and without MoLE for single and multi-task. Note the MoLE model outperforms non-MoLE models. (right, bottom) Loss plots when varying the number of experts from 1 to 128 for UMA-S.

2.4 Where UMA fits, and where it does not

Meta FAIR's UMA is a family of universal models for atoms trained across molecules, materials, and catalysts [**?**]. The published UMA family is trained on half a billion unique 3D atomic structures, and the medium model has 1.4B total parameters with roughly 50M active parameters per structure through a mixture-of-linear-experts design [**?**]. The FAIR Chemistry software stack exposes UMA through an ASE-compatible calculator with task-specific heads such as `omol` for molecules and provides examples of relaxation, MD, and multi-GPU/LAMMPS execution [5]. From the OMol25 companion release, the training data can be summarized as more than 100 million DFT calculations at the ωB97M-V/def2-TZVPD level, with broad elemental, chemical, and structural diversity and systems up to about 350 atoms [15]. The OMol25 paper explicitly frames the goal as quantum-chemical accuracy at a fraction of the computational cost [15].

These properties are highly appealing for protein design, and the public FAIR Chemistry release materially changes the practical feasibility picture. The repository documents a worked 8000-atom multi-GPU MD example and reports current wall-clock MD throughput of roughly 1 ns/day for 100k+-atom systems, indicating that large-system UMA-MD is computationally viable rather than confined to toy molecules [5]. The remaining scientific question is therefore calibration, not merely throughput. Proteins often contain slow collective motions, protonation ambiguities, solvent effects, and long-range allostery that are not directly benchmarked by the public OMol25 release, whose underlying DFT systems extend to $\sim$ 350 atoms [15]. A rigorous methodology must therefore separate computational scalability from thermodynamic validity on proteins.

In this paper, we recommend defensible utilization of UMA as sitting within the whole-protein paradigm up to $\sim$ 8000 atoms, with weak down-weighting for the $8000 - 10000$ range. When the

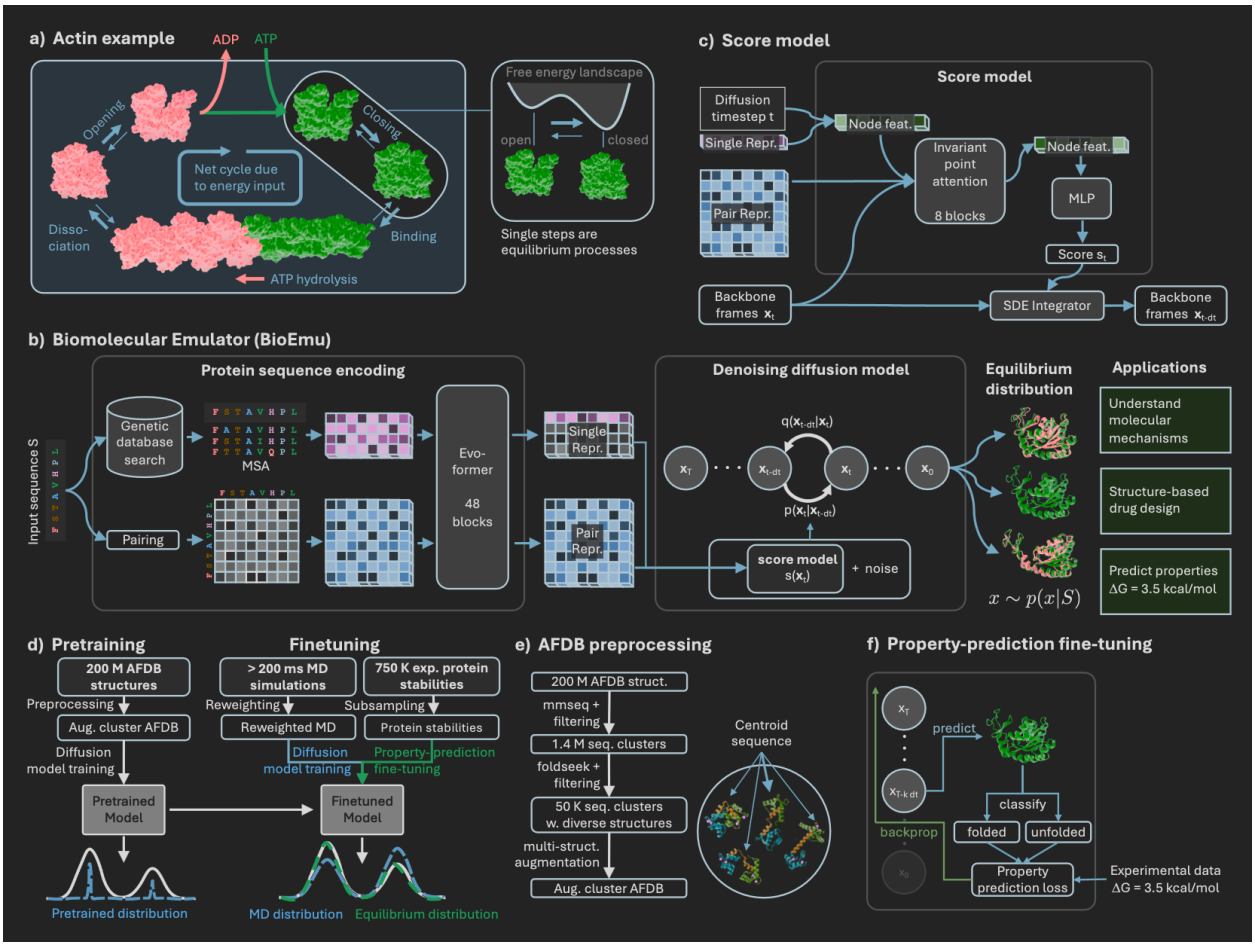


Figure 4

prepared protein system (protein, hydrogens, retained cofactors or ions, and optionally a restrained first hydration shell) fits within roughly $8000 - 10000$ atoms, run whole-protein UMA-MD and derive calibrated thermostability estimates from global folded-versus-disrupted basin statistics. We do not include seed structures above this rough threshold, focusing on monomers between $20 - 400$ residues, and dimers between $40 - 800$ residues, and therefore we do not worry with any sort of local or hybrid methods.

2.5 Why BioEmu is the correct fast dynamics oracle, especially above the whole-protein UMA paradigm

BioEmu is a sequence-conditioned generative system for sampling approximate protein equilibrium ensembles [16]. In the BioEmu preprint, the model is trained by combining AlphaFoldDB-derived structural data, more than 200 milliseconds of all-atom MD simulations, and large-scale experimental protein stability data. The system can generate thousands of statistically independent conformational samples per hour on a single GPU and is explicitly evaluated not only on qualitative conformational diversity, but also on quantitative free-energy and stability metrics [16]. That combination makes BioEmu unusually relevant to the present design problem.

The BioEmu paper reports three properties that matter directly here. First, on a range of

conformational-change benchmarks, BioEmu samples known domain motions, local unfolding transitions, and cryptic pocket formation. Second, on proteins with extensive MD references, BioEmu approximates equilibrium landscapes with macrostate free-energy errors on the order of 1 kcal/mol; the paper reports mean absolute errors of about 0.74 kcal/mol on DESRES fast folders and about 0.91 kcal/mol on a CATH-domain benchmark [16]. Third, when fine-tuned with experimental stabilities, BioEmu predicts folding free energies with mean absolute error below 0.8 kcal/mol on Megascale-like data and qualitatively explains mutation-induced destabilization through shifts in sampled ensembles [16]. These literature results do not make BioEmu a substitute for experiment, but they do make it a compelling dynamics-based oracle.

BioEmu fills a gap between SPURS and UMA. SPURS is extremely fast and mutation-local, but it is not a dynamics model. UMA can be more physically expressive, but it is much more expensive and its whole-protein thermodynamic validity must be calibrated carefully. BioEmu provides a third option: a cheap, scalable, sequence-conditioned whole-chain ensemble model. Because BioEmu samples directly from an approximate equilibrium ensemble rather than integrating Newtonian dynamics, its cost profile is much closer to a fast inference model than to MD. This is especially attractive for larger proteins whose prepared all-atom systems exceed roughly 8000–10000 atoms or for classes where whole-protein UMA-MD is computationally practical but scientifically less well calibrated.

BioEmu also comes with important limitations, and these limitations determine how it should be used. The current model is designed for single protein chains at a fixed thermodynamic condition around 300 K; it is not trained to handle membranes, explicit ligands, or arbitrary multi-chain complexes as first-class citizens [16]. We therefore use BioEmu as a whole-chain dynamics oracle for soluble, predominantly single-chain systems or for chain-wise approximations inside larger systems. In ligand-dependent tasks, BioEmu should be treated as a somewhat weaker prior since it does not recognize ligands in the input, and should be utilized in tandem with UMA; proteins that are membrane bound, situations where annealing is needed, or multimers are not included in our training methodology. However, BioEmu is exactly the sort of fast dynamics screen that can augment SPURS and reduce overutilization of UMA.

The resulting oracle hierarchy is therefore tri-fidelity rather than bi-fidelity. SPURS supplies dense cheap mutation landscapes and explicit single/double-mutant supervision. BioEmu supplies cheap whole-chain equilibrium diagnostics. UMA supplies the highest-fidelity MLIP-based MD signal on a much smaller subset. For proteins with prepared systems above roughly 8000 atoms, BioEmu becomes the primary whole-chain dynamics oracle and UMA is used selectively as a soft-gated correction in the 8000–10000 atom range and as a more heavily trusted whole-protein branch below about 8000 atoms. That design respects both the public evidence behind each model and the present training scope, which is restricted to systems whose prepared atom counts remain below roughly 10 000. We leave it to later work to train on larger structures with UMA.

2.6 Why catalytic-oracle models are required, and how KcatNet, GraphKcat, and MMKcat differ

Thermostability and binding do not determine catalytic competence by themselves. A sequence can be highly stable yet catalytically weak because it disrupts electrostatics, substrate positioning, pocket desolvation, product handling, or the balance between turnover and binding. Catalytic design therefore needs its own oracle family. In the present manuscript, catalytic supervision is expressed through three assay-facing kinetic quantities: turnover number k_{cat}, Michaelis constant K_{M}, and

catalytic efficiency $k_{\mathrm{cat}}/K_{\mathrm{M}}$. We represent them on a logarithmic scale because the published predictors and the underlying wet-lab datasets span multiple orders of magnitude:

$$\ell_{k_{\mathrm{cat}}} = \log_{10} k_{\mathrm{cat}}, \qquad \ell_{K_M} = \log_{10} K_{\mathrm{M}}, \qquad \ell_{\mathrm{eff}} = \log_{10}(k_{\mathrm{cat}}/K_{\mathrm{M}}). \tag{1}$$

These channels are not redundant. Larger k_{cat} is desirable when one cares about maximal catalytic throughput, smaller K_{M} is desirable when one cares about substrate affinity under Michaelis-Menten kinetics, and larger $k_{\mathrm{cat}}/K_{\mathrm{M}}$ is desirable when one cares about catalytic efficiency in the low-substrate regime.

KcatNet is the natural wide catalytic screen in the current stack [19]. It takes an enzyme sequence together with a substrate SMILES string and maps them to a turnover prediction using a geometric deep learning architecture that combines ProtT5 and ESM protein embeddings with molecular graph features and cross-graph interaction layers. This gives KcatNet an attractive operating point: it is richer than a plain sequence-only regressor, but much cheaper than a pocket-aware structure-conditioned pipeline. In the present methodology, KcatNet is therefore the catalytic analogue of a wide first-pass screen: it can annotate a large candidate pool quickly, provided substrate metadata are available.

GraphKcat occupies a different point on the catalytic cost-accuracy frontier [17]. It is explicitly pocket informed. Given a protein structure and a ligand pose or ligand-derived pocket, it extracts an 8 Å catalytic neighborhood, embeds the ligand with UniMol, embeds the protein with ESM2, and reasons over a hierarchical graph that couples enzyme-pocket geometry and ligand context. This extra structure pays for three things the present manuscript needs. First, GraphKcat predicts not only turnover but also K_{M} and catalytic efficiency. Second, it provides a more defensible signal when the mutational mechanism is expected to be pocket local rather than purely sequence global. Third, it gives the methodology a structure-aware catalytic oracle that is directly comparable in spirit to the structure-aware stability and binding branches.

MMKcat fills the complementary missing-modality niche [29]. Its key idea is that catalytic prediction is multimodal but the available modalities are often incomplete. The model treats substrates and enzyme sequence as essential inputs and allows other channels, notably products and structure-derived graphs, to be masked. In the present repository, MMKcat is run over a set of mask patterns and the resulting predictions are aggregated into a mean and a mask-disagreement standard deviation. Methodologically, this is valuable for two reasons. First, it preserves catalytic supervision on examples where product annotation is incomplete or structure materialization is noisy. Second, it exposes a native uncertainty signal that is semantically meaningful: disagreement across modality masks measures sensitivity to missing information rather than only checkpoint noise.

The three catalytic models should therefore not be collapsed into a single generic "Kcat oracle." They answer different questions. KcatNet is the wide enzyme-substrate turnover screen. MMKcat is the robust multimodal turnover oracle when products or structure channels are incomplete. GraphKcat is the narrower pocket-aware structural oracle that additionally predicts K_{M} and $k_{\mathrm{cat}}/K_{\mathrm{M}}$. In the methodology developed later in the manuscript, catalytic routing follows exactly this logic: wide KcatNet or MMKcat screening first, then narrower GraphKcat refinement when structure and ligand geometry are available, followed by explicit multi-oracle fusion rather than blind trust in any single predictor.

However, predictive catalytic models are not enough by themselves when the repository already contains docked reactant-bound and product-bound complexes for each reaction. In that setting the physically richer catalytic question is not only "what turnover would a supervised regressor

predict," but also "does the full enzyme actually preorganize the reactants, and how difficult is it to move between the reactant-like and product-like bound basins under a realistic atomistic potential?" This is the motivation for the new catalytic UMA branch implemented in the repository. It uses whole-enzyme equilibrium UMA dynamics to estimate generalized near-attack or productive-pose populations, then uses forward and reverse steered UMA dynamics between the docked reactant and product endpoint complexes to reveal likely reaction-pathway geometry, harvest near-transition-state-like structures, and initialize optional path umbrella PMFs. In other words, KcatNet, MMKcat, and GraphKcat remain important learned catalytic predictors, but the default catalytic RL path is no longer purely predictive. It is predictive *and* dynamical.

3 Background on GFlowNets for variable-order sequence editing

3.1 Core objective

A GFlowNet defines a stochastic policy over construction trajectories in a directed acyclic graph (DAG). Let s_0 denote the source state and x a terminal state. If training converges ideally, the induced terminal distribution satisfies

$$p_\theta(x) \propto R(x), \tag{2}$$

where $R(x) > 0$ is a nonnegative reward. The key insight is that diverse high-reward solutions remain present in the sampling distribution instead of being suppressed by a maximization objective [2]. This is especially attractive when the reward is noisy or multi-modal.

The original flow-matching objective and the later detailed-balance objective are local, but trajectory balance (TB) often propagates terminal reward information more efficiently over long trajectories [2, 22]. Given a trajectory $\tau = (s_0 \to s_1 \to \cdots \to s_T = x)$, TB minimizes the squared residual

$$\mathcal{L}_{\mathrm{TB}}(\tau) = \left(\log Z_\theta(s_0) + \sum_{t=0}^{T-1} \log PF(s_{t+1} \mid s_t) - \log R(x) - \sum_{t=0}^{T-1} \log P_{\mathrm{B}}(s_t \mid s_{t+1}) \right)^2, \tag{3}$$

where $Z_\theta(s_0)$ is the learned partition function at the root. TB enjoys stronger long-range credit assignment than detailed balance in many settings [22]. For longer or noisier trajectories, subtrajectory balance SubTB(λ) interpolates between local and global credit assignment and can be more stable in practice [21]. In this paper we therefore recommend SubTB(λ) for the primary optimization loop and TB for validation diagnostics.

3.2 Why variable-order protein editing forms a natural GFlowNet DAG

A seed-centered protein editing problem begins from a seed sequence and repeatedly applies mutations until a stop action is chosen or no editable positions remain. The terminal object is therefore a variable-order multi-mutant variant whose Hamming distance can range from 1 up to L. This naturally forms a DAG if edits are prevented from cycling and each position is edited at most once in the default action space. The graph is compositional: each action appends one mutation to a partial edit set, and long trajectories simply correspond to higher-order mutants. The terminal reward can be defined by any differentiable or non-differentiable oracle, making GFlowNets suitable for black-box stability optimization. Moreover, several existing GFlowNet extensions are directly relevant:

- **Biological sequence design.** GFlowNets have already been shown to improve diversity and novelty for biological sequence generation [8].
- **Biological sequence editing.** Seed-centered editing with bounded edit counts has been formalized in GFNSeqEditor, which is particularly close to the present protein editing setup [6]. We use the same edit-trajectory viewpoint but remove the hard mutation-count cap, replacing it with soft regularization and mutation-order curricula.
- **Multi-objective and conditional GFlowNets.** Reward-conditional or preference-conditional variants can represent a family of scalarized objectives rather than a single one [9**?**]. This is relevant because thermostability optimization usually trades off against structural plausibility, oracle uncertainty, and sometimes mutation radius, even though the default protocol in this paper does not hard-cap mutation radius.
- **Temperature-conditional GFlowNets.** Conditioning the policy on reward temperature β allows the same model to span exploratory and exploitative paradigms, with $p(x \mid \beta) \propto R(x)^{\beta}$ [11]. We recommend this mechanism because it makes early broad exploration and later sharp exploitation a training-time choice rather than a separate algorithm.

3.3 Why we do not use a many-path formulation by default

If edits can be applied in any order, the same terminal mutant can be reached by many action permutations. GFlowNets can in principle handle such many-to-one trajectory mappings, but doing so increases variance in the credit assignment problem. For the methodology proposed here, we choose a simpler canonical edit ordering: positions may only be edited in increasing residue index order in the default edit-once action space. This does *not* restrict the reachable set of terminal mutants; it only assigns each edited position set a unique construction path. The resulting DAG is easier to train, yields a deterministic backward policy, and reduces the complexity of debugging the reward pipeline. A non-canonical many-path variant, or a later extension with bundled multi-site macro-actions, is still valuable as an ablation because it could improve exploratory flexibility for very high-order mutants, but it is not the recommended starting point.

4 Problem formulation, notation, and target quantities

4.1 Objects, units, and sign conventions

Let $B \in \mathbb{R}^{L\times 4\times 3}$ denote a fixed backbone with L residues and backbone atoms (N, C_α, C, O) expressed in Å. Let $s \in \mathcal{A}^L$ denote an amino-acid sequence over the canonical alphabet $\mathcal{A}$ of size 20. Let X^{sc} denote the packed sidechain coordinates associated with the current sequence state. In the ADFLIP-centered formulation these sidechains are produced by the native discrete-flow sampling loop together with PIPPACK-based sidechain packing [**?** **?**]. The base ADFLIP model provides a seed sequence $s^{(0)}$ and seed sidechains $X^{\mathrm{sc},(0)}$ for each backbone or bound complex.

Stability prediction uses multiple sign conventions across the literature. We therefore define the quantities used in this manuscript explicitly in Table 1. The most important choices are that SPURS-style mutation effects are handled in the destabilization convention, BioEmu-derived equilibrium proxies are expressed as calibrated folded-basin stability scores at the model's reference condition, and the UMA branch is converted into an absolute stability convention where larger positive values

mean a more stable folded basin. Rewards are always constructed only after standardization, calibration, and sign harmonization, so downstream optimization does not inherit the ambiguity.

Table 1: Core quantities, units, and optimization direction. "Better" refers to the direction favored by the reward.

Quantity	Definition	Unit	Better
$\widehat{\Delta\Delta G}^{\text{SPURS}}_{\text{destab}}$	SPURS-predicted mutation destabilization relative to a reference sequence on a fixed backbone. We retain the predictor's natural convention in which more negative values indicate stabilizing mutations.	kcal/mol	More negative
$\widehat{\Delta G}^{\text{BioEmu}}_{\text{stab}}$	BioEmu-derived and calibrated whole-chain stability proxy obtained from equilibrium-ensemble samples. Larger positive values imply greater folded-basin occupancy relative to disrupted states at the model's reference condition.	kcal/mol	Larger
$\widehat{\Delta G}^{\text{UMA}}_{\text{stab}}$	UMA-MD-derived or calibrated absolute stability of the folded basin. In this manuscript, larger positive values imply a more stable folded basin relative to the unfolded or disrupted basin.	kcal/mol	Larger
$\widehat{\ell}_{k_{\text{cat}}}$	Predicted logarithmic turnover number, typically $\widehat{\ell}_{k_{\text{cat}}} = \widehat{\log_{10} k_{\text{cat}}}$, from KcatNet, MMKcat, GraphKcat, or a fused catalytic predictor.	$\log_{10}(\text{s}^{-1})$	Larger
$\widehat{\ell}_{K_M}$	Predicted logarithmic Michaelis constant, typically $\widehat{\ell}_{K_M} = \widehat{\log_{10} K_M}$, from GraphKcat or a fused catalytic predictor.	$\log_{10}(\text{M})$	Smaller
$\widehat{\ell}_{\text{eff}}$	Predicted logarithmic catalytic efficiency, typically $\widehat{\ell}_{\text{eff}} = \log_{10}\widehat{(k_{\text{cat}}/K_M)}$.	$\log_{10}(\text{M}^{-1}\text{s}^{-1})$	Larger
u_{pack}	Packing-quality proxy formed from ADFLIP terminal entropy or purity, heavy-atom clash score, and sidechain packing disagreement across repeated packs or structural states.	unitless	Smaller
scTM	AlphaFold/ColabFold self-consistency TM-score between predicted and target backbone.	unitless	Larger
scRMSD	Self-consistency backbone RMSD.	Å	Smaller
K	Number of mutated positions relative to the seed. In the default protocol it is logged and optionally regularized, but not hard-capped.	count	Task-dependent; larger values are allowed when rewarded

4.2 Computational targets and experimental readouts

The training loop in this manuscript uses five distinct classes of quantities, and it is important not to conflate them. First are the cheap mutation-level scores, which include SPURS mutation-effect estimates, ADFLIP sidechain confidence, structure plausibility filters, and novelty or mutation-order terms used only to regularize the search. Second are the fast whole-chain dynamics summaries produced by BioEmu. Third are the higher-fidelity MD-derived quantities produced by the UMA branch from whole-protein simulations up to the default atom budget. Fourth are catalytic-kinetic predictions from KcatNet, MMKcat, and GraphKcat. Fifth are the laboratory readouts that would ultimately be used to validate the method, such as experimental $\Delta\Delta G$, ΔT_m, soluble yield, aggregation onset temperature, Michaelis-Menten assays of k_{cat} and K_{M}, and retained activity after heat challenge. Table 2 makes those distinctions explicit.

The role of the cheap branch is to guide exploration. The role of the BioEmu branch is to add scalable whole-chain dynamics information, especially on larger proteins. The role of the UMA branch is to provide a more physically informed stability signal and to reduce exploitation of weaknesses in the cheaper branches, with the goal of aggressive de-risking before wet-lab validation.

Table 2: Training targets, calibration targets, and downstream experimental readouts used throughout the paper.

Role in the pipeline	Representative quantities	Interpretation
Cheap training signals	SPURS point-mutation and multi-mutation scores; ADFLIP u_{pack}; self-consistency filters; mutation-order and novelty regularizers	Fast search guidance and local ranking. Useful everywhere, but still subject to model bias and extrapolation error.
Whole-chain dynamics targets	BioEmu folded-basin occupancy, equilibrium-derived stability proxies, local-unfolding scores, hinge or pocket opening indicators	Fast scalable dynamics signal that is especially useful on proteins whose prepared all-atom systems are larger than the direct whole-protein UMA setting.
High-fidelity computational targets	Whole-protein UMA-MD features, calibrated UMA stability scores, or fallback local and hybrid MD summaries	Higher-cost, more physically informed stability signal used for reward correction, selection, and calibration.
Catalytic-kinetic targets	KcatNet and MMKcat turnover predictions; GraphKcat predictions of $\log_{10} k_{\text{cat}}$, $\log_{10} K_{\text{M}}$, and $\log_{10}(k_{\text{cat}}/K_{\text{M}})$; fused catalytic-oracle scores	Reaction-conditioned supervision for enzyme design. These channels depend on substrate identity and may also depend on products, assay metadata, and pocket geometry.
Downstream laboratory readouts	Differential scanning fluorimetry (ΔT_m), chemical denaturation curves ($\Delta\Delta G$), Michaelis-Menten estimates of k_{cat} and K_{M}, catalytic efficiency, soluble yield after expression, aggregation onset, retained activity after heat challenge	External validation and eventual scientific claim. These readouts are not assumed during training; they are the quantities the eventual study should test.

4.3 Complexes, components, and target properties

The monomeric notation above is sufficient for thermostability-only design, but it is not sufficient for the full intended scope of this manuscript. We therefore define a generic design object

$$\mathcal{C} = \big(\mathcal{P}_{\text{des}}, \mathcal{P}_{\text{ctx}}, \mathcal{L}_{\text{ctx}}, B, \mathfrak{D}\big), \tag{4}$$

where $\mathcal{P}_{\text{des}}$ are designable protein chains or residues, $\mathcal{P}_{\text{ctx}}$ are non-designable protein partners used only as structural context, $\mathcal{L}_{\text{ctx}}$ are non-protein context atoms such as small molecules, fragments, cofactors, ions, or retained waters, and $\mathfrak{D}$ is a component-decomposition map. The decomposition map specifies which bound object is scored as the full complex and which separated objects are scored as the components. In a protein-protein interaction (PPI) or multimer problem, the separated components are typically the individual partner chains or subcomplexes. In a protein-ligand problem, the separated components are typically the apo protein and the free ligand or fragment.

For any oracle O that supports the relevant example type, let $\widehat{G}^{O}_{\mathrm{cx}}(x)$ denote the calibrated free-energy-like quantity predicted for the bound complex built from candidate sequence x, and let $\widehat{G}^{O}_{u}(x)$ denote the analogous quantity for separated component u. Then the manuscript defines binding free energy by explicit complex-versus-component translation:

$$\widehat{\Delta G_{\mathrm{bind}}}^{O}(x) = \widehat{G}^{O}_{\mathrm{cx}}(x) - \sum_{u=1}^{U} \nu_u \widehat{G}^{O}_{u}(x), \tag{5}$$

where ν_u is the stoichiometric multiplicity of component u in the complex. Relative binding effects are then

$$\widehat{\Delta\Delta G_{\mathrm{bind}}}^{O}(x; x^{(0)}) = \widehat{\Delta G_{\mathrm{bind}}}^{O}(x) - \widehat{\Delta G_{\mathrm{bind}}}^{O}(x^{(0)}). \tag{6}$$

For ligand systems with fixed ligand identity, the free-ligand term is constant across sequence edits and therefore cancels in Eq. (6); it must still be retained when reporting absolute $\widehat{\Delta G_{\mathrm{bind}}}^{O}$ values.

The methodology also introduces a target vector

$$c = \{(m, y^{\star}_{m}, \epsilon_m, w_m) : m \in \mathcal{M}_{\mathrm{target}}\}, \tag{7}$$

where each property channel m has a desired target value $y^{\star}_{m}$, an acceptable tolerance ϵ_m, and a weight w_m. Representative channels are calibrated thermostability $\widehat{T_{\mathrm{m}}}$, monomeric $\widehat{\Delta G}^{\mathrm{stab}}$, binding free energy $\widehat{\Delta G_{\mathrm{bind}}}$, relative binding effect $\widehat{\Delta\Delta G_{\mathrm{bind}}}$, and optional soft constraints such as maximum mutation order or minimum component stability. The target vector is used both during training and during bounded-retry inference.

4.4 Reaction-conditioned catalytic properties

Catalytic design requires more metadata than stability-only design. We therefore define a reaction context

$$\mathcal{R} = (\mathcal{S}_{\mathrm{sub}}, \mathcal{S}_{\mathrm{prod}}, \eta), \tag{8}$$

where $\mathcal{S}_{\mathrm{sub}}$ denotes substrate identity, usually represented by one or more SMILES strings or a bound ligand graph; $\mathcal{S}_{\mathrm{prod}}$ denotes optional product identities; and η denotes assay-context metadata such as pH, temperature, organism, and any task-level categorical tags. For a candidate sequence x and catalytic oracle O, we then write

$$\widehat{y}^{O}_{\mathrm{cat}}(x, \mathcal{R}) \in \left\{ \widehat{\ell}^{O}_{k_{\mathrm{cat}}}, \widehat{\ell}^{O}_{K_M}, \widehat{\ell}^{O}_{\mathrm{eff}} \right\}. \tag{9}$$

KcatNet and MMKcat primarily expose $\widehat{\ell}_{k_{\mathrm{cat}}}$, while GraphKcat exposes all three channels. The catalytic target vector can therefore include requests such as

$$c_{\mathrm{cat}} = \left\{ (\ell_{k_{\mathrm{cat}}}, \ell^{\star}_{k_{\mathrm{cat}}}, \epsilon_{k_{\mathrm{cat}}}, w_{k_{\mathrm{cat}}}), (\ell_{K_M}, \ell^{\star}_{K_M}, \epsilon_{K_M}, w_{K_M}), (\ell_{\mathrm{eff}}, \ell^{\star}_{\mathrm{eff}}, \epsilon_{\mathrm{eff}}, w_{\mathrm{eff}}) \right\}, \tag{10}$$

possibly together with thermostability and binding targets. This explicit separation is important because catalytic turnover, substrate affinity, and catalytic efficiency can move in different directions under mutation.

4.5 State space and edit actions

We define the GFlowNet state at step t as

$$\xi_t = (B, s_t, X_t^{\text{sc}}, M_t, i_t^{\max}), \tag{11}$$

where s_t is the current sequence, X_t^{sc} the current full-atom sidechains produced or updated by ADFLIP, $M_t \subseteq \{1, \ldots, L\}$ the set of edited positions, and $i_t^{\max}$ the largest position edited so far in the canonical ordering. The canonical ordering constraint requires future edit positions j to satisfy $j > i_t^{\max}$ in the default edit-once action space. A forward action is either

$$a_t = (j, a), \qquad j > i_t^{\max},\ a \in \mathcal{A} \setminus \{s_t(j)\}, \tag{12}$$

or a distinguished stop action $a_t = \texttt{STOP}$. The transition mutates residue j to amino-acid a and invokes local sidechain repacking to obtain X_{t+1}^{sc}.

The terminal state is reached when the stop action is taken or when no allowable edit positions remain. We do *not* impose a scientific hard cap on Hamming distance from the seed in the default protocol. Under the edit-once action space the natural maximum is simply L. For numerical safety we impose only a rollout horizon equal to the number of still-editable positions as an the primary method with the ablation experiments given by edit-revision actions, providing a natural and practical safety horizon of $2L$ in such revision-edit cases, which is sufficient to prevent pathological loops without redefining the scientific objective. Mutation order

$$K(x) = |M(x)| \tag{13}$$

is tracked for every terminal candidate and enters the reward only through optional soft regularization or curriculum mechanisms, never as a hard truncation criterion in the primary protocol.

Mutation-order curriculum rather than mutation-order cap. Allowing large K is not enough; one must also ensure that training actually visits such states. We therefore recommend a mutation-order curriculum that affects the stop action rather than the reachable state space. At the start of each rollout, sample a minimum-order target $K_{\min}$ from a broad mixture such as

$$K_{\min} \sim 0.30\,\delta_1 + 0.35\,\text{Uniform}\{2,3,4\} + 0.20\,\text{Uniform}\{5,\ldots,8\} + 0.15\,\text{Uniform}\{9,\ldots,\min(L,16)\}. \tag{14}$$

The stop action is masked until $K_{\min}$ edits have been made; after that point it becomes available, and the trajectory may stop immediately or continue to any larger Hamming distance. This simple device materially increases the share of multi-mutant trajectories in training while preserving the ability to discover both low-order and high-order solutions.

Optional bundled macro-actions. The primary implementation uses single-site actions because they keep the DAG simple and align naturally with SPURS's dense single-step mutation landscape. However, for very long redesign trajectories one may optionally add bundled macro-actions that apply 2 to 4 non-overlapping edits in one forward step. We do not make this the default because it enlarges the action space and complicates the backward policy, but it is a valuable ablation once the basic single-edit system is stable.

4.6 Target distribution

For a fixed backbone B, let x denote a terminal edited sequence together with its packed sidechains. The GFlowNet target is

$$p_\theta(x \mid B, \beta, w) \propto R_w(x \mid B)^\beta, \tag{15}$$

where R_w is a scalarized reward and $\beta > 0$ is a temperature parameter. Conditioning on w and β is not strictly required, but it is recommended because it turns one model into a family of samplers that can interpolate between exploration and exploitation and between different objective trade-offs [9, 11]. In the simplest setting used in our default protocols, w is static and only β is varied during training.

5 Parameterizing the edit GFlowNet with ADFLIP

5.1 Seed generation

For each target object we assume a starting structure and starting sequence are provided. In the revised methodology these starting examples are themselves design inputs rather than a mutation dataset in the classical sense: during training we treat them as de novo proteins or complexes with total protein length sampled from a broad uniform distribution over roughly 20–400 residues, together with their associated bound or unbound structural context. Before any redesign occurs, we evaluate the starting sequence or complex with all admissible oracles and cache a baseline property vector

$$y^{\text{base}} = \big(\Delta G^{\text{base}}_{\text{fold}},\, T_{\text{m}}^{\text{base}},\, \Delta G^{\text{base}}_{\text{bind}},\, \text{other active task properties}\big). \tag{16}$$

This baseline is used in two ways. First, it allows every later candidate to receive absolute and relative labels such as $\Delta\Delta G$, ΔT_{m}, or $\Delta\Delta G_{\text{bind}}$ with respect to the original undesigned input. Second, it allows the very first full-redesign step to be conditioned on the same target property vector that will later be enforced at inference time.

The initial generator pass is a *full redesign* by ADFLIP, not merely a repair of the starting sequence. Given target-conditioning vector c (which may encode absolute targets, relative improvements over y^{base}, or both), we sample a seed redesign $s^{(0)}$ and packed sidechains $X^{\text{sc},(0)}$ using a pretrained ADFLIP checkpoint. When multiple structural states are available, for example from NMR ensembles, BioEmu snapshots, or short UMA/BioEmu pre-equilibration of the starting object, we use ADFLIP's native ensemble mode and average denoiser predictions across states before committing tokens [**?**]. When only a single state is available, the same checkpoint is run in single-structure mode. The default seed pass uses adaptive purity sampling rather than a fixed left-to-right order: high-confidence positions are committed first, PIPPACK sidechains are packed, and the process continues until no masked positions remain [**? ?**]. For the first implementation we recommend a purity threshold $\tau_{\text{seed}} = 0.9$, a maximum of 32 Euler steps, and one final full-sequence PIPPACK pass, and BioEmu and UMA sampling for initial ensembles.

The full-redesign seed is itself scored. We therefore distinguish three objects throughout training: the starting design input x^{base}, the first ADFLIP redesign $x^{(0)}$, and any edited descendants $x^{(1)}, x^{(2)}, \ldots$. Oracle scores are recorded for all three. This is important because the full-redesign jump may already satisfy or nearly satisfy a target such as "attain or exceed $T_{\text{m}}^\star$" or "improve ΔG_{bind} while preserving foldability," and training should give the policy credit for that possibility instead of only rewarding later edit steps.

The seed is not assumed to be optimal. It is only assumed to be a plausible point inside the pretrained generator's support and compatible with the target structure or structural ensemble. Starting from a full-redesign seed shortens the action horizon relative to unconditional sequence generation while still allowing the GFlowNet to explore variable-order multi-mutant trajectories. In practice, one should store the starting-sequence properties, the seed-sequence properties, the final ADFLIP per-position purity values, the denoiser entropies, and a cheap packing-quality proxy

$$u_{\text{pack}}(x) = \alpha_H \, \overline{H}_{\text{AD}}(x) + \alpha_C \, C_{\text{clash}}(x) + \alpha_D \, D_{\text{pack}}(x), \tag{17}$$

where $\overline{H}_{\text{AD}}$ is mean terminal categorical entropy, C_{clash} is a normalized heavy-atom clash count after sidechain packing, and D_{pack} is a packing-disagreement statistic across repeated packs or across available structural states. Smaller u_{pack} is better.

5.2 Encoder and trainable parameters

Let $h_t \in \mathbb{R}^{L \times d}$ denote the residue embeddings from ADFLIP's multi-scale denoiser applied to state ξ_t, and let a_t denote the corresponding atom embeddings. The denoiser consumes the current partially edited sequence s_t, the available sidechain context, the protein backbone, the non-protein atoms, and the discrete-flow time parameter t [**?**]. In the *primary* training methodology we full-fine-tune the trainable ADFLIP denoiser blocks that act on these representations: atom encoder projections, local atom-attention blocks, atom-to-residue and residue-to-atom exchange blocks, residue GNN blocks, and the decoder projections. The frozen components are the sidechain packer, the oracle stack (SPURS, BioEmu, UMA), and the optional reference copy of ADFLIP used for anchoring losses.

On top of the residue embeddings we introduce three lightweight heads:

1. a stop head g_{stop} producing a scalar logit,
2. a position head g_{pos} producing logits over currently allowable edit positions,
3. an amino-acid head g_{aa} producing logits over 19 non-current amino acids for the selected position.

The forward policy is factorized as

$$PF(a_t \mid \xi_t) = PF(\texttt{STOP} \mid \xi_t) \text{ or } PF(j \mid \xi_t, \neg\texttt{STOP}) \, PF(a \mid j, \xi_t, \neg\texttt{STOP}). \tag{18}$$

This factorization improves numerical stability and enables position-level masking for canonical ordering, conservation masks, symmetry tying, and user-defined hard constraints.

A practical design choice is that the GFlowNet still acts in edit space even though ADFLIP itself is a full sequence generator. We keep that choice because variable-order edit DAGs make credit assignment and mutation-order control easier. In other words, ADFLIP serves simultaneously as a seed generator, a state encoder, and a full-context repair operator, while the GFlowNet provides reward-proportional exploration over edit trajectories.

Parameter-efficient fine-tuning remains worthwhile as an *ablation*, not the default. Specifically, we recommend a memory-constrained comparison in which the released ADFLIP checkpoint is mostly frozen and low-rank or adapter-style updates are inserted into the same modules listed above. LoRA or adapter tuning is therefore part of the ablation matrix, but all mainline algorithms, schedules, and hyperparameter defaults in this revised manuscript assume full fine-tuning of the non-frozen ADFLIP components.

5.3 Using SPURS primarily as an external oracle; SPURS-guided proposals only as an ablation

SPURS is not injected into the proposal path by default. The main policy in the primary ADFLIP workflow is computed from ADFLIP embeddings and the target-conditioning channels alone. This choice is intentional. ADFLIP is already trained as a ligand-aware, all-atom generative model and is small enough to support large-batch backpropagation. The most direct route to a strong policy is therefore to fine-tune ADFLIP itself against the fused reward, rather than to delegate local action selection to an external stability regressor.

SPURS is still evaluated densely when this is computationally useful, but its outputs are consumed as *oracle information*: reward terms, acquisition priorities, cached diagnostics, and optional hindsight labels. In the primary method, the position and amino-acid heads do not concatenate SPURS summaries. This avoids a failure mode where the policy becomes over-attached to a single cheap branch and loses the ability to exploit ADFLIP's own all-atom and ligand-aware context modeling.

A scientifically sensible ablation is nevertheless to introduce SPURS-guided proposals in one of two limited ways. The first is *proposal shortlisting*: use SPURS to define a top-k candidate set of mutations per state and sample only within that set. The second is *feature augmentation*: append a small number of SPURS summaries, such as $\min_a \delta_t^{\mathrm{S}}(j, a)$ or the entropy of the per-position mutation distribution, to the ADFLIP residue embeddings before the position head. Both ablations should be compared against the primary full-fine-tuning baseline in terms of convergence speed, diversity, and sensitivity to oracle hacking.

The overall philosophy is therefore simple. ADFLIP proposes. SPURS scores. BioEmu and UMA refine. SPURS may optionally bias proposals in a controlled ablation, but it is not the default controller of the generative trajectory.

5.4 Terminal scoring for variable-order multi-mutants

A critical distinction in the revised methodology is that *terminal* objects may be variable-order multi-mutant variants even though the primary cheap supervision is deliberately conservative. We do not replace SPURS's native global architecture by a hand-built local block factorization. Instead, we use the full-chain SPURS model in the mutation orders where its intended use is strongest, and we shift higher-order reliance to BioEmu and UMA.

Let x denote a terminal candidate relative to reference sequence s_{ref}, where the reference is the starting undesigned sequence for the training example unless otherwise noted. Let $M(x)$ be the edited position set and $K(x) = |M(x)|$ its Hamming distance from the reference. The primary SPURS terminal score is

$$\widehat{\Delta\Delta G}^{\,\mathrm{SPURS,primary}}_{\mathrm{destab}}(x \mid B, s_{\mathrm{ref}}) = \begin{cases} \widehat{\Delta\Delta G}^{\,\mathrm{SPURS}}_{\mathrm{single}}(x \mid B, s_{\mathrm{ref}}), & K(x) = 1, \\ \widehat{\Delta\Delta G}^{\,\mathrm{SPURS}}_{\mathrm{double}}(x \mid B, s_{\mathrm{ref}}), & K(x) = 2, \\ \widehat{\Delta\Delta G}^{\,\mathrm{SPURS}}_{\mathrm{tuple}}\big(i \mid B, s_{\mathrm{ref}}\big), & K(x) \geq 3, \end{cases} \tag{19}$$

where the single-mutant and double-mutant quantities are obtained from SPURS on the *full sequence and full structure context*. The $K(x) \geq 3$ line is done with the explicit higher-order SPURS epistasis, and thus should be down-weighted with respect to point, and double mutants, with more emphasis placed on BioEmu and UMA.

This leads directly to the training curriculum. The primary method includes three mutation-order settings:

1. **Full-redesign and single-mutation setting.** The model learns to score the initial full ADFLIP redesign and subsequent single-edit trajectories relative to the starting sequence or complex.
2. **Explicit epistatic double-mutant setting.** A second training stage emphasizes terminal candidates with $K(x) = 2$ and uses SPURS's full-chain double-mutant decoder as the cheap epistatic label. This is the primary mechanism by which the policy learns non-additive interactions from the cheap branch.
3. **Higher-order setting.** Candidates with $K(x) \geq 3$ are fully allowed in the search space, but their cheap supervision is treated conservatively and they are preferentially escalated to BioEmu and UMA.

The key scientific point is that we are not claiming SPURS cannot model global dependencies. The opposite is true: SPURS already carries a full-protein representation. The conservative choice is instead about *training validity*. The primary methodology uses the parts of SPURS that are easiest to defend empirically—single mutants and double-mutant epistasis—while relying on BioEmu and UMA to absorb more of the burden for higher-order designs, with SPURS's own higher-order decoder applied directly to the full-chain mutant but with a mild down-weighting accompanied by BioEmu and UMA up-weighting.

In practice, we log three quantities for every terminal candidate:

$$K(x), \qquad \widehat{\Delta\Delta G}^{\,\text{SPURS,primary}}_{\text{destab}}(x), \qquad \mathbf{1}[K(x) = 2]. \tag{20}$$

The first measures redesign extent, the second is the primary cheap oracle score, and the third identifies the explicit epistatic-double setting. These quantities drive replay stratification, acquisition to BioEmu and UMA, and the stage transitions of the curriculum.

5.5 Replay stratification by mutation order and epistasis order

If one merely removes the hard edit cap, the policy can still collapse to mostly trivial variants because those trajectories are easier to sample and to fit. We therefore recommend replay stratification across four bins that match the primary supervision setting:

1. the starting example and first full-redesign candidate,
2. single mutants ($K = 1$),
3. explicit epistatic doubles ($K = 2$),
4. higher-order candidates ($K \geq 3$).

Minibatches for the GFlowNet loss are sampled approximately uniformly across non-empty bins, with the $K = 2$ bin up-weighted during the second stage of the curriculum so that explicit double-mutant epistasis is actually learned rather than appearing only sparsely in the replay distribution.

We still record coarse notions of interaction complexity, but we do not define those notions through local block decomposition inside the cheap branch. Instead, the primary diagnostics are the total

Hamming distance $K(x)$, whether a candidate belongs to the explicit double-mutant stage, and whether a candidate with $K(x) \geq 3$ has received heavier BioEmu and/or UMA supervision. This keeps the replay logic aligned with the revised oracle philosophy: global SPURS where supported, explicit double-mutant supervision in the primary method, and higher-order escalation rather than arbitrary local partitioning.

5.6 Backward policy

Under canonical edit ordering, the backward policy may be deterministic: the parent of a non-root state is obtained by removing the last-applied edit. In that case the backward log-probability terms in Eq. (3) are zero. We nevertheless recommend implementing an explicit backward policy for two reasons. First, it makes the code path compatible with later experiments using non-canonical edit ordering. Second, it enables richer diagnostics: if a learned backward head fails to concentrate on the true parent under canonical ordering, this often reveals a bug in the state encoding or transition function.

5.7 Preserving the pretrained generator during RL fine-tuning

A frequent failure mode in RL fine-tuning is catastrophic drift away from the original generator. We therefore recommend four regularizers. They are active by default in Simul-MF and Seq-Curr, and they also regularize the teacher and student components of OneShot-AL whenever those components are trainable:

$$\mathcal{L}_{\text{anchor}} = \mathbb{E}_{(x,t,s_t)}\Big[\mathrm{KL}\big(p_{\theta_0}(\cdot \mid s_t, x, t) \,\|\, p_\theta(\cdot \mid s_t, x, t)\big)\Big], \tag{21}$$

$$\mathcal{L}_{\text{distill}} = -\mathbb{E}_{x\sim\mathcal{B}}\Big[w(x) \sum_{j\in M(x)} \log p_\theta\big(s_x(j) \mid s_t, x, t\big)\Big], \tag{22}$$

$$\mathcal{L}_{\text{denoise}} = -\mathbb{E}_{(x,t,s_t)}\Big[\log p_\theta\big(s_1 \mid s_t, x, t\big)\Big], \tag{23}$$

$$\mathcal{L}_{\text{pack}} = \mathbb{E}_{x\sim\mathcal{B}}\Big[\ell_{\text{clash}}(x) + \ell_{\text{pack-cons}}(x)\Big]. \tag{24}$$

Here θ_0 are the pretrained ADFLIP parameters, s_t is drawn from the discrete flow corruption process, $\mathcal{B}$ is a replay buffer of high-reward edited sequences, and $w(x)$ is a reward-dependent importance weight. The first term anchors the model to its original denoising distribution on generic masked-sequence examples. The second distills good edited sequences back into the model's native residue predictor. The third preserves the base discrete-flow inverse-folding competence. The fourth penalizes sidechain clashes and unstable packing on replayed candidates; if PIPPACK is frozen, this term only updates the policy through the chosen edits and not the packer itself.

The total training objective in both methods is therefore

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{SubTB}(\lambda)} + \lambda_{\text{anchor}}\mathcal{L}_{\text{anchor}} + \lambda_{\text{distill}}\mathcal{L}_{\text{distill}} + \lambda_{\text{den}}\mathcal{L}_{\text{denoise}} + \lambda_{\text{pack}}\mathcal{L}_{\text{pack}}. \tag{25}$$

Recommended defaults are $\lambda_{\text{anchor}} = 0.1$, $\lambda_{\text{distill}} = 0.2$, $\lambda_{\text{den}} = 0.05$, and $\lambda_{\text{pack}} = 0.05$ initially. These weights should be increased if the generator begins to drift off support or if packing quality degrades.

5.8 Full-context structural refresh after each edit

The previous local-remasking formulation is unnecessary for ADFLIP. Because ADFLIP is comparatively small and discrete flow matching is substantially cheaper to iterate than a diffusion-style

denoiser, both inference and backpropagation can operate with *full-context refresh* rather than neighborhood-restricted remasking. After each accepted edit, or after a short burst of accepted edits if throughput needs to be improved, we rerun ADFLIP on the full current object with all current context present: entire chain, bound partners, ligands, ions, and any ensemble states being tracked. We then run a full PIPPACK repair pass on the entire sequence or on the entire designable scope.

This change has four advantages. First, it preserves global couplings that may matter for stability or binding, rather than assuming the consequences of an edit are confined to a hand-defined neighborhood. It considers allostery via the global context. Second, it matches the way ADFLIP was actually trained: full-context denoising over all-atom inputs rather than locality-restricted patch inference [**?**]. Third, it removes a source of implementation complexity and a potential mismatch between monomeric and complex settings.

During training, a fast full-context refresh can use a small number of Euler steps (for example 2–6) with warm-started tokens and sidechains, followed by a final high-quality cleanup pass before expensive oracle evaluation. During evaluation or shortlist generation, the final cleanup pass should use the full ADFLIP inference paradigm and one final PIPPACK pass. Terminal candidates whose global packing proxy u_{pack} exceeds a threshold should be down-weighted, repaired with a higher-quality full pass, or excluded from expensive evaluation. This is the ADFLIP analogue of sidechain-quality filtering that earlier formulations attempted to approximate with local remasking.

6 ADFLIP-native complexes, ensembles, and controllable property conditioning

6.1 No ligand-specific architectural retrofit is required

A major consequence of using ADFLIP as the generator substrate is that the methodology does not need to retrofit ligand or fragment conditioning into the inverse-folding backbone. ADFLIP already represents proteins together with non-protein atoms in a multi-scale graph, already uses those atoms during denoising, and already supports multi-structure conditioning [**?**]. Therefore monomers, multimers, protein-protein interfaces, protein-ligand complexes, nucleotide complexes, metal-bound systems, and NMR-like structural ensembles are all treated as native input modes of the same generator.

The practical implication is simple. Whenever a bound complex is the design object, we pass the bound complex to ADFLIP directly. Whenever a separated component is needed for thermodynamic bookkeeping, we pass that separated component to the same checkpoint. The design logic changes only at the level of the GFlowNet state, the oracle runner, and the property heads; the core all-atom inverse-folding architecture does not require a special ligand-conditioning patch.

6.2 Additional modules that are still required

Although no ligand-specific retrofit is needed, three additions remain essential for the present manuscript.

First, we add GFlowNet action heads on top of ADFLIP's residue representations, as already described in Section 5. Second, we add pooled property heads that summarize the bound complex and each separated component, because the tri-fidelity reward uses both folding and binding quantities. Third, we add explicit control tokens for target-conditioned generation, since the base

ADFLIP paper demonstrates guidance with external regressors but does not define a persistent goal-conditioned design interface for target vectors such as $T_{\mathrm{m}}^{\star}$ or $\Delta G_{\mathrm{bind}}^{\star}$.

Let g_{cx} denote an attention-pooled embedding over all design-relevant residues and nearby context atoms in the bound complex, and let g_u denote the analogous pooled embedding for separated component u. The additional property heads are

$$\widehat{y}_u^{\mathrm{stab}} = h_u^{\mathrm{stab}}(g_u, c), \tag{26}$$

$$\widehat{y}^{\mathrm{cx}} = h^{\mathrm{cx}}(g_{\mathrm{cx}}, c), \tag{27}$$

$$\widehat{y}^{\mathrm{bind,dir}} = h^{\mathrm{bind}}(g_{\mathrm{cx}}, g_1, \ldots, g_U, c), \tag{28}$$

$$\widehat{y}^{T_{\mathrm{m}}} = h^{T_{\mathrm{m}}}(g_{\mathrm{cx}}, c). \tag{29}$$

The direct binding head is never trusted in isolation. It is constrained by a decomposition-consistency loss that forces the direct binding prediction to agree with the explicit complex-versus-component translation:

$$\mathcal{L}_{\mathrm{bind-cons}} = \left\| \widehat{y}^{\mathrm{bind,dir}} - \left(\widehat{y}^{\mathrm{cx}} - \sum_{u=1}^{U} \nu_u \widehat{y}_u^{\mathrm{stab}} \right) \right\|_1 . \tag{30}$$

This loss prevents the network from reporting apparently good binding scores that are inconsistent with the underlying component energetics.

6.3 Control tokens for target-conditioned generation

The conditioning vector c is embedded once per example and injected into the GFlowNet action heads, the auxiliary property heads, and any optional ADFLIP guidance modules by additive conditioning or FiLM-style affine modulation. Each target channel contributes six pieces of information: the requested absolute value $y_m^{\star}$, the requested improvement over baseline $\Delta y_m^{\star}$ when applicable, the observed baseline value y_m^{base}, the tolerance ϵ_m, a direction flag (maximize, minimize, or hit target), and a missingness mask. This lets the same model handle unconstrained optimization, absolute-target conditioning (for example "attain $T_{\mathrm{m}} \geq T_{\mathrm{m}}^{\star}$"), relative-target conditioning (for example "improve ΔG_{bind} by at least δ"), or mixed objectives such as "increase T_{m} while keeping ΔG_{bind} below a threshold."

A practical training trick is hindsight relabeling. For a terminal candidate x with fused predicted property vector $\widehat{y}(x)$ and corresponding baseline vector y^{base}, create an auxiliary conditioning vector $\widetilde{c}$ by setting the target channels to a perturbed version of $\widehat{y}(x)$ and the relative-improvement channels to $\widehat{y}(x) - y^{\mathrm{base}}$. Training the generator to reproduce x under $\widetilde{c}$ teaches the model that target tokens have semantic meaning, even before large quantities of hand-specified target-conditioned data exist.

Because the starting undesigned protein or complex is explicitly scored by BioEmu and/or UMA before redesign, the conditioning channels are available from the very first ADFLIP full-redesign step. This is important: the initial full redesign should already know whether it is being asked to optimize ΔG , attain or exceed a target T_{m}, improve ΔG_{bind}, or satisfy a compound objective, rather than receiving those constraints only after several edit steps have been taken.

6.4 Additional losses introduced by property conditioning

Whenever oracle-derived pseudo-labels are available, we add a property regression term

$$\mathcal{L}_{\mathrm{prop}} = \sum_{m \in \mathcal{M}_{\mathrm{avail}}} \lambda_m \, \ell_m\big(\widehat{y}_m, y_m^{\mathrm{oracle}}\big) + \sum_{m \in \mathcal{M}_{\mathrm{avail}}} \lambda_m^{\Delta} \, \ell_m^{\Delta}\big(\widehat{y}_m - \widehat{y}_m^{\mathrm{base}}, y_m^{\mathrm{oracle}} - y_m^{\mathrm{base}}\big), \tag{31}$$

where the available channels may include calibrated monomer stability, complex stability, ΔG_{bind}, $\Delta\Delta G_{\text{bind}}$, and T_{m}. The second summation explicitly trains the model on *improvements over baseline*, ensuring that the first full-redesign step and all later edits are supervised in the same coordinate system.

The full training loss in goal-conditioned complex mode becomes

$$\mathcal{L}_{\text{total,cx}} = \mathcal{L}_{\text{total}} + \lambda_{\text{prop}}\mathcal{L}_{\text{prop}} + \lambda_{\text{bind}}\mathcal{L}_{\text{bind-cons}}. \tag{32}$$

Typical defaults are $\lambda_{\text{prop}} \in [0.1, 0.5]$ and $\lambda_{\text{bind}} \in [0.05, 0.25]$, with the consistency loss weighted more heavily once the component heads have become numerically stable.

7 Extension to complexes, binding free energies, and target-conditioned generation

7.1 Task classes and thermodynamic bookkeeping for complexes

The monomeric formulation introduced earlier is only the first member of a broader family of design tasks. In practice, the same all-atom generator should also support two additional paradigms: (i) protein-protein interactions and multimers, where one or more designed chains interact with fixed or co-designed partner chains, and (ii) protein-ligand systems, where the designed protein is conditioned on bound non-protein atoms such as a small molecule, cofactor, ion cluster, peptide fragment, carbohydrate fragment, or nucleic-acid fragment. We therefore lift the monomer state into a general complex object

$$\mathcal{C} = \big(\mathcal{P}, \mathcal{L}, \mathcal{D}, B^{\text{bound}}, \Pi(\mathcal{C})\big), \tag{33}$$

where $\mathcal{P} = \{P_1, \ldots, P_C\}$ is the set of protein chains, $\mathcal{L} = \{L_1, \ldots, L_M\}$ is the set of non-protein bound components, $\mathcal{D}$ is the set of designable residues across one or more protein chains, B^{bound} denotes the bound-state coordinates supplied to ADFLIP, and $\Pi(\mathcal{C})$ is the set of separated objects used for thermodynamic bookkeeping. Monomers are the special case $C = 1$ and $M = 0$.

For any candidate design x on complex $\mathcal{C}$, the oracles are asked to score the bound system and the separated components. Let $\Pi(\mathcal{C}) = \{\mathcal{C}_1^{\text{sep}}, \ldots, \mathcal{C}_R^{\text{sep}}\}$ denote the separated states used for thermodynamic bookkeeping. For protein-protein interactions, $\Pi(\mathcal{C})$ usually contains the isolated chains or a predefined partition into subcomplexes. For protein-ligand design, $\Pi(\mathcal{C})$ contains the apo protein and the ligand or fragment; when the ligand identity is fixed across the design problem, its internal term cancels from mutation-induced differences and may be omitted from $\Delta\Delta G_{\text{bind}}$ calculations after that cancellation has been verified. The oracle-specific bound-state free energy is denoted

$$\widehat{\Delta G}_o^{\text{bound}}(x \mid \mathcal{C}), \tag{34}$$

and the component free energies are $\widehat{\Delta G}_{o,r}^{\text{sep}}(x \mid \mathcal{C}_r^{\text{sep}})$. The induced binding free-energy estimate is then

$$\widehat{\Delta G_{\text{bind}}}_o(x \mid \mathcal{C}) = \widehat{\Delta G}_o^{\text{bound}}(x \mid \mathcal{C}) - \sum_{r=1}^{R} \widehat{\Delta G}_{o,r}^{\text{sep}}(x \mid \mathcal{C}_r^{\text{sep}}), \tag{35}$$

with corresponding relative mutation effect

$$\widehat{\Delta\Delta G_{\text{bind}}}_o(x; x_{\text{ref}} \mid \mathcal{C}) = \widehat{\Delta G_{\text{bind}}}_o(x \mid \mathcal{C}) - \widehat{\Delta G_{\text{bind}}}_o(x_{\text{ref}} \mid \mathcal{C}). \tag{36}$$

Here x_{ref} is usually the ADFLIP seed or the wild-type complex sequence. The same decomposition is used under the hood for all three oracles. For monomeric stability tasks, only the component-wise folding terms are active. For multimeric or ligand tasks, the reward may include both folded-state stability terms and binding terms.

This bookkeeping matters because a mutation can improve the bound complex for the wrong reason. A hydrophobic substitution might appear to strengthen an interface simply because it destabilizes the isolated component and distorts the state definition. By explicitly evaluating both bound and separated components, the training signal can distinguish interface-specific stabilization from nonspecific global destabilization. This is the correct thermodynamic translation of oracle scores into affinity-aware design.

7.2 How ADFLIP is executed on bound and separated objects

Because ADFLIP already accepts all-atom complex input, the generator-side execution rule is comparatively simple.

1. **Bound pass.** Build the bound complex using the current designed protein sequence together with the fixed partner chains or ligand atoms. Run ADFLIP in its native all-atom mode, optionally over multiple structural states if a bound-state ensemble is available, and pack sidechains with PIPPACK.
2. **Separated passes.** For each separated object in $\Pi(\mathcal{C})$, build the corresponding apo or isolated-component structure and run the same ADFLIP checkpoint. No ligand-specific or partner-specific architecture change is introduced; only the input structure changes.
3. **Shared proposal logic.** The GFlowNet action space still edits protein sequence tokens only. Non-protein atoms are never mutated unless one explicitly introduces a second design space, which this manuscript does not require.

The resulting sequence proposals are therefore evaluated by the same generator under two physical contexts: the full complex and the separated components. This makes it straightforward to attach complex-aware thermodynamic labels without changing the core inverse-folding architecture.

7.3 Complex-aware cheap oracle: extending SPURS to bound and separated states

SPURS remains the cheap external oracle, but in complex tasks it is applied consistently to both the bound object and the separated components. We denote this bookkeeping-aware usage by cSPURS. For a candidate x on complex $\mathcal{C}$,

$$\widehat{\Delta\Delta G_{\text{bind}}}^{\text{SPURS}}(x) = \widehat{\Delta\Delta G}_{\text{bound}}^{\text{SPURS}}(x) - \sum_{r=1}^{R} \widehat{\Delta\Delta G}_{\text{sep},r}^{\text{SPURS}}(x), \tag{37}$$

where the bound-complex score is computed with the partner or ligand context present and the component scores are computed on the separated objects. For ligand systems with fixed ligand identity, the ligand-only term is constant across sequence edits and only reappears when absolute ΔG_{bind} is reported.

In the primary methodology these cSPURS quantities are interpreted conservatively. They are cheap thermodynamic surrogates used for ranking, replay annotation, and relative bookkeeping between bound and separated contexts. They are not used to override ADFLIP's proposal mechanism, and they are not treated as a universal binding-energy predictor. When the candidate is a single mutant or an explicit double mutant, cSPURS can use the corresponding full-sequence SPURS heads directly. For higher-order candidates, cSPURS becomes a coarse branch whose uncertainty is inflated and whose outputs primarily decide whether BioEmu or UMA should be invoked next. This is the complex analogue of the monomeric policy described earlier.

7.4 Generalized reward for mixed folding and binding objectives

Once the generator and the cheap oracle are complex aware, the scalarized reward can be generalized without changing the GFlowNet formalism. For any candidate x on context $\mathcal{C}$, define standardized fold, binding, and target-attainment terms,

$$\tilde{z}_{\text{fold}}(x) = \sum_{r \in \mathcal{R}_{\text{des}}} w_{\text{fold}}^{(r)} \, \widetilde{\Delta G}_{\text{stab},r}^{\text{sep}}(x), \tag{38}$$

$$\tilde{z}_{\text{bind}}(x) = w_{\text{bind}} \, \widetilde{\Delta G_{\text{bind}}}(x), \tag{39}$$

$$d_{\text{target}}(x; y^\star) = \sum_{p \in \mathcal{P}_{\text{active}}} \lambda_p \, \varphi_p\big(\hat{y}_p(x), y_p^\star, \varepsilon_p\big) \,, \tag{40}$$

where $\mathcal{R}_{\text{des}}$ denotes the design-relevant components, $\widetilde{\Delta G}_{\text{stab},r}^{\text{sep}}$ and $\widetilde{\Delta G_{\text{bind}}}$ are sign-harmonized calibrated summaries fused across SPURS, BioEmu, and UMA, and φ_p is a property-specific normalized deviation. For a lower-bound target such as "achieve at least $T_{\text{m}}^\star$," we use

$$\varphi_p(\hat{y}, y^\star, \varepsilon) = \max\left\{0, \frac{y^\star - \hat{y} - \varepsilon}{s_p}\right\}, \tag{41}$$

where s_p is a scale parameter; for two-sided targets, the absolute deviation outside the tolerance band is used. The resulting mixed-task reward is

$$R(x \mid \mathcal{C}, y^\star) = \exp\Big(\tilde{z}_{\text{fold}}(x) + \tilde{z}_{\text{bind}}(x) - d_{\text{target}}(x; y^\star) - p_{\text{pack}}(x) - p_{\text{collapse}}(x) - p_{\text{OOD}}(x)\Big). \tag{42}$$

The collapse penalty p_{collapse} is important in complex design: if a mutation artificially improves the bound score by destroying one isolated component, the separated-state terms should dominate and reduce the reward.

7.5 Complex tasks are part of the standard training distribution

Complex training is not a separate special mode; it is part of the standard training distribution. At update step u, a task type $\tau_u \in \{\text{monomer}, \text{ppi/multimer}, \text{ligand}\}$ is sampled from a schedule $p_u(\tau)$. The recommended default is a warmup period with monomers only for $u < U_0$, followed by a linear ramp of the complex probabilities until $u = U_1$,

$$p_u(\text{ppi}) = \alpha_{\text{ppi}} \min\left(1, \frac{u - U_0}{U_1 - U_0}\right), \qquad p_u(\text{lig}) = \alpha_{\text{lig}} \min\left(1, \frac{u - U_0}{U_1 - U_0}\right), \tag{43}$$

with $p_u(\text{monomer}) = 1 - p_u(\text{ppi}) - p_u(\text{lig})$. If one wishes to skip multimers or ligands entirely, the corresponding asymptotic weight is simply set to zero; the same ADFLIP checkpoint and the same training code path still support them.

Within each complex batch, the design mask $\mathcal{D}$ may cover one chain, multiple chains, or only residues inside an interface or ligand shell. For heteromers, all designated chains may be co-designed. For ligand systems, the default is to design the full protein while treating ligand atoms as fixed context. These choices are metadata, not new algorithms, which is why the same training loops in Methods 10–12 remain valid.

7.6 How BioEmu and UMA are used on complexes

BioEmu and UMA are both retained in the complex extension, but their role changes. BioEmu remains the fast whole-chain dynamics screen wherever its scope is appropriate. For monomers and chain-wise approximations, the same ensemble-derived stability features defined later in the manuscript are used. For multimeric or ligand tasks, BioEmu can still be used in two ways: first, as a component-wise dynamics oracle on the isolated protein chains or designed chain groups; second, as a chain-wise conformational prior for the bound complex when one chooses to evaluate only the designed chain while keeping partner or ligand geometry fixed. We do *not* claim that BioEmu becomes a universal explicit-complex simulator; rather, it becomes a fast source of whole-chain conformational diagnostics that can be combined with the component bookkeeping above.

UMA is the natural branch for direct complex-versus-components rescoring. For protein-protein complexes and multimers inside the whole-system atom budget, the default high-fidelity path is to run the bound complex and each separated component under the same prepared-state protocol, then compute ΔG_{bind} and $\Delta\Delta G_{\text{bind}}$ by subtraction. For ligand systems, the same rule applies; when the ligand is fixed across mutants, the ligand-only term cancels in relative $\Delta\Delta G_{\text{bind}}$ and can be omitted after verifying that the preparation protocol is mutation independent. Above the whole-system UMA paradigm, the same logic introduced earlier applies: BioEmu carries the scalable whole-chain burden, UMA runs on monomers and multimers/complexes up to $\sim 10K$ atoms, and we do not train on larger structures.

7.7 Goal-conditioned generation and explicit stopping rules

The complex-aware extension naturally supports goal-conditioned design. Let the user specify a target vector

$$y^\star = \big(T_{\text{m}}^\star,\, \Delta G_{\text{fold},1}^\star, \ldots, \Delta G_{\text{fold},R}^\star,\, \Delta G_{\text{bind}}^\star,\, m\big), \tag{44}$$

where m is a binary mask identifying which targets are active. During training, target vectors are drawn from empirical label ranges or from oracle-calibrated synthetic schedules, and classifier-free dropout is applied to m so the same policy can operate both conditionally and unconditionally. At inference time, the policy is conditioned on the user-specified target and queried repeatedly until either a candidate satisfies all active constraints within tolerance or a bounded retry budget is exhausted.

A candidate x is accepted when

$$\forall p \in \mathcal{P}_{\text{active}}, \qquad \varphi_p\big(\hat{y}_p(x), y_p^\star, \varepsilon_p\big) = 0, \tag{45}$$

and it passes the structural plausibility checks (packing quality, self-consistency, and any task-specific interface or pocket filters). Otherwise, the system keeps the best-so-far candidate under the deviation score $d_{\text{target}}(x; y^\star)$ and retries. The retry mechanism is bounded so that deployment is predictable: after $R_{\max}$ retries, the system returns the best candidate found so far together with its predicted properties and an explicit failure flag indicating which targets were missed. This is important for

experimental workflows because a negative outcome still provides the user with the best available trade-off rather than no answer.

Algorithm 1 Target-conditioned complex-aware generation with bounded retries

Require: Complex context $\mathcal{C}$, target vector $y^\star$, tolerances $\{\varepsilon_p\}$, retry budget $R_{\max}$, candidates per retry M

Initialize best candidate $x^{\text{best}} \leftarrow \varnothing$ and best deviation $d^{\text{best}} \leftarrow +\infty$
for $r = 1, \ldots, R_{\max}$ **do**
  Sample M candidates from the conditioned generator $p_\theta(x \mid \mathcal{C}, y^\star)$
  Score candidates with cheap oracles; escalate the top subset to BioEmu and UMA according to the standard tri-fidelity acquisition rules
  for each scored candidate x **do**
    Compute predicted property vector $\hat{y}(x)$ and target deviation $d_{\text{target}}(x; y^\star)$
    if $d_{\text{target}}(x; y^\star) < d^{\text{best}}$ **then**
      $x^{\text{best}} \leftarrow x$, $d^{\text{best}} \leftarrow d_{\text{target}}(x; y^\star)$, $\hat{y}^{\text{best}} \leftarrow \hat{y}(x)$
    end if
    if Eq. (45) holds and structural plausibility filters pass **then**
      return $(x, \hat{y}(x), \texttt{success} = \texttt{true}, \texttt{retries} = r)$
    end if
  end for
  Optionally widen temperature or sample-count settings for the next retry while keeping the same target vector
end for
return $(x^{\text{best}}, \hat{y}^{\text{best}}, \texttt{success} = \texttt{false}, \texttt{retries} = R_{\max})$

8 Reward design and tri-fidelity oracle fusion

8.1 Desiderata

The reward should satisfy eight properties. First, it must remain strictly positive because GFlowNets model an unnormalized density. Second, it must align sign conventions across SPURS, BioEmu, UMA, and the catalytic oracles. Third, it should penalize obviously implausible structures and, when desired, higher-order cheap-branch extrapolation or overly conservative near-seed behavior. Fourth, it should incorporate uncertainty, since all oracle families are imperfect in different ways. Fifth, it should be scalable: SPURS and KcatNet can be applied densely, BioEmu and MMKcat on larger filtered subsets, GraphKcat on narrower structure-qualified subsets, and UMA only selectively. Sixth, it must avoid reward hacking by sequences that exploit a single oracle while violating structural plausibility. Seventh, it should gracefully change emphasis with protein scale, so that BioEmu naturally becomes more important when prepared all-atom systems exceed the whole-protein UMA paradigm. Eighth, it should degrade gracefully when catalytic metadata are incomplete, rather than forcing every reaction-conditioned example through the same predictor.

8.2 Cheap branch: SPURS-derived mutation score

Let $x = (s, X^{\text{sc}})$ be a terminal candidate relative to the baseline starting sequence or complex-specific reference x^{base}. The cheap branch uses the global SPURS score from Eq. (19), not a local block

decomposition. In the primary methodology the trusted SPURS settings are $K(x) = 1$ and $K(x) = 2$, with $K(x) \geq 3$ treated as down-weighted and uncertainty-inflated. We define a baseline-aware standardized stability score as

$$z_{\mathrm{S}}(x) = \frac{-\widehat{\Delta\Delta G}^{\,\mathrm{SPURS,primary}}_{\mathrm{destab}}(x \mid B, s_{\mathrm{ref}}) - \mu_{\mathrm{S}}}{\sigma_{\mathrm{S}}}, \tag{46}$$

where μ_{S} and σ_{S} are running estimates on the replay buffer or a calibration set and the minus sign harmonizes the convention so that larger z_{S} means more stable.

Whenever uncertainty is available, we use a risk-adjusted variant

$$z_{\mathrm{S}}^{\mathrm{risk}}(x) = \frac{-\mu\big[\widehat{\Delta\Delta G}^{\,\mathrm{SPURS,primary}}\big] - \kappa_{\mathrm{S}}\,\sigma\big[\widehat{\Delta\Delta G}^{\,\mathrm{SPURS,primary}}\big] - \mu_{\mathrm{S}}}{\sigma_{\mathrm{S}}}. \tag{47}$$

A practical uncertainty estimate can be obtained by checkpoint ensembling, since SPURS is lightweight relative to very large sequence models. A conservative default is an ensemble of the top five validation checkpoints. For candidates with $K(x) \geq 3$, we additionally inflate the uncertainty term by a mutation-order factor or force the candidate into the BioEmu and possibly UMA queues. The point is not to pretend that SPURS becomes invalid above doubles; rather, it is to avoid letting the cheap branch dominate the reward in settings where its calibration is least certain.

Two points are important here. First, one utilizes for the $K(x) \geq 3$ branch, SPURS's higher-order decoder evaluated on the full-chain candidate. Second, one may swap the internal ESM-2 and ProteinMPNN branches for MINT and LigandMPNN, respectively, then repeat the same calibration and evaluation pipeline. The drop in replacement models should be reported separately from the primary method as an ablation, but could enhance performance on PPI and protein-ligand contexts.

8.3 Middle branch: BioEmu-derived ensemble stability score

BioEmu is used as a sequence-conditioned whole-chain dynamics oracle [16]. Given candidate sequence s, draw an equilibrium ensemble

$$\mathcal{Y}(x) = \{y_m\}_{m=1}^{M_{\mathrm{B}}}, \qquad y_m \sim p_{\mathrm{BioEmu}}(\cdot \mid s),$$

with M_{B} typically between 512 and 4096 during training. Because BioEmu emits backbone-dominant conformations rather than sidechain-resolved all-atom states, we compare its samples to the target backbone or seed fold using whole-chain observables:

$$Q_{\mathrm{nat}}^{\mathrm{B}}(m) = \frac{1}{|C_{\mathrm{nat}}|} \sum_{(i,j)\in C_{\mathrm{nat}}} \mathbf{1}\Big[d_{ij}(y_m) \leq \lambda_B d_{ij}^{(0)}\Big], \tag{48}$$

$$\mathrm{RMSD}_{\mathrm{bb}}^{\mathrm{B}}(m) = \mathrm{RMSD}_{\mathrm{bb}}(y_m, B), \tag{49}$$

$$H_{\mathrm{sec}}^{\mathrm{B}}(m) = \text{secondary-structure retention of } y_m \text{ relative to } B, \tag{50}$$

$$R_g^{\mathrm{B}}(m) = \text{radius of gyration of } y_m, \tag{51}$$

$$U_{\mathrm{glob}}^{\mathrm{B}}(m) = \text{global unfolding score or unfolded-frame indicator for } y_m. \tag{52}$$

The unfolding statistic $U_{\mathrm{glob}}^{\mathrm{B}}$ may be implemented as a scalar function of native-contact loss, secondary-structure loss, and overexpansion relative to the baseline ensemble. It does not need to be localized to mutation blocks.

We define BioEmu folded and disrupted basins by slightly looser thresholds than in the UMA branch because BioEmu is a coarse-grained equilibrium emulator rather than an atomistic MD engine:

$$\mathcal{F}_{\mathrm{B}} = \left\{ Q^{\mathrm{B}}_{\mathrm{nat}} \geq q^{\mathrm{B}}_{F},\ \mathrm{RMSD}^{\mathrm{B}}_{\mathrm{bb}} \leq r^{\mathrm{B}}_{F},\ H^{\mathrm{B}}_{\mathrm{sec}} \geq h^{\mathrm{B}}_{F} \right\}, \tag{53}$$

$$\mathcal{U}_{\mathrm{B}} = \left\{ Q^{\mathrm{B}}_{\mathrm{nat}} \leq q^{\mathrm{B}}_{U} \text{ or } \mathrm{RMSD}^{\mathrm{B}}_{\mathrm{bb}} \geq r^{\mathrm{B}}_{U} \text{ or } R^{\mathrm{B}}_{g} \geq g^{\mathrm{B}}_{U} \text{ or } U^{\mathrm{B}}_{\mathrm{glob}} \geq u^{\mathrm{B}}_{U} \right\}, \tag{54}$$

with defaults $(q^{\mathrm{B}}_{F}, q^{\mathrm{B}}_{U}, r^{\mathrm{B}}_{F}, r^{\mathrm{B}}_{U}, h^{\mathrm{B}}_{F}) = (0.70, 0.40, 3.0\,\text{Å}, 7.0\,\text{Å}, 0.65)$. Let

$$p^{\mathrm{B}}_{\mathcal{F}}(x) = \frac{1}{M_{\mathrm{B}}} \sum_{m=1}^{M_{\mathrm{B}}} \mathbf{1}[y_m \in \mathcal{F}_{\mathrm{B}}], \qquad p^{\mathrm{B}}_{\mathcal{U}}(x) = \frac{1}{M_{\mathrm{B}}} \sum_{m=1}^{M_{\mathrm{B}}} \mathbf{1}[y_m \in \mathcal{U}_{\mathrm{B}}].$$

Then define a BioEmu-based stability proxy at the model's reference condition $T_{\mathrm{ref}} \approx 300\,\mathrm{K}$:

$$\widehat{\Delta G}^{\,\mathrm{BioEmu}}_{\mathrm{stab}}(x) = k_B T_{\mathrm{ref}} \log \frac{p^{\mathrm{B}}_{\mathcal{F}}(x) + \varepsilon_{\mathrm{B}}}{p^{\mathrm{B}}_{\mathcal{U}}(x) + \varepsilon_{\mathrm{B}}}, \tag{55}$$

with $\varepsilon_{\mathrm{B}} = 10^{-4}$ to prevent numerical blow-up when one basin is rarely sampled. This quantity should be interpreted as a calibrated equilibrium proxy, not as a claim that BioEmu directly outputs exact experimental folding free energies.

To preserve more information than a single scalar, define a BioEmu feature vector

$$u_{\mathrm{B}}(x) = \left[\widehat{\Delta G}^{\,\mathrm{BioEmu}}_{\mathrm{stab}},\ p^{\mathrm{B}}_{\mathcal{F}},\ p^{\mathrm{B}}_{\mathcal{U}},\ \overline{Q^{\mathrm{B}}_{\mathrm{nat}}},\ \overline{H^{\mathrm{B}}_{\mathrm{sec}}},\ \overline{R^{\mathrm{B}}_{g}},\ \overline{U^{\mathrm{B}}_{\mathrm{glob}}},\ \mathrm{Var}(R^{\mathrm{B}}_{g}),\ \text{hinge and pocket indicators if relevant} \right]. \tag{56}$$

A calibration model g_ω then maps these ensemble statistics to a scalar thermostability quantity in the same experimental units used elsewhere:

$$\widehat{\Delta G}^{\,\mathrm{BioEmu}}_{\mathrm{stab,cal}}(x) = g_\omega(u_{\mathrm{B}}(x)). \tag{57}$$

We recommend a monotone regressor or a ridge model with spline features as the first implementation. The purpose of calibration is explicit: BioEmu is a protein-equilibrium emulator, and the regressor makes transparent how its sampled ensemble statistics are translated into the scalar used by the design reward.

The standardized and risk-adjusted BioEmu score is

$$z^{\mathrm{risk}}_{\mathrm{B}}(x) = \frac{\mu\left[\widehat{\Delta G}^{\,\mathrm{BioEmu}}_{\mathrm{stab,cal}}\right] - \kappa_{\mathrm{B}}\, \sigma\left[\widehat{\Delta G}^{\,\mathrm{BioEmu}}_{\mathrm{stab,cal}}\right] - \mu_{\mathrm{B}}}{\sigma_{\mathrm{B}}}. \tag{58}$$

Its uncertainty should include at least three terms: sample bootstrap variance over $\mathcal{Y}(x)$, checkpoint disagreement if multiple BioEmu checkpoints are retained, and threshold sensitivity of the folded/disrupted basin classifier. In practice, BioEmu is the default whole-chain dynamics oracle whenever the protein is BioEmu-eligible; UMA is then run as a follow-up.

8.4 High-fidelity branch: UMA-MD absolute stability score

We define the high-fidelity oracle generically as $\widehat{\Delta G}^{\,\mathrm{UMA}}_{\mathrm{stab}}(x \mid B)$, but its actual implementation depends on prepared-system atom count and calibration paradigm. When the prepared system fits within the default whole-protein budget, whole-protein UMA-MD is used. Above that scale, UMA

is still valuable but should be focused on local or hybrid questions rather than asked to carry the entire stability prediction alone.

For whole-protein runs, define global observables on the terminal packed structure:

$$Q_{\mathrm{nat}}^{\mathrm{U}}(t) = \frac{1}{|C_{\mathrm{nat}}|} \sum_{(i,j)\in C_{\mathrm{nat}}} \mathbf{1}\Big[d_{ij}(t) \le \lambda_Q d_{ij}^{(0)}\Big], \tag{59}$$

$$R_{\mathrm{g}}^{\mathrm{U}}(t) = \text{radius of gyration}, \tag{60}$$

$$\mathrm{RMSD}_{\mathrm{bb}}^{\mathrm{U}}(t) = \text{backbone RMSD to the starting structure}, \tag{61}$$

$$H_{\mathrm{sec}}^{\mathrm{U}}(t) = \text{secondary-structure retention fraction}. \tag{62}$$

Define the folded basin $\mathcal{F}_{\mathrm{U}}$ and disrupted basin $\mathcal{U}_{\mathrm{U}}$ by conservative thresholds, then estimate

$$\widehat{\Delta G}_{\mathrm{stab}}^{\mathrm{UMA}}(T) = k_B T \log \frac{p_{\mathcal{F}_{\mathrm{U}}}(T)}{p_{\mathcal{U}_{\mathrm{U}}}(T)}. \tag{63}$$

A temperature ladder and a calibration model g_ψ convert trajectory-derived observables into the scalar experimental scale used by the reward. The detailed protocol is given later in Section 14.

The risk-adjusted expensive score is

$$z_{\mathrm{U}}^{\mathrm{risk}}(x) = \frac{\mu\big[\widehat{\Delta G}_{\mathrm{stab,cal}}^{\mathrm{UMA}}\big] - \kappa_{\mathrm{U}}\,\sigma\big[\widehat{\Delta G}_{\mathrm{stab,cal}}^{\mathrm{UMA}}\big] - \mu_{\mathrm{U}}}{\sigma_{\mathrm{U}}}. \tag{64}$$

Its uncertainty should include replicate trajectory variance, disagreement between `uma-s-1p1` and `uma-m-1p1`, calibration error, and disagreement between whole-protein and local/hybrid fallback estimates when both are available.

8.5 Complex and component translation to binding quantities

For monomer examples, the reward uses only the thermostability channels already defined. For PPI, multimer, or ligand examples, the reward is extended by an explicit binding term built from the complex-versus-component decomposition in Eqs. (5) and (6). The cheap SPURS proxy is

$$\widehat{\Delta\Delta G_{\mathrm{bind}}}^{\mathrm{SPURS}}(x) = \widehat{\Delta\Delta G}_{\mathrm{destab,cx}}^{\mathrm{SPURS}}(x) - \sum_{u=1}^{U} \nu_u \widehat{\Delta\Delta G}_{\mathrm{destab},u}^{\mathrm{SPURS}}(x), \tag{65}$$

where the bound-complex score is computed with the partner or ligand context present and the component scores are computed on the separated objects. The ligand-only term is constant across sequence edits and only reappears when absolute ΔG_{bind} is reported.

For UMA, the corresponding quantity is obtained directly from the complex and component MD runs:

$$\widehat{\Delta G_{\mathrm{bind}}}_{\mathrm{cal}}^{\mathrm{UMA}}(x) = \widehat{G}_{\mathrm{cx,cal}}^{\mathrm{UMA}}(x) - \sum_{u=1}^{U} \nu_u \widehat{G}_{u,\mathrm{cal}}^{\mathrm{UMA}}(x). \tag{66}$$

For BioEmu, we do *not* introduce a universal direct binding term. In protein-protein or ligand tasks BioEmu is used only for chain-wise stability, with reliability gate is reduced accordingly.

We standardize and harmonize the binding term so that larger is better:

$$z_{\mathrm{bind}}^{\mathrm{risk}}(x) = \frac{-\mu\big[\widehat{\Delta\Delta G_{\mathrm{bind}}}(x)\big] - \kappa_{\mathrm{bind}}\sigma\big[\widehat{\Delta\Delta G_{\mathrm{bind}}}(x)\big] - \mu_{\mathrm{bind}}}{\sigma_{\mathrm{bind}}}. \tag{67}$$

The source of $\Delta\widehat{\Delta G}_{\mathrm{bind}}$ depends on which oracle branch is available. For cheap screening it is SPURS-based. For shortlisted candidates it is UMA-based and treated as the primary binding label.

8.6 Catalytic branch: whole-enzyme UMA dynamics, GraphKcat, and auxiliary kinetic predictors

For catalytic tasks, the reward operates on a reaction-conditioned candidate $(x, \mathcal{R})$ rather than on a sequence alone. Let the reaction context be $\mathcal{R} = (\mathcal{S}_{\mathrm{sub}}, \mathcal{S}_{\mathrm{prod}}, \eta)$ as defined above, and assume that the current implementation has access to a reactant-bound complex and a product-bound complex for the same enzyme candidate. The default catalytic branch in the repository then proceeds in three layers.

First, a broad whole-enzyme UMA equilibrium screen estimates the productive-pose or generalized-NAC occupancy $p_{\mathrm{gNAC}}(x, \mathcal{R})$ in the reactant basin and converts it into a gating free energy

$$\widehat{\Delta G}_{\mathrm{gate}}(x, \mathcal{R}) = -RT \log\left(\frac{\widehat{p}_{\mathrm{gNAC}}(x, \mathcal{R})}{1 - \widehat{p}_{\mathrm{gNAC}}(x, \mathcal{R})}\right). \tag{68}$$

Second, forward and reverse steered whole-enzyme UMA dynamics between the reactant-bound and product-bound endpoint complexes estimate a nonequilibrium barrier surrogate $\widehat{\Delta G}^{\ddagger}_{\mathrm{sMD}}(x, \mathcal{R})$ and harvest near-transition-state-like frames. When the optional PMF stage is enabled, those path seeds initialize an umbrella PMF and the repository uses

$$\widehat{\Delta G}_{\mathrm{bar}}(x, \mathcal{R}) = \begin{cases} \widehat{\Delta G}^{\ddagger}_{\mathrm{PMF}}(x, \mathcal{R}), & \text{if the PMF succeeds,} \\ \widehat{\Delta G}^{\ddagger}_{\mathrm{sMD}}(x, \mathcal{R}), & \text{otherwise.} \end{cases} \tag{69}$$

Third, the catalytic barrier and gating terms are translated into a transition-state-style log-rate proxy,

$$\widehat{\ell}_{\mathrm{UMAcat}}(x, \mathcal{R}) = \log_{10}\left(\frac{k_B T}{h}\right) - \frac{\widehat{\Delta G}_{\mathrm{gate}}(x, \mathcal{R}) + \widehat{\Delta G}_{\mathrm{bar}}(x, \mathcal{R})}{RT \ln 10}. \tag{70}$$

This quantity is the primary catalytic scalar used by the default Method III implementation.

GraphKcat provides the orthogonal learned structural catalytic branch. In the current repository implementation, the fused catalytic reward uses the turnover channel $\widehat{\ell}^{\mathrm{GK}}_{k_{\mathrm{cat}}}$ together with the UMA-cat log-rate proxy. The richer GraphKcat $\log_{10} K_{\mathrm{M}}$ and $\log_{10}(k_{\mathrm{cat}}/K_{\mathrm{M}})$ channels remain scientifically useful and are retained elsewhere in the manuscript, but the default code path currently fuses UMA-cat with the GraphKcat turnover channel. We therefore define

$$z^{\mathrm{risk}}_{\mathrm{UCAT}}(x, \mathcal{R}) = \mu\left[\widehat{\ell}_{\mathrm{UMAcat}}(x, \mathcal{R})\right] - \kappa_{\mathrm{UCAT}}\sigma\left[\widehat{\ell}_{\mathrm{UMAcat}}(x, \mathcal{R})\right], \tag{71}$$

$$z^{\mathrm{risk}}_{\mathrm{GK}}(x, \mathcal{R}) = \mu\left[\widehat{\ell}^{\mathrm{GK}}_{k_{\mathrm{cat}}}(x, \mathcal{R})\right] - \kappa_{\mathrm{GK}}\sigma\left[\widehat{\ell}^{\mathrm{GK}}_{k_{\mathrm{cat}}}(x, \mathcal{R})\right], \tag{72}$$

$$z_{\mathrm{agree}}(x, \mathcal{R}) = -\left|\widehat{\ell}_{\mathrm{UMAcat}}(x, \mathcal{R}) - \widehat{\ell}^{\mathrm{GK}}_{k_{\mathrm{cat}}}(x, \mathcal{R})\right|. \tag{73}$$

The implemented fused catalytic score is then

$$z^{\mathrm{risk}}_{\mathrm{cat}}(x, \mathcal{R}) = \rho_{\mathrm{UCAT}}\, w_{\mathrm{UCAT}}\, z^{\mathrm{risk}}_{\mathrm{UCAT}}(x, \mathcal{R}) + \rho_{\mathrm{GK}}\, w_{\mathrm{GK}}\, z^{\mathrm{risk}}_{\mathrm{GK}}(x, \mathcal{R}) + \rho_{\mathrm{UCAT}}\rho_{\mathrm{GK}}\, w_{\mathrm{agree}}\, z_{\mathrm{agree}}(x, \mathcal{R}), \tag{74}$$

where ρ_{UCAT} and ρ_{GK} are reliability gates. In the current repository, failed UMA-cat evaluations are not allowed to silently collapse to a neutral score: they are clamped to a strongly negative reward contribution. This is a deliberate anti-cheating design choice.

KcatNet and MMKcat remain valuable auxiliary catalytic predictors, especially for settings where product-bound structures are unavailable or when one wants a purely predictive wide-stage catalytic view. They can therefore be used as additional terms or ablations layered on top of Eq. (74). However, they are not the primary implemented control loop in the repository's current catalytic RL specialization.

8.7 Component-protection terms and no-cheating constraints

A binding-aware design system can cheat by destabilizing the separated components, thereby making the bound complex appear artificially favorable in Eq. (5). To prevent this, every complex reward includes component-protection penalties. Let s_u^{floor} be a desired minimum component stability or a maximum allowed loss relative to the seed for component u. Define

$$z_{\text{comp}}(x) = -\frac{1}{U}\sum_{u=1}^{U} \max\Big(0, s_u^{\text{floor}} - \widehat{y}_u^{\text{stab}}(x)\Big)\,. \tag{75}$$

When the design objective is to improve affinity without destroying the monomers or subunits, z_{comp} should carry nontrivial weight. For ligand-binding projects the analogous protection term usually applies to the apo protein stability rather than to the ligand itself.

8.8 Target-conditioned reward extension

When a target vector c is supplied, the scalarized score is augmented by a property-attainment term. Let $\widehat{y}_m(x)$ be the fused prediction for channel m and define the normalized violation

$$\delta_m(x;c) = \frac{\max\big(0,\, |\widehat{y}_m(x) - y_m^\star| - \epsilon_m\big)}{s_m}, \tag{76}$$

where s_m is a channel-specific scale, for example 1 kcal/mol for $\Delta\Delta G$ or ΔG_{bind} and 2–5 K for T_{m}. The target-attainment score is then

$$z_{\text{target}}(x;c) = -\sum_{m\in\mathcal{M}_{\text{target}}} w_m\, \delta_m(x;c). \tag{77}$$

This term is inactive when no target vector is provided. For catalytic targets, sensible defaults are $s_m = 0.25$ to 0.5 on the log scale for $\log_{10} k_{\text{cat}}$, $\log_{10} K_{\text{M}}$, and $\log_{10}(k_{\text{cat}}/K_{\text{M}})$ depending on assay variability and calibration quality.

8.9 Reliability gates for BioEmu and UMA

Not every oracle should have the same influence on every candidate. We therefore introduce reliability gates $\rho_{\text{B}}(x), \rho_{\text{U}}(x) \in [0,1]$:

$$\rho_{\text{B}}(x) = \begin{cases} 1, & \text{soluble single-chain or chain-wise compatible protein,} \\ 0.5, & \text{approximate use outside strict BioEmu scope,} \\ 0, & \text{BioEmu skipped,} \end{cases} \tag{78}$$

$$\rho_{\text{U}}(x) = \begin{cases} 1, & N_{\text{atom}} \leq\sim 10,000 \text{ and whole-protein UMA calibration acceptable,} \\ 0.35\text{–}0.6, & N_{\text{atom}} \geq 10000, \\ 0, & \text{UMA not run or scientifically ruled out.} \end{cases} \tag{79}$$

These gates are not hard scientific laws. They are explicit bookkeeping devices that prevent the reward from pretending all branches are equally reliable on every candidate. In the large-protein paradigm emphasized in this project, $\rho_{\mathrm{B}}(x)$ stays near one while $\rho_{\mathrm{U}}(x)$ is reduced for the $> 10,000$ atom setting, and stays near 1 otherwise. We do not train on systems outside of that approximate size range.

8.10 Structure plausibility and mutation-order regularization

We include the following auxiliary terms:

$$z_{\mathrm{pack}}(x) = -\frac{\overline{u_{\mathrm{pack}}}(x) - \mu_{\mathrm{pack}}}{\sigma_{\mathrm{pack}}}, \tag{80}$$

$$z_{\mathrm{ord}}(x) = -\mathbf{1}[K(x) \geq 3]\,\mathbf{1}[\text{no BioEmu/UMA supervision yet}], \tag{81}$$

$$z_{\mathrm{rad}}(x) = -\max\left(0, \frac{K(x) - K_{\mathrm{rad,soft}}}{L}\right), \tag{82}$$

$$z_{\mathrm{OOD}}(x) = -\mathrm{OOD}(x). \tag{83}$$

The new z_{ord} term is intentionally weak and operational. It does *not* impose a scientific Hamming-distance cap and it does not attempt to reconstruct local mutation blocks. Its sole purpose is to prevent the cheap branch from dominating reward estimates for higher-order candidates before they have been seen by BioEmu or UMA. Once a $K(x) \geq 3$ candidate has received a higher-fidelity score, the penalty vanishes. The optional radius term z_{rad} remains off by default and is only activated in deployment modes that explicitly prefer nearer-seed solutions.

8.11 Final scalarized reward

With weights $w = (w_{\mathrm{S}}, w_{\mathrm{B}}, w_{\mathrm{U}}, w_{\mathrm{cat}}, w_{\mathrm{bind}}, w_{\mathrm{comp}}, w_{\mathrm{target}}, w_{\mathrm{pack}}, w_{\mathrm{ord}}, w_{\mathrm{rad}}, w_{\mathrm{OOD}})$, define the scalarized score

$$\begin{aligned} s_w(x; c, \mathcal{R}) = {} & w_{\mathrm{S}} z_{\mathrm{S}}^{\mathrm{risk}}(x) + \rho_{\mathrm{B}}(x) w_{\mathrm{B}} z_{\mathrm{B}}^{\mathrm{risk}}(x) + \rho_{\mathrm{U}}(x) w_{\mathrm{U}} z_{\mathrm{U}}^{\mathrm{risk}}(x) \\ & + w_{\mathrm{cat}} z_{\mathrm{cat}}^{\mathrm{risk}}(x, \mathcal{R}) + w_{\mathrm{bind}} z_{\mathrm{bind}}^{\mathrm{risk}}(x) + w_{\mathrm{comp}} z_{\mathrm{comp}}(x) + w_{\mathrm{target}} z_{\mathrm{target}}(x; c) \\ & + w_{\mathrm{pack}} z_{\mathrm{pack}}(x) + w_{\mathrm{ord}} z_{\mathrm{ord}}(x) + w_{\mathrm{rad}} z_{\mathrm{rad}}(x) + w_{\mathrm{OOD}} z_{\mathrm{OOD}}(x). \end{aligned} \tag{84}$$

The GFlowNet reward is then

$$R_w(x; c, \mathcal{R}) = \varepsilon_R + \exp\Big(\mathrm{clip}(s_w(x; c, \mathcal{R}), s_{\min}, s_{\max})/\tau_R\Big), \tag{85}$$

with defaults $\varepsilon_R = 10^{-6}$, $s_{\min} = -8$, $s_{\max} = 8$, and $\tau_R = 1$. The exponential ensures positivity and allows the GFlowNet to model a broader dynamic range; clipping protects optimization from heavy-tailed or miscalibrated oracle outputs.

In the updated protocol, w_{rad} defaults to zero. This means the primary objective does not force candidates to remain close to the seed in Hamming distance. If a downstream use case prefers conservative redesign, one can reintroduce a positive w_{rad} at inference or in a separate conditional policy. By contrast, w_{ord} is nonzero by default because the primary methodology wants the cheap branch to be conservative for $K \geq 3$ candidates until BioEmu or UMA supervision arrives. BioEmu is weighted strongly by default whenever it is applicable.

Table 3: Recommended initial reward weights and interpretation. These are proposed defaults rather than validated optima.

Component	Default weight	Role
w_{S}	1.0	Primary cheap oracle driver. Most trusted for single mutants and explicit double mutants; conservative above that.
w_{B}	0.75 overall; 1.0 in the large-protein setting	Whole-chain dynamics branch. Up-weight when BioEmu is the main dynamics oracle ($8k$–$10k$ range).
w_{U}	1.0 with direct whole-protein UMA; 0.25–0.6 with local/hybrid fallback	Highest-fidelity MLIP-MD branch. Weighted more heavily when whole-protein calibration is strong.
w_{cat}	0.5 overall; task dependent up to 1.0	Aggregate catalytic branch weight. Increase on reaction-conditioned enzyme tasks and set to zero when no catalytic target is active.
w_{bind}	0.5 on complex tasks; 0 on monomers	Reward term for binding improvement derived from complex-versus-component translation.
w_{comp}	0.25 on complex tasks	Protects separated components from being destabilized to create artificial affinity gains.
w_{target}	0 when unconstrained; 0.75 with user targets	Activates target-attainment pressure for specified property vectors.
w_{pack}	0.25	Discourages poorly packed candidates and sidechain artifacts.
w_{ord}	0.25	Prevents higher-order candidates from being overtrusted before BioEmu or UMA supervision arrives.
w_{rad}	0.0 by default; optional 0.05–0.20	Optional bias toward nearer-seed designs for interpretation or synthesis constraints.
w_{OOD}	0.15	Down-weights candidates that appear too far from the SPURS or BioEmu calibration manifolds.

9 Mixed training on monomers, multimers, and protein-ligand systems

9.1 Standard batch sampler with optional warmup and delayed multimer introduction

Complex-aware and catalysis-aware training are part of the standard protocol rather than afterthoughts. Let the batch-type sampler at update u be

$$p_u(\text{mono}, \text{ppi}, \text{lig}) = \big(1 - \alpha_{\text{ppi}}(u) - \alpha_{\text{lig}}(u),\ \alpha_{\text{ppi}}(u),\ \alpha_{\text{lig}}(u)\big). \tag{86}$$

During an optional warmup phase $u < U_0$, set $\alpha_{\text{ppi}}(u) = \alpha_{\text{lig}}(u) = 0$ so that the generator first stabilizes on monomeric redesign. During the ramp $U_0 \leq u < U_1$, increase the complex weights linearly or sigmoidally until they reach user-chosen steady-state values. If a project does not need multimers or ligands, these weights simply remain zero. The methodology therefore supports three operating modes with the same code path: monomer-only, mixed from the start, or warmup-then-mixed.

The training examples themselves are not framed as a classical mutation dataset. Instead, each batch element is a de novo-designed protein or complex with total protein length sampled from approximately 20–400 residues, together with its current starting sequence and structural context. A good default is to begin with monomers only, then ramp to a steady-state mixture such as 60% monomers, 25% PPIs or multimers, and 15% protein-ligand systems. These are engineering defaults rather than scientific claims.

Within each sampled example we always evaluate the starting undesigned sequence or complex first, obtaining a baseline property vector from the admissible oracles. The first ADFLIP full-redesign proposal is then conditioned on the training target relative to that baseline, scored, and inserted into replay. Subsequent edit trajectories are therefore learned in a consistent baseline-relative coordinate system from the first redesign step onward. Reaction-conditioned catalytic examples are handled as a metadata overlay on top of these structural examples: the same backbone or complex may be paired with substrate, product, and assay metadata so that catalytic oracle channels are activated only when the required reaction context exists.

9.2 Under-the-hood complex and component oracle runner

Every training example is evaluated in three layers: the starting undesigned object, the first ADFLIP full-redesign proposal, and any later edited descendants. For complex tasks, each of those layers is evaluated twice conceptually: once as the bound complex and once as a set of separated components. The user does not manage this bookkeeping explicitly. Instead, the training pipeline constructs the required objects internally. For a candidate sequence x built on complex $\mathcal{C}$:

1. build the bound complex with the mutated designable protein chains and a full sidechain repack performed in the presence of the partner or ligand context;
2. construct the separated components specified by $\mathfrak{D}$, preserving the same mutated sequences on designable chains and the same fixed context objects on non-designable chains or ligands when appropriate;
3. run the admissible oracle branches on the *starting undesigned object* to obtain baseline values $(\Delta G\ , T_{\text{m}}, \Delta G_{\text{bind}}, \ldots)^{\text{base}}$;

4. run the same oracle stack on the first full-redesign proposal $x^{(0)}$ so that the initial ADFLIP redesign receives absolute and baseline-relative labels;
5. run the same oracle stack on later edited descendants $x^{(1)}, x^{(2)}, \ldots$ as required by the acquisition policy;
6. translate the resulting outputs to $\widehat{\Delta G_{\text{bind}}}$, $\widehat{\Delta\Delta G_{\text{bind}}}$, $\widehat{\Delta T_{\text{m}}}$, component stability floors, and target-violation scores;
7. return the resulting property bundle to the reward and auxiliary heads.

This is the precise sense in which binding calculations and baseline-relative redesign calculations are handled "under the hood." The user provides a structure, a starting sequence, a design scope, and optionally a target vector; the system handles complex-versus-component decomposition, baseline scoring, oracle dispatch, and thermodynamic translation internally.

Algorithm 2 Complex and component oracle runner

Require: Complex example $\mathcal{C}$, starting sequence x^{base}, candidate sequence x, decomposition map $\mathfrak{D}$, oracle budget policy

Build bound complex structure for x^{base} and run full repack in bound context
Construct separated components $\{\mathcal{C}_u^{\text{base}}\}_{u=1}^{U}$ from $\mathfrak{D}$
Evaluate admissible oracles on the bound starting object and separated starting components to cache baseline property vector y^{base}
Build bound complex structure for candidate x and run full repack in bound context
Construct separated components $\{\mathcal{C}_u\}_{u=1}^{U}$ from $\mathfrak{D}$ with the same mutated protein sequences
Initialize property bundle $\mathcal{B}(x) \leftarrow \{y^{\text{base}}\}$
for object $o \in \{\text{cx}, 1, \ldots, U\}$ **do**
    if SPURS is admissible for o **then**
        Compute cheap mutation score and uncertainty for o
    end if
    if BioEmu is admissible for o **then**
        Compute chain-wise or object-wise dynamics features and uncertainty for o
    end if
    if UMA is selected for o by the acquisition policy **then**
        Run whole-object or local/hybrid UMA protocol and calibrate the result
    end if
    Append all outputs for o to $\mathcal{B}(x)$
end for
Translate $\mathcal{B}(x)$ to $\widehat{\Delta G_{\text{bind}}}(x)$, $\widehat{\Delta\Delta G_{\text{bind}}}(x)$, $\widehat{\Delta T_{\text{m}}}(x)$, component stability floors, and target-violation scores
return $\mathcal{B}(x)$

9.3 Task-specific remarks for multimers and ligands

For homomers or symmetric multimers, the design scope may cover one protomer with symmetry-tied edits or several protomers with independent edits. The default is symmetry tying when the backbone geometry itself is symmetric. For asymmetric multimers, each chain receives its own chain-role embedding and may or may not be editable.

For protein-ligand systems, the default designable objects are protein residues only. Small molecules and fragments are treated as fixed typed context atoms. Biomolecular fragments such as peptide epitopes, glycans, or nucleic-acid fragments can be represented either as typed atoms or as non-editable residue-like context tokens depending on which implementation is easier to maintain. Because ADFLIP is already ligand-aware, the proposal network remains unchanged across monomer, PPI, and ligand tasks. The main changes are in the oracle runner and the property translation to bound-versus-separated thermodynamic quantities.

BioEmu is used conservatively in complexes: chain-wise or component-wise when that is scientifically defensible, not as a universal explicit-complex simulator. UMA remains the preferred branch for direct bound-versus-separated rescoring whenever the prepared system is inside the working size setting. SPURS provides cheap ranking and double-mutant supervision, and with mild down-weighting, higher order mutants, but ADFLIP remains the sole proposal backbone in the primary method.

9.4 Task-specific remarks for reaction-conditioned catalytic design

Catalytic tasks are not a separate generator architecture; they are a reaction-conditioned specialization of the same sequence-editing problem. The designable object remains the enzyme sequence, but the oracle stack now depends on reaction metadata. At minimum, KcatNet and GraphKcat require substrate identity, usually provided as a SMILES string. MMKcat can additionally use product identities when they are available and remains usable under curated missing-modality patterns when they are not. GraphKcat further requires either a protein structure path or a structure that can be materialized from the current candidate.

Practically, catalytic examples therefore carry three additional bundles of metadata. The first is a substrate bundle, which must be present for any Kcat-mode training example. The second is an optional product bundle, which improves MMKcat supervision and helps represent product-dependent turnover effects. The third is an assay-context bundle, including pH, temperature, organism label, and any task-specific categorical descriptors. These metadata are not cosmetic. They determine which catalytic oracle branches are admissible and how their predictions should be calibrated.

The generator-side implication is simple. ADFLIP still edits residues only. Substrates, products, and assay metadata do not become part of the mutation action space. Instead, they condition the reward and target vector. This makes catalytic design compatible with the rest of the manuscript's GFlowNet machinery: the same edit DAG, the same replay logic, and the same one-shot student framework can all be reused once reaction context is introduced as oracle-conditioning metadata.

9.5 How the mixed sampler enters Methods I, II, and III

Methods I–III all inherit the same example-type sampler and the same complex/component runner. In Simul-MF, complex examples appear directly in on-policy rollouts and are scored online with the mixed-fidelity pipeline; catalytic reaction metadata simply activate additional oracle channels on the examples that carry them. In Seq-Curr, the curriculum may introduce monomers first, then PPIs, then ligand examples, or jump directly to the mixed setting if the initial generator is already stable, with catalytic targets delayed until substrate metadata quality is reliable. In OneShot-AL, the active-learning dataset D_r is stratified both by example type and by oracle-eligibility pattern so that each round acquires stability, binding, and catalytic labels according to the project's needs.

10 Method I: simultaneous tri-fidelity GFlowNet fine-tuning

10.1 Conceptual overview

Simul-MF updates a single GFlowNet/ADFLIP policy using all three oracle levels at once. The same training loop applies to monomers, multimers, protein–protein complexes, and ligand-conditioned systems; the only difference is that the state encoder, cheap oracle, and reward all use the complex-aware bookkeeping introduced in Section 7. The cheap SPURS branch is evaluated densely and synchronously on every rollout state and on every terminal candidate. The BioEmu branch is evaluated on a substantially wider subset of terminal candidates than UMA because it is fast, whole-chain, and dynamics-aware. The UMA branch is evaluated on a much smaller and more carefully chosen subset, with whole-protein UMA-MD up to roughly 8000 atoms as the default path and local or hybrid fallback above that ~ 10000 atoms until UMA is validated in a more rigorous way on larger systems. We already have seen it is capable of running on $100K+$ atoms, but the level of eval on such systems is not as rigorous as it could be. The reward for every trajectory is therefore a tri-fidelity fusion in which missing BioEmu and UMA terms are either imputed, down-weighted, or reliability-gated, not silently assumed absent.

This design is appropriate when one wants physical feedback during training but also wants large-protein coverage and a single mixed-task learner rather than a separate model per task class. For proteins whose prepared systems exceed roughly 8000–10000 atoms, BioEmu becomes the main whole-chain dynamics oracle and UMA contributes local or hybrid corrections only where it adds mechanistic value, such as for complexes where BioEmu is less viable. For proteins inside the whole-protein UMA setting, the model benefits from agreement or disagreement between BioEmu and UMA, because that disagreement itself is informative about uncertainty and possible reward hacking.

10.2 Two-stage acquisition for BioEmu and UMA

Not every sampled candidate should be sent to every oracle. We therefore define a wider BioEmu acquisition score

$$A_{\mathrm{B}}(x) = \alpha_1 z_{\mathrm{S}}^{\mathrm{risk}}(x) + \alpha_2 \sigma_{\mathrm{S}}(x) + \alpha_3 d_{\mathrm{nov}}(x) + \alpha_4 u_{\mathrm{pack}}(x) + \alpha_5 \mathbf{1}[K(x) = 2] + \alpha_6 \frac{K(x)}{L}, \tag{87}$$

where $\sigma_{\mathrm{S}}(x)$ is SPURS uncertainty, $d_{\mathrm{nov}}(x)$ is novelty relative to the already screened pool, and $u_{\mathrm{pack}}(x)$ is an optional pack-uncertainty term derived from u_{pack}. A simple default is $(\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5, \alpha_6) = (0.40, 0.15, 0.15, 0.10, 0.10, 0.10)$. This rule deliberately allocates part of the BioEmu budget to uncertain, novel, explicit double-mutant, and higher-order candidates, not only to the top SPURS scorers.

From the BioEmu-scored pool, define a tighter UMA acquisition score

$$\begin{aligned} A_{\mathrm{U}}(x) = \beta_1 z_{\mathrm{B}}^{\mathrm{risk}}(x) + \beta_2 \sigma_{\mathrm{B}}(x) + \beta_3 \left| z_{\mathrm{B}}^{\mathrm{risk}}(x) - z_{\mathrm{S}}^{\mathrm{risk}}(x) \right| + \beta_4 d_{\mathrm{nov}}(x) \\ + \beta_5 \mathbf{1}[N_{\mathrm{atom}}(x) \le 10000] + \beta_6 \mathbf{1}[K(x) \ge 3], \end{aligned} \tag{88}$$

with a sensible starting point $(\beta_1, \ldots, \beta_6) = (0.35, 0.15, 0.20, 0.10, 0.10, 0.10)$. The disagreement term is important: candidates that look good to SPURS but unstable to BioEmu, or vice versa, are often precisely the ones that most benefit from MLIP-based rescoring. If $N_{\mathrm{atom}}(x)$ exceeds the preferred whole-protein UMA paradigm, the final indicator is replaced by a smaller value and the

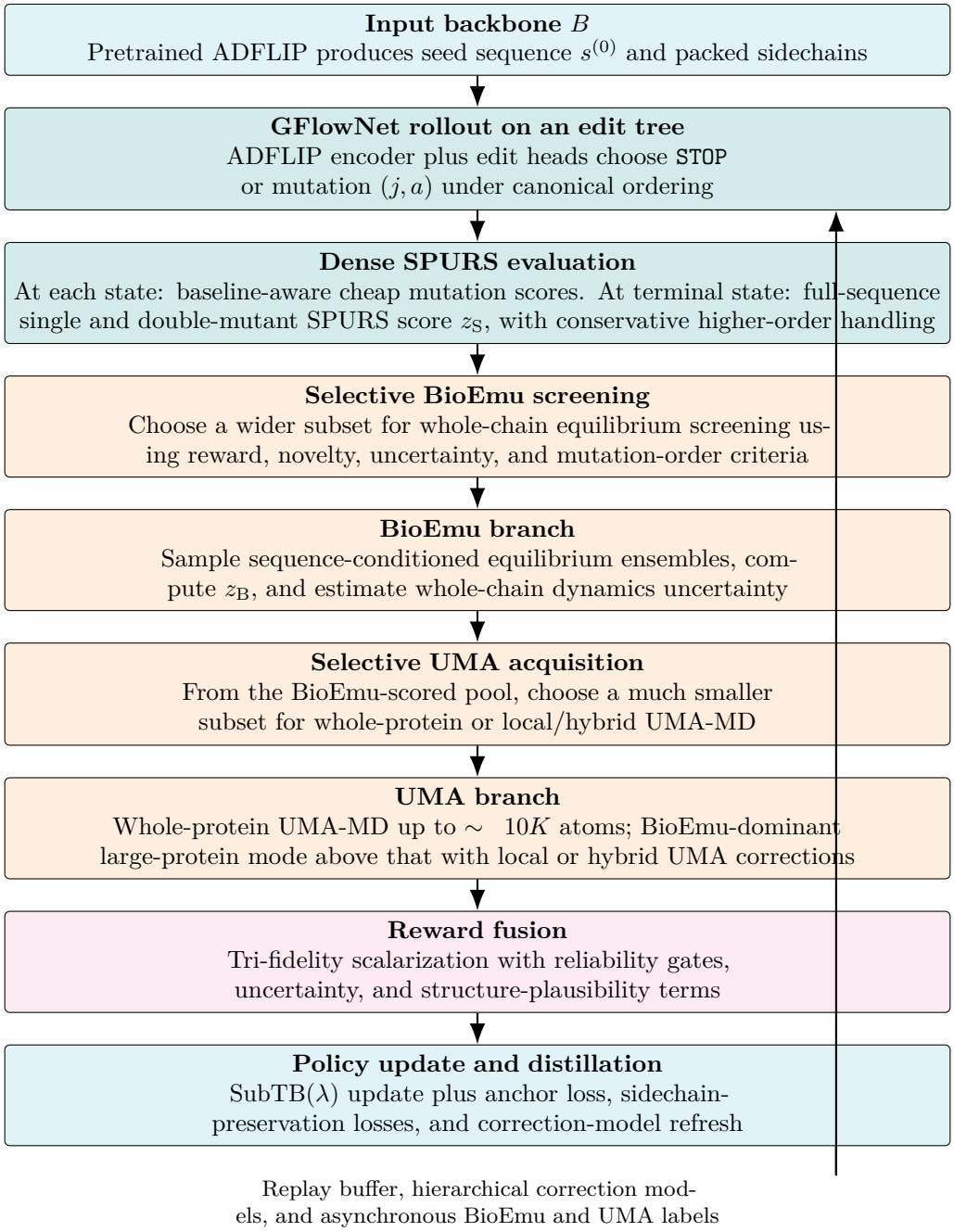


Figure 5: Vertical overview of Simul-MF, the simultaneous tri-fidelity training loop. SPURS is dense and cheap, BioEmu is wider and dynamics-aware, and UMA is sparse but physically richer.

candidate is only sent to local or hybrid UMA if its BioEmu score and disagreement profile justify the cost.

We recommend a BioEmu evaluation rate between 15% and 50% of sampled terminal candidates depending on available compute, and a UMA evaluation rate between 1% and 5% overall or between 5% and 20% of the BioEmu-scored pool. In early training, a higher BioEmu rate is usually worthwhile because it quickly stabilizes the dynamics signal. UMA can remain sparse from the start.

10.3 Hierarchical correction models

Because only a subset of candidates receives direct BioEmu or UMA evaluation, we train hierarchical correction models. The first predicts the BioEmu contribution from dense features,

$$\widehat{z}_{\mathrm{B}}^{\mathrm{corr}}(x) = c_{\phi_{\mathrm{B}}}\big(z_{\mathrm{S}}(x), \sigma_{\mathrm{S}}(x), \overline{u_{\mathrm{pack}}}(x), K(x), \mathbf{1}[K(x)=2], \text{sequence features}, \text{cheap structure features}\big). \tag{89}$$

The second predicts the UMA contribution from both lower-fidelity branches,

$$\begin{aligned}\widehat{z}_{\mathrm{U}}^{\mathrm{corr}}(x) = c_{\phi_{\mathrm{U}}}\big(&z_{\mathrm{S}}(x), z_{\mathrm{B}}(x), \sigma_{\mathrm{S}}(x), \sigma_{\mathrm{B}}(x), \overline{u_{\mathrm{pack}}}(x),\\ &K(x), \mathbf{1}[K(x)=2], N_{\mathrm{atom}}(x), \rho_{\mathrm{B}}(x), \rho_{\mathrm{U}}(x), \text{whole-protein and global structure features}\big).\end{aligned} \tag{90}$$

We recommend starting with ridge ensembles because they remain robust under small label counts and make uncertainty estimation straightforward. If larger labeled sets accumulate, gradient-boosted regressors or shallow neural residual models can replace them.

The key point is hierarchical amortization. BioEmu learns a dynamics correction over SPURS. UMA then learns a physically richer correction over the combination of SPURS and BioEmu. This is substantially better matched to the intended oracle hierarchy than forcing the UMA residual to be predicted directly from SPURS alone.

10.4 Why Simul-MF can work

The simultaneous method exploits the asymmetry between the oracles. SPURS annotates and screens a large number of states while ADFLIP itself remains the sole proposal policy. BioEmu supplies whole-chain equilibrium information at a cost low enough to screen a nontrivial fraction of candidates. UMA anchors the reward on a much smaller but strategically chosen subset. The correction models then amortize what the higher-fidelity branches have learned so far. In effect, Simul-MF turns sparse expensive evaluations into a denser supervisory signal while remaining explicit about uncertainty and branch-specific scope.

This method benefits especially from temperature-conditioning. We recommend sampling a reward temperature $\beta \sim \mathrm{Uniform}[0.75, 1.75]$ during training. Lower values explore flatter reward distributions and are particularly useful early in training or when the BioEmu and UMA branches are still immature. Higher values increase exploitation once reward fusion is better calibrated. Temperature-conditioning across β can be implemented using the logit-scaling strategy of Kim et al. [11], although a simpler learned embedding of β is also acceptable for a first implementation.

10.5 Detailed training schedule

A practical schedule for Simul-MF is as follows.

1. **Baseline and full-redesign warm start.** For each sampled training object, first cache baseline oracle values for the starting undesigned sequence or complex. Then train the ADFLIP policy and newly initialized edit heads with *full fine-tuning* of the non-frozen ADFLIP blocks, using the first full-redesign proposal and single-mutant trajectories under SPURS-only reward. No LoRA is used in the primary schedule.

2. **Explicit double-mutant stage.** Up-weight replay from $K = 2$ trajectories and introduce SPURS double-mutant epistasis labels on the full candidate sequence. This is the primary cheap-epistasis stage and should be treated as mandatory in the main method.

3. **Joint SPURS plus BioEmu phase.** Turn on the BioEmu acquisition branch and the BioEmu correction model. Continue full fine-tuning with SubTB(λ) and the auxiliary losses in Eq. (25). Candidates with $K \geq 3$ are now common, but they are mainly shaped by BioEmu and not by trusted higher-order SPURS supervision.

4. **Full tri-fidelity phase.** Turn on UMA acquisition from the BioEmu-screened pool, refresh the UMA correction model periodically, and continue training with the full fused reward. Systems in the 8000–10000 atom range keep UMA active with a soft downweight when calibration weakens; the training corpus does not include systems beyond roughly 10 000 atoms.

5. **Optional higher-order SPURS ablation.** As an ablation, replace the coarse $K \geq 3$ SPURS statistic with SPURS's own higher-order decoder on the full candidate sequence and repeat the same schedule. This is not part of the primary method.

6. **Stabilization phase.** If packing quality degrades, increase the anchor and packing penalties temporarily. If the policy collapses to trivial near-seed sequences, reduce any active radius bias, lower the stop-action prior, or widen the temperature range.

Recommended optimizer defaults are AdamW with learning rate 1×10^{-4} for newly initialized heads and 2×10^{-5} to 5×10^{-5} for the unfrozen pretrained ADFLIP blocks, with cosine decay and 5% warmup. A parameter-efficient LoRA or adapter version should be reported only as an ablation against this full-fine-tuning baseline.

10.6 Catalytic-oracle augmentation of Simul-MF

When the task is reaction conditioned, Simul-MF gains a catalytic branch in parallel with the thermostability and binding branches. Let $\mathcal{R}$ be the reaction context for a candidate sequence x. The first catalytic acquisition stage mirrors the wide-stage logic already used for BioEmu: annotate a broad set of terminal candidates with a cheap or medium-cost catalytic score before sending only a narrower subset to the structure-aware branch. A simple wide-stage acquisition score is

$$A_{\text{cat,wide}}(x, \mathcal{R}) = \alpha_1^{\text{cat}} d_{\text{nov}}(x) + \alpha_2^{\text{cat}} u_{\text{pack}}(x) + \alpha_3^{\text{cat}} \frac{K(x)}{L} + \alpha_4^{\text{cat}} \mathbf{1}[K(x) \geq 2], \tag{91}$$

which favors novel, structurally plausible, and nontrivial edits rather than only trivial near-seed variants. Candidates with valid substrate metadata are eligible for KcatNet, and candidates with valid product or curated missing-modality patterns are also eligible for MMKcat. The resulting KcatNet/MMKcat labels define the wide catalytic screen.

The second catalytic stage is GraphKcat refinement. Define

$$A_{\text{cat,struct}}(x, \mathcal{R}) = \beta_1^{\text{cat}} z_{\text{cat,wide}}^{\text{risk}}(x, \mathcal{R}) + \beta_2^{\text{cat}} d_{\text{nov}}(x) + \beta_3^{\text{cat}} \mathbf{1}[K(x) \geq 2], \tag{92}$$

where $z^{\text{risk}}_{\text{cat,wide}}$ denotes the fused wide-stage KcatNet/MMKcat score before GraphKcat refinement. Only candidates with admissible protein structure and ligand geometry are sent onward. GraphKcat then provides the pocket-aware $\log_{10} k_{\text{cat}}$, $\log_{10} K_{\text{M}}$, and $\log_{10}(k_{\text{cat}}/K_{\text{M}})$ channels, which are fused through Eq. (74). In other words, catalytic training in Simul-MF mirrors the stability stack structurally: a dense wide screen first, then a narrower structure-aware refinement stage, then unified reward fusion.

11 Method II: sequential curriculum from cheap exploration to dynamics and catalytic refinement

11.1 Conceptual overview

Seq-Curr is the staged alternative to Simul-MF. Instead of exposing every oracle family throughout a single online optimization loop, it introduces them in sequence. The cheap stage first establishes broad coverage with SPURS and, on catalytic tasks, KcatNet or MMKcat. The middle stage then introduces BioEmu or wide catalytic fusion, depending on whether the task is stability dominated or reaction conditioned. The final stage introduces the narrowest structure-aware branches: UMA for physical stability refinement and GraphKcat for pocket-aware kinetic refinement. This staged view is especially attractive when the narrow branches are scarce, slow, or not yet trustworthy enough to shape the full replay from the beginning.

11.2 Pseudocode

Algorithm 3 SEQ-CURR: sequential curriculum from SPURS exploration to BioEmu and UMA refinement

Require: Pretrained ADFLIP θ_0, SPURS oracle $\mathcal{O}_{\mathrm{S}}$, BioEmu oracle $\mathcal{O}_{\mathrm{B}}$, UMA-MD oracle $\mathcal{O}_{\mathrm{U}}$

Stage 1A: baseline-aware full redesign and single-mutation exploration

Cache oracle values on each starting undesigned example and train the full-finetuned ADFLIP edit GFlowNet with SPURS-based reward only

Stage 1B: explicit double-mutant epistatic stage

Up-weight replay from $K = 2$ candidates and train with SPURS full-sequence double-mutant supervision

Save model $\theta^{(1)}$ and replay buffer $\mathcal{B}^{(1)}$

Stage 2: BioEmu refinement

Select top and diverse candidates from $\mathcal{B}^{(1)}$, stratified by mutation order and target-conditioning bucket, for BioEmu screening

Run BioEmu on the selected pool and calibrate the resulting ensemble-derived score on a held-out calibration set

Continue GFlowNet training from $\theta^{(1)}$ using BioEmu-weighted reward and a slightly narrower β range

Save refined model $\theta^{(2)}$ and replay buffer $\mathcal{B}^{(2)}$

Stage 3: UMA refinement

Select top and disagreement-rich candidates from $\mathcal{B}^{(2)}$ for direct UMA evaluation

Run whole-protein UMA-MD for candidates in the working size setting and local or hybrid fallback otherwise; calibrate the resulting thermostability score on a held-out calibration set

Continue GFlowNet training from $\theta^{(2)}$ using BioEmu-plus-UMA weighted reward and the narrowest β range

Stage 4: optional deployment distillation

Distill reward-weighted high-scoring candidates into ADFLIP's native sequence head if a fast deployable generator is desired

12 Method III: round-based active learning with a one-shot ADFLIP student

12.1 Why a third method is needed

The first two methods in this manuscript are edit-trajectory methods. They are appropriate when the user wants an explicit mutation path, when per-step credit assignment is central, or when the dynamics oracles can be integrated naturally into or after an edit policy. There is, however, a second use case that deserves a different design: high-throughput proposal generation across many backbones, where the final deployment system should output complete multi-mutant variants in a single forward pass rather than by walking an edit tree.

That use case is directly connected to earlier biological sequence-design work with GFlowNets. Jain et al. proposed a round-based active-learning algorithm in which a GFlowNet acts as a generator of diverse biological sequences, is trained from both reward evaluations and off-policy trajectories reconstructed from an existing dataset, and is coupled to uncertainty-aware acquisition so that each round proposes a diverse, informative batch rather than a single maximizer [8]. In the same paper,

the authors explicitly identified non-autoregressive generators and multi-oracle outer loops as natural future extensions [8]. The original GFlowNet paper also framed the approach as non-iterative diverse candidate generation in the sense that search is amortized during training and i.i.d. samples can then be drawn rapidly at deployment [1]. OneShot-AL is the adaptation of that idea to ADFLIP, SPURS, BioEmu, UMA, and the catalytic-oracle bundle.

The key distinction is the following. The *ideal* OneShot-AL uses a GFlowNet internally as the reward-proportional teacher, because the target distribution over complete sequences is multimodal and expensive. But the deployed proposal model is a one-shot student. For a given backbone and seed sequence, it predicts mutation count, mutation locations, and replacement residues in parallel, then runs a single structural realization step. There is no edit-time tree walk at deployment. This makes the method attractive when one wants to screen many candidate proteins or when a downstream workflow prefers a single precomputed batch per design round.

The current repository implementation now instantiates the teacher as a real trajectory-balance GFlowNet over the canonical edit DAG. Let $\tau = (s_0 \to s_1 \to \cdots \to s_T = x)$ denote the canonical edit trajectory from the baseline seed to terminal candidate x, where each nonterminal action factorizes into `STOP` / `continue`, a position choice, and a replacement-residue choice. The implemented forward policy is

$$PF_\theta(a_t \mid s_t) = PF_\theta(\texttt{STOP} \mid s_t) \quad \text{or} \quad PF_\theta(\neg\texttt{STOP} \mid s_t)\, PF_\theta(j_t \mid s_t, \neg\texttt{STOP})\, PF_\theta(a_t^{\text{aa}} \mid j_t, s_t, \neg\texttt{STOP}), \tag{93}$$

with deterministic backward reconstruction under canonical edit ordering. The teacher is trained with a trajectory-balance residual

$$\mathcal{L}_{\text{TB}}(\tau) = \left(\log Z_\theta(s_0) + \sum_{t=0}^{T-1} \log PF_\theta(a_t \mid s_t) - \log R_r(x) \right)^2, \tag{94}$$

where $R_r(x) > 0$ is the round-r scalar reward or risk-adjusted surrogate reward. Off-policy trajectories are reconstructed from labeled candidates in D_r, and on-policy trajectories are sampled from the teacher itself and scored with the current surrogate ensemble. The one-shot student is then distilled from actual teacher samples rather than from empirical frequency tables. This means the repository path now matches the mathematical intent of OneShot-AL: reward-proportional training happens on an explicit trajectory model, while deployment remains one shot.

12.2 Conceptual structure

OneShot-AL has four trainable or updatable components per active-learning round r:

1. a **surrogate ensemble** M_r that predicts the fused thermostability target and its epistemic uncertainty from backbone, seed sequence, and candidate sequence;
2. a **trajectory-balance teacher policy** $\pi_{\theta,r}$ over canonical edit trajectories;
3. a **one-shot ADFLIP student** $q_{\phi,r}$ that learns to imitate the teacher's terminal distribution without explicit edit-time iteration;
4. a **triage stack** in which SPURS is run densely, BioEmu on a wider filtered subset, and UMA on a smaller high-value subset.

The surrogate is needed because an active-learning round should be able to score many candidate sequences cheaply. The teacher is needed because reward-proportional exploration over multi-mutant sequence space is the core reason to use a GFlowNet at all. In the implemented repository path, that role is played by an explicit flow-consistency teacher trained on reconstructed and sampled trajectories rather than by a reward-weighted empirical shortcut. The student is needed because the deployment requirement here is one-shot generation rather than trajectory sampling. In the catalytic specialization, the triage stack becomes packed-structure GraphKcat at the wide prefilter stage and whole-enzyme UMA dynamics on the narrower expensive subset, with fused catalytic labels replacing or augmenting the stability-only reward.

12.3 Teacher training with off-policy data

At round r, let D_r denote the set of all candidates evaluated so far with whichever oracle bundle was available. Teacher training uses both on-policy rollouts and off-policy reconstructed trajectories from the evaluated dataset [8]. For each sequence in D_r, reconstruct a canonical edit trajectory relative to its seed sequence. If a candidate is K mutations away from the seed, the reconstructed trajectory contains exactly those K edits in canonical order followed by stop. These off-policy trajectories stabilize training, improve local support around already interesting mutants, and make efficient use of expensive labels.

The implemented repository path now follows this construction directly. The teacher maintains a learnable $\log Z$ for each seed family, a stop policy over edit depth, seed-conditioned position logits, and residue-replacement logits conditioned on the original residue identity. This is a deliberately compact parameterization, but it is still a genuine GFlowNet teacher because the optimization target is the trajectory-balance residual on explicit trajectories. On-policy teacher rollouts are scored with the current surrogate ensemble, yielding positive surrogate rewards that play the same role as expensive labels but at lower cost.

We recommend a mixture ratio $\gamma_{\text{off}} \in [0.25, 0.60]$ between offline reconstructed trajectories and on-policy teacher rollouts, with a default of 0.5 in early rounds and a decay toward 0.25 once the student begins proposing genuinely novel high-quality candidates. The current repository path uses trajectory balance rather than SubTB, which is reasonable because the canonical edit teacher is intentionally compact and the average mutation orders in the default catalytic loop remain moderate.

12.4 One-shot student parameterization

The one-shot student is designed to generate a full multi-mutant candidate without edit-time iteration. It reuses ADFLIP's multi-scale residue-and-atom encoder but changes the output parameterization. Given a backbone B, seed sequence $s^{(0)}$, seed sidechains $X^{\text{sc},(0)}$, and control vector c, the student outputs three objects in a single forward pass:

$$p_\phi(K \mid B, s^{(0)}, c), \qquad \ell^{\text{mask}}_{1:L} \in \mathbb{R}^L, \qquad \ell^{\text{aa}}_{1:L} \in \mathbb{R}^{L\times 20}. \tag{95}$$

Here $p_\phi(K \mid \cdot)$ is a categorical distribution over mutation count, ℓ^{mask} are per-position logits indicating how likely each position is to be mutated, and ℓ^{aa} are replacement-residue logits. The control vector c can include reward temperature, desired quantile of the teacher distribution, protein-level metadata, and an oracle-mixture tag indicating whether the current round is SPURS-heavy, BioEmu-heavy, or UMA-heavy.

Candidate generation proceeds as follows.

1. Sample a mutation count $K \sim p_\phi(K \mid B, s^{(0)}, c)$.
2. Sample K mutation sites without replacement using Gumbel-top-K on ℓ^{mask}.
3. For each chosen site j, sample a replacement residue from the categorical distribution induced by ℓ_j^{aa} after masking out the seed amino acid.
4. Leave all non-chosen sites unchanged, yielding a complete candidate sequence x in one pass.
5. Run a single structural realization step: either one ADFLIP masked update restricted to the mutated sites or one sidechain-diffusion packing pass conditioned on the completed sequence.

The resulting candidate is therefore a one-shot multi-mutant proposal. The only iterative computation left is oracle evaluation and any optional post hoc rescoring; there is no edit-time tree walk.

12.5 Student losses and distillation objective

Let $\widetilde{p}_r(x)$ denote the normalized terminal distribution induced by the teacher over a candidate pool $\mathcal{C}_r$, for example a mixture of on-policy teacher samples, replay-buffer elites, and evaluated sequences in D_r. The primary distillation loss is a reward-weighted cross-entropy,

$$\mathcal{L}_{\text{distill}}(\phi) = -\sum_{x \in \mathcal{C}_r} \widetilde{p}_r(x) \log q_\phi(x \mid B, s^{(0)}, c), \tag{96}$$

where the one-shot probability factorizes as

$$q_\phi(x \mid B, s^{(0)}, c) = p_\phi(K_x \mid B, s^{(0)}, c)\; q_\phi(M_x \mid K_x, B, s^{(0)}, c)\; q_\phi(x_{M_x} \mid M_x, B, s^{(0)}, c). \tag{97}$$

A practical implementation does not need exact normalization over the full sequence space; it only needs accurate probabilities on the candidate pool. In that setting Eq. (96) can be estimated from minibatches of teacher samples and replay items.

We recommend augmenting the distillation loss with four auxiliary terms:

$$\mathcal{L}_{\text{student}} = \lambda_{\text{distill}}\mathcal{L}_{\text{distill}} + \lambda_K \mathcal{L}_K + \lambda_{\text{rank}}\mathcal{L}_{\text{rank}} + \lambda_{\text{anchor}}\mathcal{L}_{\text{anchor}} + \lambda_{\text{side}}\mathcal{L}_{\text{side}}. \tag{98}$$

- $\mathcal{L}_K$ is a count-calibration loss on $p_\phi(K \mid \cdot)$ using teacher samples and evaluated sequences.
- $\mathcal{L}_{\text{rank}}$ is a pairwise ranking loss so that candidates with higher fused reward receive larger student log-probability than nearby lower-reward candidates from the same backbone.
- $\mathcal{L}_{\text{anchor}}$ keeps the student close to the pretrained ADFLIP denoising distribution on unedited or lightly edited seeds to avoid structural drift.
- $\mathcal{L}_{\text{side}}$ preserves packing quality on the realized candidate structures through clash and packing-consistency penalties.

A good default is $(\lambda_{\text{distill}}, \lambda_K, \lambda_{\text{rank}}, \lambda_{\text{anchor}}, \lambda_{\text{side}}) = (1.0, 0.2, 0.25, 0.5, 0.2)$, but these weights should be tuned on held-out backbones.

12.6 Round structure and evaluation budget

A typical OneShot-AL run uses 5–15 active-learning rounds. Each round has six stages.

1. Fit or update the surrogate ensemble M_r on D_r.
2. Train the teacher policy as a trajectory-balance GFlowNet with acquisition reward R_r and a mixture of off-policy reconstructed trajectories and on-policy surrogate-scored rollouts.
3. Distill the teacher into the one-shot student $q_{\phi,r}$.
4. Sample a large candidate pool from the student, for example 10^4–10^5 candidates across the current backbone minibatch, then filter by chemistry, structure (matching the starting backbone, and with desirable properties such as being well-packed), sequence diversity, and novelty.
5. Run SPURS on all filtered candidates and BioEmu on a larger prioritized subset.
6. Evaluate a compact diverse subset with UMA, append results to D_{r+1}, and continue.

The highest-fidelity evaluation budget is therefore controlled at the round level rather than the step level. This matches the biological-sequence active-learning workflow in Jain et al. more closely than Methods I and II do [8]. It is often the right choice when the expensive branch has low throughput.

The default selection rule for the BioEmu and UMA batches is cluster-balanced top-b. First cluster candidates by sequence identity or mutation-set Jaccard distance. Then within each cluster rank by acquisition score, with an additional preference for higher-order candidates that have not yet received BioEmu or UMA supervision. Use the wider quota for BioEmu and the narrower quota for UMA. This keeps batch diversity high while still favoring high acquisition. In the large-protein setting, it is perfectly reasonable for every selected candidate to receive BioEmu but only a small fraction to receive local or hybrid UMA.

12.7 Why OneShot-AL is genuinely non-iterative at deployment

The GFlowNet teacher remains sequential internally; that is unavoidable because reward-proportional training for compositional discrete objects is naturally expressed on a DAG. The implemented repository path keeps that sequential structure inside the teacher and collapses it only at the student-distillation stage, where teacher samples are summarized into one-shot student outputs. The user-facing generation mechanism therefore remains non-iterative in the practically relevant sense. Once a round is complete, the student produces a complete multi-mutant proposal in one forward pass and one structural realization step. Compared with Methods I and II, this changes three things.

1. **Latency:** proposal generation becomes much faster for large screening batches.
2. **Engineering simplicity:** deployment no longer needs replay buffers, online reward correction, or tree traversal.
3. **Loss of edit-path interpretability:** the student outputs the terminal sequence directly, so explicit trajectory-level explanations are weaker unless one also stores the teacher distribution for attribution.

A practical deployment stack can still run SPURS on the full proposal batch, BioEmu on a filtered subset, and UMA only on the top handful, without changing the fact that the proposal model itself is one-shot.

12.8 Pseudocode

Algorithm 4 OneShot-AL: active-learning teacher policy with one-shot ADFLIP student

Require: Initial dataset D_0, pretrained ADFLIP parameters θ_0, student parameters ϕ_0, round budget R, BioEmu batch size b_{B}, UMA batch size b_{U}

```
for r = 0 to R − 1 do
    Fit surrogate ensemble M_r on D_r to obtain mean μ_r(x) and uncertainty σ_r(x)
    Define acquisition-based reward R_r(x) from μ_r(x), σ_r(x), diversity penalties, and validity filters
    Train teacher policy for round r as a trajectory-balance GFlowNet using reconstructed off-policy trajectories and surrogate-scored on-policy rollouts
    Build candidate pool C_r from teacher samples, replay elites, and evaluated sequences
    Distill teacher distribution on C_r into one-shot student q_{φ,r}
    Sample a large proposal batch B_r from q_{φ,r} in one pass per backbone
    Repack mutated sidechains once, optional BioEmu RMSD validation, and discard invalid structures
    Rescore surviving candidates with SPURS and the surrogate
    Select a diverse subset of size b_B and run BioEmu to obtain whole-chain dynamics scores
    From the BioEmu-scored pool, select a smaller diverse subset of size b_U for UMA
    for each selected candidate x do
        if prepared system fits the whole-protein UMA budget then
            Run whole-protein UMA-MD ladder and compute calibrated expensive score
        else
            Run local or hybrid UMA fallback and compute calibrated expensive score
        end if
        Add evaluated tuple for x to D_{r+1}
    end for
end for
Return final one-shot student q_{φ,R}, teacher replay buffer, and evaluated dataset D_R
```

12.9 Catalytic active learning with whole-enzyme UMA and GraphKcat

The catalytic specialization of OneShot-AL uses the same teacher-student structure but changes the oracle stack and the physical interpretation of the expensive stage. Each candidate is paired with a reaction context $\mathcal{R}$ and, in the default implemented repository path, with two endpoint structures: a reactant-bound complex and a product-bound complex. The surrogate ensemble predicts a fused catalytic score rather than only a thermostability score. The teacher then assigns reward-weighted probability mass to catalytic candidates, and the deployed student remains one shot: it proposes a full enzyme variant in one forward pass, after which packed-structure GraphKcat and whole-enzyme UMA dynamics determine whether the candidate is worth retaining.

The local repository already implements a concrete self-contained orchestration of this pattern. The current round runner first validates that each example carries substrate metadata, endpoint

complex paths, protein chain identifiers, and pocket positions. It then trains or refreshes the surrogate, trajectory-balance teacher, and one-shot student, generates a candidate pool, repacks the full pool on endpoint complexes with LigandMPNN, runs packed-structure GraphKcat across that pool, keeps the top GraphKcat tranche, selects a catalytic batch from within that screened set, runs the broad whole-enzyme UMA screen, runs forward and reverse steered UMA dynamics, optionally launches a path umbrella PMF, fuses the resulting catalytic labels, and finally appends those labels to the active-learning dataset. The mathematical abstraction is the same as in the generic OneShot-AL loop, but the concrete oracle routing is now explicitly dynamical rather than only predictive.

The corresponding catalytic acquisition scores reflect this asymmetry. For the broad catalytic stage we recommend

$$A_{\mathrm{UCAT}}(x, \mathcal{R}) = \alpha_1^{\mathrm{UCAT}} d_{\mathrm{nov}}(x) + \alpha_2^{\mathrm{UCAT}} u_{\mathrm{pack}}(x) + \alpha_3^{\mathrm{UCAT}} \frac{K(x)}{L} + \alpha_4^{\mathrm{UCAT}} \mathbf{1}[K(x) \geq 2], \tag{99}$$

which preserves diversity and mutation-order coverage before the expensive catalytic screen. For the narrower GraphKcat stage we recommend

$$A_{\mathrm{GK}}(x, \mathcal{R}) = \beta_1^{\mathrm{GK}} z_{\mathrm{UCAT}}^{\mathrm{risk}}(x, \mathcal{R}) + \beta_2^{\mathrm{GK}} d_{\mathrm{nov}}(x) + \beta_3^{\mathrm{GK}} \mathbf{1}[K(x) \geq 2]. \tag{100}$$

Only candidates whose protein structure and ligand geometry can be materialized are eligible for GraphKcat. This design keeps the narrower learned catalytic branch focused on cases where its structural inductive bias is actually usable, while the broad dynamical stage remains the default source of catalytic signal.

Algorithm 5 Catalytic OneShot-AL round with whole-enzyme UMA and GraphKcat

Require: Active-learning dataset D_r, reaction metadata, docked reactant-bound and product-bound endpoint complexes, pretrained generator, broad UMA-cat budget b_{UCAT}, narrower GraphKcat budget b_{GK}

Validate substrate metadata, endpoint complex paths, protein chain IDs, and pocket positions
Fit surrogate ensemble M_r on fused catalytic labels from prior rounds
Train teacher policy on catalytic or joint catalytic-stability reward as a trajectory-balance GFlowNet over canonical edit trajectories
Distill teacher terminal distribution into one-shot student $q_{\phi,r}$
Sample a large candidate pool from $q_{\phi,r}$ and apply structural plausibility filters
Select a broad candidate subset by A_{UCAT}
Repack each selected candidate onto reactant-bound and product-bound complexes with LigandMPNN
Run whole-enzyme UMA equilibrium screening on the reactant-bound complexes to estimate productive-pose occupancy and $\widehat{\Delta G}_{\mathrm{gate}}$
Run forward and reverse steered whole-enzyme UMA dynamics between the reactant-bound and product-bound complexes
Optionally seed a path umbrella PMF from the steered trajectories and estimate $\widehat{\Delta G}^{\ddagger}_{\mathrm{PMF}}$
Convert the catalytic dynamics outputs into $\widehat{\ell}_{\mathrm{UMAcat}}$ through Eq. (70)
Select a narrower structure-qualified subset by A_{GK} and run GraphKcat
Fuse UMA-cat and GraphKcat through Eq. (74) to obtain catalytic rewards
Append the newly labeled tuples to D_{r+1} and continue to the next round

12.10 Recommended defaults for the first implementation

For a first implementation of OneShot-AL, we recommend 8 active-learning rounds, a teacher-training phase of 20k–40k gradient steps per round, and a student-distillation phase of 10k–20k steps per round. Use 8–16 surrogate ensemble members if compute allows; otherwise use 4 members and increase the conservative exploration bonus κ_r. Set the default round budget so that every selected candidate receives SPURS, at least 20–40% of the filtered round pool receives BioEmu, and at least 30% of the UMA batch comes from candidates whose mutation order exceeds the median mutation order in D_r. This prevents the active-learning loop from collapsing to small edits. Use a higher off-policy fraction in the first two rounds and decay it later. Because the student is one-shot, it is worth oversampling candidate proposals during rounds 4–8 and tightening cluster-based selection rather than narrowing the student itself too early.

12.11 Instrumentation, logging, and reproducibility

Multi-oracle active learning is difficult to debug if observability is treated as an afterthought. We therefore recommend that every round expose both optimization diagnostics and oracle-stage diagnostics as first-class outputs. In the current repository implementation, this instrumentation is not merely advisory; it is part of the default runtime path. Let u index teacher updates and let v index student-distillation updates. The teacher logs a telemetry vector

$$m_u^{\text{teacher}} = \big(\mathcal{L}_{\text{TB}}, \mathcal{L}_{\text{off}}, \mathcal{L}_{\text{on}}, \mathcal{L}_{\text{reg}}, \delta_{\text{abs}}, \|\nabla\|_{\text{pre}}, \|\nabla\|_{\text{post}}, \bar{p}_{\text{stop}}, \overline{\log Z}, \eta\big)_u, \tag{101}$$

where η is the learning rate and δ_{abs} is the absolute TB residual. The student logs

$$m_v^{\text{student}} = \big(f_{\text{sampled}}, \mathbb{E}[K], H[K], N_{\text{distinct } K}, N_{\text{seed families}}\big)_v, \tag{102}$$

which exposes whether the one-shot distillation path is collapsing to a narrow mutation-order regime or a narrow set of seed families. The surrogate logs bootstrap-level statistics rather than only a final checkpoint summary, allowing the user to detect whether the ensemble members are seeing materially different resampled data.

The same principle applies to the oracle stages. For each stage $\mathcal{O} \in \{\text{GraphKcat}, \text{UMAcat}\}$, the repository now emits a structured summary

$$m_r^{\mathcal{O}} = \big(N_{\text{attempted}}, N_{\text{ok}}, f_{\text{ok}}, \mathbb{E}[\hat{y}_{\mathcal{O}}], \text{Var}[\hat{y}_{\mathcal{O}}], \text{stage-specific failure counts}\big)_r, \tag{103}$$

for each active-learning round r. At the round level, the orchestration layer also records gate outcomes, append summaries, and teacher-student evaluation summaries. The intended use is twofold. First, these quantities make training failures diagnosable rather than anecdotal. Second, they make experimental reporting more credible because a later empirical paper can distinguish optimization failure, oracle failure, and data-censoring failure rather than collapsing them into a single negative result.

In the repository implementation, these telemetry streams are written both to structured JSON/JSONL artifacts and to Weights & Biases. The default runtime uses an automatic W&B mode: it syncs online when credentials are available and falls back to offline artifact logging otherwise, so that telemetry is preserved on both connected and restricted machines. Progress reporting is likewise layered: an experiment-level progress bar tracks rounds, a round-level progress bar tracks orchestration stages, and the trainer and oracle subprocesses expose their own detailed progress bars or heartbeat-style logs. This is a mundane implementation detail compared with the oracle physics, but it is essential if the methodology is to be reproducible and if expensive catalytic runs are to be trusted rather than guessed at.

12.12 Expected strengths and likely failure modes

OneShot-AL should excel when the lab needs large, diverse proposal batches and the highest-fidelity oracle can only be queried in coarse rounds. It is also well aligned with industrial screening workflows, where one often wants tens of thousands of proposals reduced to a few hundred computationally filtered sequences and then a few dozen experimental candidates. The main risk is student undercoverage of thin but valuable teacher modes, especially at high mutation order. That risk is manageable if the round-level evaluation keeps measuring teacher-student KL on held-out candidate pools and if the student is retrained whenever that gap widens beyond a preset tolerance.

A second risk is that the one-shot factorization of mutation count, mask, and amino-acid identities may underrepresent long-range dependencies that an edit-trajectory policy captures naturally. When this happens, one can either enrich the student with low-rank pairwise coupling heads over mutated sites or allow a tiny fixed number of masked refinement passes after the initial one-shot proposal. The latter does not change the core character of the method: the student remains a one-shot proposer, and any refinement is a bounded post-processing step rather than a sequential search policy.

13 BioEmu protocol as a fast dynamics oracle and large-protein complement

13.1 Why a dedicated BioEmu protocol is required

BioEmu is not merely another scalar stability regressor. It is a sequence-conditioned generative equilibrium-ensemble model [16]. That changes how it should be integrated into a design pipeline. A scalar predictor can be dropped into a reward function with only calibration and uncertainty estimation. An equilibrium-ensemble model provides a much richer object: a distribution over backbone conformations. The challenge is to transform that ensemble into a scientifically meaningful thermostability signal that is aligned with the fixed target backbone and with the multi-mutant design objective of the present manuscript.

The BioEmu paper provides strong motivation for this effort. It reports that BioEmu can generate thousands of statistically independent samples from a protein structure ensemble per hour on a single GPU, that it qualitatively captures conformational changes such as domain motions, local unfolding, and cryptic-pocket formation, and that it quantitatively matches equilibrium observables with free-energy errors on the order of 1 kcal/mol in several benchmark settings [16]. The same paper also reports direct stability prediction through generated ensembles, with folding free-energy mean absolute error below 0.8 kcal/mol on its train/test splits [16]. These are precisely the properties that make BioEmu attractive as a fast dynamics oracle.

13.2 Eligibility and scope

The protocol in this manuscript uses BioEmu only where the model's public evidence supports it. The default BioEmu-eligible setting is:

1. soluble proteins or chain-wise approximations of larger systems;
2. single-chain proteins, or multi-chain settings reduced to a designated design chain when the missing partners are not expected to dominate the fold;

3. no essential membrane environment and no stability mechanism that fundamentally depends on an explicit ligand absent from the BioEmu input.

If any of these conditions fail, BioEmu should be treated as a weak exploratory prior or omitted entirely. This is not a weakness of the methodology; it is explicit scope control.

Within that scope, BioEmu is especially useful for proteins whose prepared all-atom systems exceed roughly 8000–10000 atoms. In that setting, one often still wants a whole-chain dynamics signal, but forcing every candidate through whole-protein UMA-MD may be scientifically or computationally inefficient. BioEmu is then the natural first-line dynamics oracle, with UMA reserved for local or hybrid follow-up for larger system until it is validated further.

13.3 From BioEmu samples to thermostability features

Given candidate sequence s, generate M_{B} BioEmu samples

$$\mathcal{Y}(x) = \{y_m\}_{m=1}^{M_{\mathrm{B}}}, \qquad y_m \sim p_{\mathrm{BioEmu}}(\cdot \mid s).$$

For training-time ranking, we recommend $M_{\mathrm{B}} = 2048$ by default. For shortlisted candidates or calibration experiments, $M_{\mathrm{B}} = 10{,}000$ is a sensible target because the BioEmu paper itself uses 10k-sample ensembles in several analyses [16]. Each sample is compared to the target backbone using the observables defined earlier:

$$Q_{\mathrm{nat}}^{\mathrm{B}}, \quad \mathrm{RMSD}_{\mathrm{bb}}^{\mathrm{B}}, \quad H_{\mathrm{sec}}^{\mathrm{B}}, \quad R_g^{\mathrm{B}}, \quad U_b^{\mathrm{B}}.$$

The first four define global fold retention. The block-local unfolding scores U_b^{B} define where the ensemble destabilizes relative to the mutation pattern. We also recommend recording optional motif-specific indicators such as loop opening, hinge variance, or pocket-volume proxies whenever the target fold class makes them relevant.

The core BioEmu stability statistic is the two-basin estimator in Eq. (55). However, the full feature vector

$$u_{\mathrm{B}}(x) = \left[\widehat{\Delta G}_{\mathrm{stab}}^{\mathrm{BioEmu}},\ p_{\mathcal{F}}^{\mathrm{B}},\ p_{\mathcal{U}}^{\mathrm{B}},\ \overline{Q_{\mathrm{nat}}^{\mathrm{B}}},\ \overline{H_{\mathrm{sec}}^{\mathrm{B}}},\ \overline{R_g^{\mathrm{B}}},\ \max_b \overline{U_b^{\mathrm{B}}},\ \mathrm{Var}(R_g^{\mathrm{B}})\right]$$

is more informative for calibration. In particular, it is common to see candidates with similar folded-basin occupancy but very different local-unfolding behavior around mutated regions. Those candidates should not be treated as thermodynamically equivalent for design purposes.

13.4 Calibration and uncertainty

Calibrate BioEmu-derived features to experimental stability on held-out proteins. We recommend a monotone regressor as the default because one expects, all else equal, larger folded-basin occupancy and higher contact retention to imply greater stability. If training data are sparse, ridge regression with spline features is a strong baseline. If larger calibration sets are available, monotone gradient-boosted trees are attractive.

Uncertainty should combine at least three components:

1. **Sample uncertainty:** bootstrap variance over the finite BioEmu sample set.

2. **Model uncertainty:** checkpoint or ensemble disagreement if multiple BioEmu parameter snapshots are retained.

3. **Classifier uncertainty:** threshold sensitivity of the folded/disrupted basin definition.

Candidates with high BioEmu uncertainty should not be discarded automatically. They are often precisely the ones worth sending to UMA, because disagreement between a fast equilibrium emulator and a local or global MLIP-MD refinement is informative.

13.5 How BioEmu and UMA interact above the whole-protein UMA setting

For proteins whose prepared systems exceed roughly 8000–10000 atoms, the recommended dynamic-screen workflow is:

1. run SPURS densely to define the local mutation landscape;

2. run BioEmu on the widest feasible filtered candidate pool to obtain whole-chain equilibrium diagnostics;

3. select a small subset for local or hybrid UMA based on BioEmu score, BioEmu-SPURS disagreement, mutation order, and mechanistic interest;

4. fuse the three branches with reliability gating so that BioEmu carries the dominant whole-chain dynamics weight and UMA contributes targeted physical corrections.

14 UMA-MD protocol as the high-fidelity oracle

14.1 Why a dedicated UMA protocol is still required

It is tempting to say "run UMA MD on the protein and read off ΔG ." That is not rigorous enough. Thermostability is a free-energy property. Even with a strong MLIP, ΔG depends on sampling, basin definitions, protonation handling, environment specification, and the system-size setting in which the model is deployed. The purpose of this section is to spell out a protocol that is technically defensible and aligned with the requirement that UMA be used on whole proteins whenever practical, while also explaining exactly where BioEmu should take over as the main whole-chain dynamics oracle.

The strongest published facts we can lean on are the following. OMol25 consists of more than 100 million DFT calculations at the ωB97M-V/def2-TZVPD level, with broad chemistry including biomolecules and explicit solvation, and systems up to ~ 350 atoms [15]. UMA is trained on half a billion unique 3D atomic structures across multiple domains, and a single model can be used through the FAIR Chemistry stack to run domain-specific prediction and MD workflows such as molecular MD under the `omol` task [5**?**]. The FAIR Chemistry repository also documents a worked large atom count MD example on 8×H100 GPUs and reports current wall-clock MD throughput around 1 ns/day for 100k+-atom systems [5] on a single H100. This establishes that large-system UMA-MD is computationally feasible.

What remains nontrivial is thermodynamic validity on proteins. The public training data are not a direct benchmark suite for whole-protein folding free energies. We therefore adopt the following principle: use whole-protein UMA-MD up to $\sim 8000 - 10000$ atoms as the default high-fidelity

branch whenever the prepared system fits, but treat the resulting thermodynamic quantities as calibrated stability estimates whose relationship to experimental thermostability must be validated explicitly. Above that scenario, BioEmu is the default whole-chain dynamics oracle and UMA is focused on local or hybrid corrections, unless the context involves multimers or complexes with non-protein systems.

14.2 Prepared-system definition and default operating scenarios

We distinguish four operating settings.

Setting A: BioEmu plus whole-protein UMA up to around 8000 **atoms (default).** If the prepared protein system satisfies $N_{\text{atom}} \leq 8000$ and the protein is BioEmu-eligible, run both whole-chain BioEmu screening and whole-protein UMA-MD. BioEmu helps prioritize which candidates deserve the MLIP-MD budget and provides an independent equilibrium signal. UMA then supplies the higher-fidelity branch.

Setting B: BioEmu-dominant large-protein mode above about 8000 **atoms.** If the prepared whole-protein system exceeds the atom budget or falls into the 8000–10000-atom gray zone where calibration may weaken, use BioEmu as the primary whole-chain dynamics oracle. UMA is still useful, but should be down-weighted compared to BioEmu unless the system is a complex.

Setting C: local or hybrid UMA for targeted mechanistic questions. Even below the atom budget, some candidates may deserve local mechanistic scrutiny around a mutation cluster, catalytic ion, or particularly uncertain region. Extract a local subsystem, cap bonds, retain first-shell waters or ions if chemically required, and use UMA-MD to estimate local free-energy or structural robustness proxies.

Setting D: BioEmu-ineligible cases. If the system fundamentally depends on a membrane, explicit ligand, or multimeric context that BioEmu does not model well, use SPURS plus the appropriate local or hybrid UMA protocol and do not pretend that a BioEmu score exists.

14.3 Pre-processing, protonation, and system construction

Every candidate sequence must be converted into a chemically sensible starting structure before UMA-MD begins. We recommend the following sequence of steps.

1. Start from the ADFLIP terminal full-atom structure and perform one full cleanup or repacking pass if the mean local $u_{\text{pack}}$is above threshold.
2. Assign protonation states at a stated pH (default pH 7.0) using a deterministic pipeline. Histidine tautomer choices must be cached because they affect charge and local energetics.
3. Add missing hydrogens and retain cofactors, catalytic waters, or structural ions when these are part of the intended fold or stabilization mechanism.
4. Compute the prepared-system atom count N_{atom}. If $N_{\text{atom}} \leq 8000$, proceed to setting A whole-protein UMA-MD. If $N_{\text{atom}} > 8000$, trigger the BioEmu-dominant large-protein mode, run down-weighted UMA, or only run local or hybrid UMA as needed for more than $10K$ atoms.

5. For local fallback systems, cap cut peptide bonds with ACE/NME or equivalent neutral caps, neutralize terminal artifacts when possible, and record total charge and spin multiplicity explicitly because UMA's molecular interface supports charge and spin annotations [5].

6. Exclude metal-containing proteins from the default benchmark unless oxidation state and spin state can be assigned robustly. Metal chemistry is scientifically important, but it introduces enough extra ambiguity that it should be benchmarked separately.

14.4 Whole-protein MD observables for thermostability

For whole-protein UMA runs, compute the global observables already used in the reward construction:

$$Q_{\mathrm{nat}}^{\mathrm{U}}(t) = \frac{1}{|C_{\mathrm{nat}}|} \sum_{(i,j)\in C_{\mathrm{nat}}} \mathbf{1}\left[d_{ij}(t) \le \lambda_Q d_{ij}^{(0)}\right], \tag{104}$$

$$\mathrm{RMSD}_{\mathrm{bb}}^{\mathrm{U}}(t) = \text{backbone RMSD to the starting structure}, \tag{105}$$

$$R_{\mathrm{g}}^{\mathrm{U}}(t) = \text{radius of gyration}, \tag{106}$$

$$H_{\mathrm{sec}}^{\mathrm{U}}(t) = \text{secondary-structure retention fraction}, \tag{107}$$

$$S_{\mathrm{hb}}(t) = \text{fraction of stability-critical hydrogen bonds retained}. \tag{108}$$

Here C_{nat} is the set of native heavy-atom contacts in the starting structure and λ_Q is a slack factor (default 1.2). The hydrogen-bond term S_{hb} is optional but useful for motifs such as helical capping or beta-turn stabilization.

We then define folded and disrupted basins using conservative thresholds:

$$\mathcal{F}_{\mathrm{U}} = \left\{Q_{\mathrm{nat}}^{\mathrm{U}} \ge q_F,\ \mathrm{RMSD}_{\mathrm{bb}}^{\mathrm{U}} \le r_F,\ H_{\mathrm{sec}}^{\mathrm{U}} \ge h_F\right\}, \tag{109}$$

$$\mathcal{U}_{\mathrm{U}} = \left\{Q_{\mathrm{nat}}^{\mathrm{U}} \le q_U \text{ or } \mathrm{RMSD}_{\mathrm{bb}}^{\mathrm{U}} \ge r_U \text{ or } R_{\mathrm{g}}^{\mathrm{U}} \ge g_U\right\}, \tag{110}$$

with defaults $(q_F, q_U, r_F, r_U, h_F) = (0.75, 0.35, 2.5\,\text{Å}, 6.0\,\text{Å}, 0.7)$. These thresholds should be validated on the calibration set and then frozen before final evaluation.

At temperature T, let $p_{\mathcal{F}_{\mathrm{U}}}(T)$ and $p_{\mathcal{U}_{\mathrm{U}}}(T)$ be the occupancy estimates of the folded and disrupted basins from the trajectory ensemble. The two-state free-energy estimate is

$$\widehat{\Delta G}_{\mathrm{stab}}^{\mathrm{UMA}}(T) = k_B T \log \frac{p_{\mathcal{F}_{\mathrm{U}}}(T)}{p_{\mathcal{U}_{\mathrm{U}}}(T)}. \tag{111}$$

This quantity has a clear thermodynamic interpretation, but it can be noisy if basin crossings are rare. We therefore pair it with a temperature-ladder summary.

14.5 Temperature ladder and thermostability summaries

Thermostability is inherently temperature-dependent, so the high-fidelity branch should not collapse all information to a single 300 K quantity. For whole-protein UMA-MD we recommend a default ladder

$$T \in \{300, 330, 360, 390, 420\}\,\mathrm{K},$$

with the exact range tuned to the expected fold class. At each temperature, run replicated trajectories and estimate $p_{\mathcal{F}_{\mathrm{U}}}(T)$. Fit a logistic curve

$$p_{\mathcal{F}_{\mathrm{U}}}(T) \approx \frac{1}{1 + \exp\left((T - T_{1/2}^{\mathrm{UMA}})/s_T\right)} \tag{112}$$

to obtain a midpoint temperature $T_{1/2}^{\mathrm{UMA}}$ and slope s_T. The midpoint is not automatically identical to the experimental melting temperature T_m, but after calibration it can serve as a valuable thermostability feature.

The default whole-protein MD feature vector is

$$u_{\mathrm{U}}(x) = \Big[\widehat{\Delta G}_{\,300}, \widehat{\Delta G}_{\,330}, \widehat{\Delta G}_{\,360}, \widehat{\Delta G}_{\,390}, \widehat{\Delta G}_{\,420}, T_{1/2}^{\mathrm{UMA}}, s_T, \mathrm{AUC}\big(p_{\mathcal{F}_{\mathrm{U}}}(T)\big), \bar{Q}_{\mathrm{nat}}^{\mathrm{U}}, \bar{H}_{\mathrm{sec}}^{\mathrm{U}}, \bar{R}_{\mathrm{g}}^{\mathrm{U}}, \bar{S}_{\mathrm{hb}}\Big]. \tag{113}$$

A calibration model g_ψ then maps these MD-derived features to a scalar thermostability target in experimental units:

$$\widehat{\Delta G}_{\,\mathrm{stab,cal}}^{\,\mathrm{UMA}}(x) = g_\psi(u_{\mathrm{U}}(x))\,. \tag{114}$$

A monotone regressor, monotone gradient-boosted tree, or ridge model with spline basis functions is recommended.

14.6 Recommended whole-protein MD settings

The FAIR Chemistry examples for molecular MD under UMA use ASE's Langevin dynamics with a timestep of 0.1 fs [5]. We adopt that value as the conservative default. For whole-protein screening runs, we recommend the following baseline protocol:

- energy minimize until maximum force < 0.05 eV/Å,
- equilibrate for 5 ps at each temperature with mild harmonic restraints on backbone heavy atoms during the first half of equilibration,
- release or weaken restraints and run 25 ps of production at each temperature in the screening ladder,
- use 4 replicate trajectories per temperature with distinct random seeds,
- collect snapshots every 10 to 50 steps depending on storage limits,
- for shortlisted candidates, extend production to 100 ps or more per temperature and, when feasible, perform enhanced sampling around the folded/disrupted transition region.

These settings are proposed defaults for ranking and policy training. Their role is to provide a calibrated higher-fidelity signal than the cheaper branches, not to solve the full protein-folding problem in one step.

14.7 Enhanced sampling for shortlisted candidates

For top candidates or calibration subsets where spontaneous basin crossing remains rare, use enhanced sampling. Two recommended options are:

Umbrella sampling.
Define a reaction coordinate ξ such as $Q_{\mathrm{nat}}^{\mathrm{U}}$, $\mathrm{RMSD}_{\mathrm{bb}}^{\mathrm{U}}$, or a two-dimensional coordinate $(Q_{\mathrm{nat}}^{\mathrm{U}}, R_g^{\mathrm{U}})$. Run umbrella windows and combine them with WHAM or MBAR to estimate a potential of mean force.

Temperature exchange or stratified replicas.
When supported by the workflow, distribute replicas across the temperature ladder and combine occupancy estimates with MBAR. This is particularly attractive because thermostability is already a temperature-indexed property.

For the highest-value candidates, report both the screening estimate and the enhanced-sampling estimate, and keep the discrepancy as an uncertainty channel rather than silently replacing one with the other.

14.8 Fallback local and hybrid protocols

If the prepared system exceeds the whole-protein budget, or if explicit solvent is essential and cannot be represented within the atom budget, use BioEmu as the default whole-chain dynamics screen and apply UMA locally or in hybrid form. In that case, define a local folded basin relative to the seed cluster and compute the same two-state estimate as in Eq. (111). The resulting quantity is treated as a local free-energy proxy, not as direct whole-protein folding free energy. A calibration model then maps the local trajectory features to the same scalar readout scale used by the whole-protein branch:

$$\widehat{\Delta G}^{\,\mathrm{UMA}}_{\mathrm{stab,cal}} = g_\psi\Big(\widehat{\Delta G}^{\,\mathrm{UMA}}_{\mathrm{local}},\ \bar{E},\ \mathrm{Var}(E),\ \bar{Q},\ \overline{\mathrm{RMSD}},\ \text{H-bond occupancy},\ \text{SASA features}\Big). \quad (115)$$

When both whole-protein and local-cluster estimates are available, retain both and treat their disagreement as a diagnostic uncertainty term rather than averaging them blindly.

14.9 Oracle confidence and uncertainty decomposition

We recommend storing the following uncertainty channels for every high-fidelity evaluation:

1. **Replicate variance:** variance across random-seed trajectories.
2. **Model disagreement:** difference between UMA small and medium models.
3. **Calibration uncertainty:** residual uncertainty from the mapping g_ψ to experimental units.
4. **Sampling diagnostics:** effective sample size in WHAM/MBAR or occupancy stability across the second half of the trajectory.
5. **setting disagreement:** discrepancy between whole-protein and local/hybrid estimates when both are run.
6. **Cross-oracle disagreement:** discrepancy between calibrated BioEmu and calibrated UMA signals on the same candidate.

Candidates with large uncertainty should not simply be discarded; they are often precisely the candidates most useful for improving the higher-fidelity branch through active selection.

15 Catalytic-oracle protocol for enzyme kinetics and activity-aware design

15.1 Why a dedicated catalytic-oracle protocol is required

Catalytic design differs from thermostability design in a fundamental way: the property of interest is not only a function of the protein sequence and fold, but of a reaction context and a pathway. Turnover depends on the substrate, may depend on products, may depend on assay context such as pH or temperature, and can be highly sensitive to local pocket geometry even when the global fold remains intact. A catalytic design paper that only says "score k_{cat} somehow" is therefore underspecified. The purpose of this section is to make the catalytic branch as explicit as the manuscript already makes the BioEmu and UMA branches.

The present repository now implements a self-contained catalytic protocol with two endpoint complexes per candidate:

$$\mathcal{R} = \big(\mathcal{S}_{\text{sub}}, \mathcal{S}_{\text{prod}}, \eta, A, B\big), \tag{116}$$

where A is the reactant-bound enzyme complex and B is the product-bound enzyme complex. The default implemented catalytic RL path uses these endpoints directly. It does *not* rely only on static regressors. Instead, it combines whole-enzyme UMA equilibrium screening, forward and reverse steered UMA dynamics, optional path umbrella PMFs, and GraphKcat.

15.2 Reaction-conditioned inputs, metadata, and admissibility

Every catalytic example must carry substrate metadata, endpoint structures, and enough local geometry to identify the catalytic pocket. In the current repository, a valid record contains at least:

- a designed enzyme sequence s_x,
- a substrate identifier, usually a SMILES string,
- a reactant-bound complex path,
- a product-bound complex path,
- a protein chain identifier,
- a ligand chain identifier when the ligand is stored as a chain,
- a set of pocket positions.

These are not optional conveniences. The default broad whole-enzyme UMA stage and the default sMD stage cannot run without them. The repository therefore validates them explicitly before catalytic active-learning rounds begin.

Admissibility is branch specific. The whole-enzyme UMA stage is admissible only when the reactant-bound and product-bound endpoint complexes exist and fit the configured atom budget. GraphKcat is admissible only when the candidate enzyme structure can be materialized and a ligand conformer is available or can be embedded reliably enough to extract the catalytic pocket. KcatNet and MMKcat remain admissible whenever the necessary sequence and substrate or multimodal metadata are present, but they are auxiliary predictive branches rather than the default control loop.

15.3 Whole-enzyme UMA equilibrium screening and productive-pose statistics

The first catalytic question is not "what barrier does the chemistry have" but "how often does the full enzyme organize the bound reactants into a productive arrangement?" Let $\rho_A(R)$ denote the equilibrium density in the reactant-bound basin under the whole-enzyme UMA potential. The local implementation uses a default productive feature vector

$$q(R) = \Big(r_{\text{lig}}^{\text{align}}(R), N_{\text{contact}}(R), d_{\min}^{\text{pocket-lig}}(R), r_{\text{pocket}}(R)\Big), \tag{117}$$

where $r_{\text{lig}}^{\text{align}}$ is ligand RMSD after Kabsch alignment on pocket atoms, N_{contact} is the heavy-atom pocket-ligand contact count, $d_{\min}^{\text{pocket-lig}}$ is the minimum pocket-ligand heavy-atom distance, and r_{pocket} is local pocket RMSD.

The corresponding hard productive indicator is

$$I_{\text{gNAC}}(R) = \mathbf{1}\Big[r_{\text{lig}}^{\text{align}}(R) \le r_{\max} \wedge N_{\text{contact}}(R) \ge c_{\min} \wedge d_{\min}^{\text{pocket-lig}}(R) \ge d_{\text{safe}} \wedge r_{\text{pocket}}(R) \le p_{\max}\Big]. \tag{118}$$

With replicated UMA trajectories $R_t^{(k)}$, the productive-pose occupancy estimator is

$$\widehat{p}_{\text{gNAC}} = \frac{1}{\sum_{k=1}^{K} T_k} \sum_{k=1}^{K} \sum_{t=1}^{T_k} I_{\text{gNAC}}\Big(R_t^{(k)}\Big), \tag{119}$$

and the conformational gating free energy is

$$\widehat{\Delta G}_{\text{gate}} = -RT \log\left(\frac{\widehat{p}_{\text{gNAC}}}{1 - \widehat{p}_{\text{gNAC}}}\right). \tag{120}$$

The implementation additionally records soft productive scores, productive visit counts, dwell times, first-hit times, effective sample size, lower confidence bounds on $\widehat{p}_{\text{gNAC}}$, and instability penalties such as persistent undersafe pocket-ligand contacts.

15.4 Forward and reverse steered whole-enzyme UMA dynamics

The second catalytic question is how difficult it is to move the full enzyme-ligand system between the reactant-bound and product-bound basins once those endpoint complexes are known. To probe that pathway, the repository runs forward pulls from A to B and, by default, reverse pulls from B back to A. Let $\mathcal{M}$ be the mapped ligand-atom set shared across the two endpoint complexes after pocket alignment. The steering potential is

$$U_{\text{steer}}(R, t) = \sum_{i \in \mathcal{M}} \frac{k_{\text{steer}}}{2} \left\| r_i - r_i^{\star}(\lambda_t) \right\|^2, \tag{121}$$

where λ_t is the pulling progress parameter and $r_i^{\star}(\lambda_t)$ interpolates between the reactant-bound and product-bound ligand coordinates. To prevent irrelevant global drift, the protocol also applies weak harmonic anchors to non-pocket protein heavy atoms:

$$U_{\text{anchor}}(R) = \sum_{j \in \mathcal{A}_{\text{anchor}}} \frac{k_{\text{anchor}}}{2} \left\| r_j - r_j^{(0)} \right\|^2. \tag{122}$$

The total steered potential is therefore

$$U_{\mathrm{tot}}(R,t) = U_{\mathrm{UMA}}(R) + U_{\mathrm{steer}}(R,t) + U_{\mathrm{anchor}}(R). \tag{123}$$

For each pull, the runtime records cumulative nonequilibrium work,

$$W = \int_0^\tau \frac{\partial U_{\mathrm{steer}}(R,t)}{\partial t}\, dt, \tag{124}$$

and builds a work profile along the pulling schedule. These work traces are not treated as exact equilibrium barriers, but they are highly informative. They reveal whether the path between endpoint basins is mechanically smooth, whether it contains large hysteresis, whether hidden gating coordinates appear, and where near-transition-state-like structures should be harvested.

15.5 Jarzynski profiles, near-transition-state harvesting, and optional path umbrella PMFs

Repeated nonequilibrium pulls permit a free-energy-like reconstruction along the steering schedule. The repository uses a Jarzynski-style estimate

$$\Delta F(\lambda) = -\beta^{-1} \log \left\langle e^{-\beta W(\lambda)} \right\rangle \tag{125}$$

as a default path profile [14]. The forward and reverse work distributions can also be diagnosed through the Crooks fluctuation relation [12]. In practice, the implemented catalytic stage uses these quantities conservatively: as barrier surrogates, as pathway-quality diagnostics, and as a way to extract useful frames rather than as a license to claim exact kinetics.

The repository explicitly harvests near-transition-state-like structures from the steered trajectories. They are not asserted to be the true transition-state ensemble. Instead, they are ranked by local work accumulation, midpoint symmetry along the path, and balanced distance from both endpoint basins. This makes them useful seeds for the optional PMF stage.

When the PMF stage is enabled, the code builds umbrella windows from the sMD-derived path images. Let $x_{\mathcal{M}}(R)$ denote the mapped ligand coordinates in the current frame and $x_j^\star$ the target ligand coordinates for window j. The umbrella bias is

$$U_j(R) = U_{\mathrm{UMA}}(R) + \frac{k_{\mathrm{win}}}{2} \left\| x_{\mathcal{M}}(R) - x_j^\star \right\|^2 + U_{\mathrm{anchor}}(R). \tag{126}$$

Biased samples are then reweighted through an MBAR-style free-energy reconstruction to produce a PMF along the path progress variable. The resulting barrier estimate $\widehat{\Delta G}^{\ddagger}_{\mathrm{PMF}}$ is preferred whenever the PMF succeeds. This use of steered trajectories as PMF seeds closely follows the logic of nonequilibrium-to-equilibrium path reconstruction and Hummer–Szabo-style thinking, while remaining operationally grounded in the actual repository code path [13].

15.6 Catalytic scalar used by the default Method III implementation

The default implemented catalytic scalar is a transition-state-style log-rate proxy:

$$\widehat{\ell}_{\mathrm{UMAcat}} = \log_{10}\left(\frac{k_B T}{h}\right) - \frac{\widehat{\Delta G}_{\mathrm{gate}} + \widehat{\Delta G}_{\mathrm{bar}}}{RT \ln 10}, \tag{127}$$

where

$$\widehat{\Delta G}_{\text{bar}} = \begin{cases} \widehat{\Delta G}^{\ddagger}_{\text{PMF}}, & \text{if PMF is enabled and succeeds,} \\ \widehat{\Delta G}^{\ddagger}_{\text{sMD}}, & \text{otherwise.} \end{cases} \tag{128}$$

This quantity is not claimed to be an exact experimental $\log_{10} k_{\text{cat}}$. It is a physically structured catalytic proxy that penalizes both poor conformational preorganization and large reaction-path barriers. That distinction is important because two enzyme candidates with similar predictive turnover can differ strongly in how often they enter productive poses or how much nonequilibrium work is required to move between endpoint basins.

15.7 GraphKcat as the default learned catalytic refinement branch

GraphKcat remains essential because the dynamical catalytic branch and the learned structural branch answer complementary questions. Whole-enzyme UMA asks whether the full enzyme-ligand system dynamically supports productive organization and a reasonable endpoint-to-endpoint path. GraphKcat asks whether a pocket-aware learned representation of the current enzyme-ligand geometry is consistent with high catalytic turnover and favorable catalytic microenvironment features [17]. In the current repository implementation, GraphKcat is run *first* on LigandMPNN-packed mutant structures, not on the undeformed seed structure. This avoids a misleading prefilter in which catalytic pocket scoring would otherwise be inherited from the seed geometry rather than from the actual mutant-specific packed structure.

The implemented GraphKcat controller also uses model-derived uncertainty rather than a fixed scalar heuristic. With Monte Carlo dropout samples indexed by $s \in \{1, \ldots, S\}$,

$$\widehat{\ell}_{\text{GK}}(x) = \frac{1}{S} \sum_{s=1}^{S} \widehat{\ell}^{(s)}_{\text{GK}}(x), \qquad \widehat{\sigma}^2_{\text{GK}}(x) = \frac{1}{S-1} \sum_{s=1}^{S} \left(\widehat{\ell}^{(s)}_{\text{GK}}(x) - \widehat{\ell}_{\text{GK}}(x) \right)^2. \tag{129}$$

The prefilter score is then

$$z^{\text{risk}}_{\text{GK}}(x) = \widehat{\ell}_{\text{GK}}(x) - \kappa_{\text{GK}} \widehat{\sigma}_{\text{GK}}(x), \tag{130}$$

and only the top fraction of candidates by $z^{\text{risk}}_{\text{GK}}$ are passed onward to the expensive whole-enzyme UMA branch. In the current repository defaults this fraction is 50%. The fused Method III reward then uses the GraphKcat turnover channel together with the UMA-cat log-rate proxy and an agreement penalty. This makes GraphKcat the default learned catalytic refinement oracle for the self-contained catalytic RL path.

15.8 Catalytic uncertainty propagation in log-rate units

The current repository implementation also treats catalytic uncertainty in physically consistent units. Let $\widehat{p}_{\text{gNAC}}$ denote the productive-pose occupancy estimate and $\widehat{\sigma}_p$ its uncertainty. A first-order uncertainty map for the gating free energy is

$$\widehat{\sigma}_{\text{gate}} \approx RT \frac{\widehat{\sigma}_p}{\widehat{p}_{\text{gNAC}}(1 - \widehat{p}_{\text{gNAC}})}. \tag{131}$$

Let $\widehat{\sigma}_{\text{PMF}}$ denote the PMF barrier uncertainty when PMF is enabled, $\widehat{\sigma}_{\text{sMD}}$ the steered-barrier uncertainty otherwise, and let

$$\Delta_{\text{FR}} = \left| \widehat{\Delta G}^{\ddagger,\text{fwd}}_{\text{sMD}} - \widehat{\Delta G}^{\ddagger,\text{rev}}_{\text{sMD}} \right| \tag{132}$$

denote the forward-reverse mismatch. The implemented barrier uncertainty is

$$\widehat{\sigma}^2_{\text{bar}} = \begin{cases} \widehat{\sigma}^2_{\text{PMF}}, & \text{if PMF is enabled and succeeds,} \\ \widehat{\sigma}^2_{\text{sMD}} + \frac{1}{4}\Delta^2_{\text{FR}}, & \text{otherwise.} \end{cases} \tag{133}$$

The final catalytic log-rate uncertainty propagated into reward fusion is therefore

$$\widehat{\sigma}_{\ell_{\text{UMAcat}}} = \frac{\sqrt{\widehat{\sigma}^2_{\text{gate}} + \widehat{\sigma}^2_{\text{bar}}}}{RT \ln 10}. \tag{134}$$

This is a materially stronger formulation than mixing occupancy uncertainty and work variance directly in incompatible units.

15.9 Where KcatNet and MMKcat still fit

KcatNet and MMKcat are not removed from the methodology. KcatNet remains a useful wide predictive turnover screen when only sequence and substrate metadata are available [19]. MMKcat remains the right missing-modality catalytic branch when products or structure channels are incomplete [29]. They are therefore retained as auxiliary branches, ablations, and data-expansion tools. The key change is that they are no longer the default feedback loop for catalytic RL in this repository. The default loop is now whole-enzyme UMA dynamics plus GraphKcat.

15.10 Catalytic-oracle routing in active learning

The active-learning routing logic is intentionally asymmetric. The broad stage is now a real dynamics stage, not merely a predictive filter. First repack the full candidate pool onto reactant-bound and product-bound endpoint complexes and run GraphKcat with MC-dropout on those packed mutant structures. Second, keep the top risk-adjusted GraphKcat tranche and select the expensive catalytic batch from within that screened set. Third, run the broad whole-enzyme UMA screen, then forward and reverse steered UMA dynamics, and optionally a path umbrella PMF. KcatNet and MMKcat can still be inserted as additional predictive branches, but the default implemented control loop is

LigandMPNN → GraphKcat risk-adjusted prefilter → whole-enzyme UMA broad screen → sMD / optional PMF

This is the catalytic counterpart of the SPURS → BioEmu → UMA hierarchy, except that the default catalytic high-fidelity signal now comes from a physically explicit enzyme-ligand dynamics protocol rather than from a purely supervised turnover regressor.

16 Data curation, splits, and training corpora

16.1 Backbone corpus for design

The primary training corpus in this revised manuscript is *procedurally generated*, not a conventional mutation dataset. Each training example is a de novo-generated protein or protein complex with total protein length sampled approximately uniformly from 20 to 400 residues, together with a starting sequence and the corresponding all-atom structural context used for redesign. The point of this choice is scale: sequence space for proteins in this length range is so large that datasets centered

on single-point mutants are inadequate as a primary source of generative supervision. Instead, the model is trained directly on redesign trajectories over structurally specified design targets.

We nevertheless recommend three structural partitions inside this procedurally generated corpus:

1. **Monomer redesign corpus.** Soluble de novo single-chain proteins for thermostability-focused training.
2. **Complex redesign corpus.** De novo multimers and de novo protein-ligand complexes whose total prepared atom counts remain within the intended training setting.
3. **Dynamics-eligible corpus.** The subset of the above on which BioEmu and/or UMA can be run during training, with the practical upper limit near 10 000 prepared atoms and the preferred whole-protein UMA setting near or below 8000 atoms.

If natural proteins are used at all, they should enter only as auxiliary calibration or robustness checks rather than as the primary redesign corpus.

16.2 Complex and ligand training corpora

Because the primary corpus is de novo generated, complex training examples are also defined procedurally. We recommend three subcorpora.

1. **Multimer and PPI redesign corpus.** De novo homo-oligomers, hetero-oligomers, antibody-like assemblies, and generic protein-protein interfaces with explicit bound structural context.
2. **Protein-ligand or fragment redesign corpus.** De novo proteins conditioned on bound small molecules, cofactors, ions, carbohydrates, peptide fragments, or nucleic-acid fragments. The architectural precedent here is LigandMPNN and, more importantly for the generator itself, ADFLIP's native support for non-protein atomic context [**? ?**].
3. **Component corpus.** For every complex task, store not only the bound system but also the separated component states used in the thermodynamic bookkeeping. This is essential because the training and evaluation code should not improvise the separated components differently for different oracles.

Spatial cropping also changes in the complex setting. For PPI tasks, interface-centered crops should be overrepresented so that mutations far from the interface do not dominate the training mix. For ligand tasks, pocket-centered crops and ligand-shell crops are the natural analogue. If full complexes are too large for a given batch, the crop must still preserve all atoms inside the residue-context radius around designable positions; otherwise the generator sees inconsistent context during training.

16.3 Reaction-conditioned catalytic corpora and metadata

Catalytic training requires a second axis of curation beyond structure: reaction metadata. Every catalytic record should contain at least a substrate identifier, with SMILES strings as the default interoperable representation. Product identities are optional but highly valuable because they activate the MMKcat product-conditioned branch and permit downstream mechanistic analyses.

Whenever GraphKcat is intended to be used, the example must also carry either a protein structure path or enough information to materialize one reliably from the current candidate.

We therefore recommend that every catalytic record store:

1. candidate or backbone identifier,
2. protein sequence and, when available, structure path,
3. substrate SMILES or ligand file,
4. optional product SMILES,
5. optional assay metadata such as pH, temperature, and organism,
6. oracle-eligibility flags for KcatNet, MMKcat, and GraphKcat.

The local repository enforces a minimum version of this requirement through an explicit metadata validator: Kcat-mode examples lacking substrate metadata are rejected before active-learning rounds begin. This is a useful discipline to preserve in the manuscript because catalytic design pipelines fail more often from metadata sloppiness than from neural-network architecture.

16.4 RosettaFold3-derived catalytic train/test splits with ProTrek

The catalytic RF3 corpus is not split at the level of individual rows from a metadata table. It is split at the level of *paired endpoint examples*. Each example

$$z_i = (s_i, A_i, B_i, \eta_i) \tag{135}$$

contains the enzyme sequence s_i, the reactant-bound RF3 complex A_i, the product-bound RF3 complex B_i, and the associated reaction metadata η_i such as substrate identity, product identity, chain IDs, and pocket annotations. This pairing matters because the catalytic active-learning loop conditions on both endpoint basins. A split that only respects sequence identity or only one endpoint geometry would underestimate leakage.

To reduce both sequence-level and structure-level leakage, we construct train/test partitions with ProTrek [30]. Let

$$u_i \in \mathbb{R}^{d_{\text{seq}}} \tag{136}$$

denote the ProTrek sequence embedding of the enzyme sequence s_i, and let

$$v_i^A, v_i^B \in \mathbb{R}^{d_{\text{str}}} \tag{137}$$

denote ProTrek structure embeddings of the reactant-bound and product-bound RF3 endpoint structures, respectively. Sequence similarity is defined by cosine similarity,

$$S_{\text{seq}}(i,j) = \cos(u_i, u_j). \tag{138}$$

Structural similarity is defined on the pair, not on a single endpoint:

$$S_{\text{str}}(i,j) = \max\left\{\cos(v_i^A, v_j^A), \cos(v_i^A, v_j^B), \cos(v_i^B, v_j^A), \cos(v_i^B, v_j^B)\right\}. \tag{139}$$

The maximum cross-state similarity in Eq. (139) is deliberate. It prevents leakage not only between nearly identical reactant-bound complexes, but also between cases where one candidate's reactant-bound state is geometrically close to another candidate's product-bound state. That situation is common in catalytic datasets and would be missed by a one-endpoint split.

We then form a combined similarity graph over paired RF3 examples. Nodes are examples z_i, and an undirected edge (i, j) is added whenever

$$S_{\text{seq}}(i, j) \geq \tau_{\text{seq}} \qquad \text{or} \qquad S_{\text{str}}(i, j) \geq \tau_{\text{str}}. \tag{140}$$

Connected components of this graph define combined ProTrek clusters. Entire clusters, rather than individual examples, are assigned to train or test, typically with defaults near

$$\tau_{\text{seq}} = \tau_{\text{str}} = 0.90 \tag{141}$$

and a target test fraction of roughly 20%. This yields a split that is stricter than a sequence-only partition while remaining computationally practical on large RF3 corpora.

Operationally, each retained split example stores the fields needed by the catalytic Method III loop: sequence, substrate/product metadata, reactant-bound complex path, product-bound complex path, protein chain ID, ligand chain ID when present, and pocket positions. The train/test split is validated before downstream use by checking that all referenced endpoint files exist and that the split manifest, counts, and pair-level metadata are internally consistent. The resulting train/test partition is therefore defined over the same RF3 paired endpoint objects that the catalytic oracles and the GFlowNet-style RL loop actually consume.

16.5 Oracle calibration sets

Calibration of the cheap, middle, and high-fidelity oracles should use experimentally measured stability datasets with explicit homology filtering against the redesign corpus. The recommended hierarchy is:

- Megascale for dense mutation coverage and supervised SPURS comparisons [18**?**];
- FireProt(HF), S669, Ssym, S2648, and related curated benchmark sets for independent validation of mutation-effect calibration [4, 25**?**];
- proteins or domains from these corpora that are BioEmu-eligible, for direct calibration of BioEmu-derived ensemble statistics;
- proteins whose prepared systems fit within the whole-protein UMA working setting, for direct UMA calibration when feasible;
- if the MINT or LigandMPNN oracle ablations are activated, repeat the same calibration workflow after the drop-in replacement rather than reusing the original SPURS calibration tables;
- a separate large-protein subset whose prepared systems exceed roughly 8000 atoms, to validate the BioEmu-dominant large-protein setting with either down-weighted UMA, or only local or hybrid UMA corrections;
- BRENDA- and SABIO-RK-derived catalytic datasets, stratified by metadata completeness, for calibration of $\log_{10} k_{\text{cat}}$, $\log_{10} K_{\text{M}}$, and $\log_{10}(k_{\text{cat}}/K_{\text{M}})$ as well as for MMKcat missing-modality stress tests.

Crucially, calibration and final evaluation sets must be disjoint at the protein-family level. If the high-fidelity oracle is calibrated on local clusters, the split must be done at the full-protein level before clusters are extracted.

16.6 Held-out evaluation tiers

We recommend three held-out evaluation tiers.

Tier 1: oracle validation.
Compare SPURS, BioEmu, UMA, KcatNet, MMKcat, GraphKcat, and fused rewards against experimental $\Delta\Delta G$, ΔT_m, k_{cat}, K_{M}, or $k_{\text{cat}}/K_{\text{M}}$ on unseen proteins, reporting the eligible subset for each oracle explicitly.

Tier 2: design validation.
For held-out backbones without experimental labels, evaluate sampled sequences using self-consistency, pack quality, diversity, novelty, mutation-order coverage, and cross-oracle agreement.

Tier 3: experimental shortlist.
If wet-lab experiments are performed later, use a frozen model and pre-registered selection rules to avoid post hoc cherry-picking.

16.7 Held-out evaluation additions for complex tasks

Complex-aware evaluation should report at least four categories of numbers, always stratified by task class.

1. **Component stability.** For each designed chain or designed component, evaluate whether monomeric or isolated-component stability is preserved or improved.
2. **Binding or assembly metrics.** For PPI and ligand tasks, report correlation, calibration, and hit-rate metrics for $\Delta\Delta G_{\text{bind}}$ or ΔG_{bind} on held-out mutations or complexes.
3. **Thermodynamic consistency.** Measure how well the learned cheap and fused branches satisfy the identity that bound minus separated predictions should match the reported binding term. This should be treated as a first-class metric rather than an implementation detail.
4. **Target-conditioning success.** When target-conditioned generation is enabled, report the success rate at user-specified tolerance bands, the deviation of the best failed design, and the distribution of retries required before success.

A particularly important negative control is to compare complex-aware training against a monomer-only baseline evaluated on the same complex tasks. This isolates the value of explicit partner or ligand context rather than conflating it with generic scaling of the design model.

17 Recommended hyperparameters and compute budgets

17.1 Training hyperparameters

Table 4 lists concrete defaults. These are deliberately conservative. The goal is to obtain stable training before attempting aggressive speed or reward-scaling choices.

Table 4: Proposed default hyperparameters and tuning ranges.

Hyperparameter	Default	Suggested range	Rationale
Base ADFLIP checkpoint	released all-atom checkpoint	compare with single-state and multistate modes	Use the published checkpoint matching the task class; multistate mode is preferred when structural ensembles are available, such as by running BioEmu, UMA, or obtaining NMR data [**?**].
Seed generation steps	10	5–20	Matches the source paper's runtime-quality compromise.
Mutation-order policy	No hard cap; stop masked until sampled $K_{\min}$	$K_{\min}$ mixture over 1, 2–4, 5–8, 9–16	Exposes training to variable-order multi-mutant trajectories without scientifically truncating Hamming distance.
GFlowNet objective	SubTB(λ)	TB, DB as ablations	SubTB is usually more stable for long horizons [21].
λ in SubTB(λ)	0.9	0.7–0.97	Favors longer-range credit while retaining local updates.
Reward temperature β in Simul-MF	$U[0.75, 1.75]$	$[0.5, 2.5]$	Exploration-exploitation control.
Reward temperature β in Stage 1 of Seq-Curr	$U[0.5, 2.0]$	$[0.5, 2.5]$	Broad early exploration.
Reward temperature β in Stage 2 of Seq-Curr	$U[0.75, 2.0]$	$[0.5, 2.25]$	BioEmu-guided dynamics refinement.
Reward temperature β in Stage 3 of Seq-Curr	$U[1.0, 2.0]$	$[0.75, 2.25]$	Sharper exploitation after UMA refinement.
Optimizer for new heads	AdamW, 10^{-4}	5×10^{-5} to 3×10^{-4}	Standard stable default.
Optimizer for unfrozen pretrained ADFLIP blocks	AdamW, 2×10^{-5}	10^{-5} to 5×10^{-5}	Full fine-tuning default for the non-frozen ADFLIP parameters.
Replay buffer size	100k terminal candidates	20k–500k	Enough for diversity without excessive memory.
On-policy : replay : anchor batch ratio	0.6 : 0.3 : 0.1	tune per training stability	Balances current exploration with preservation of base generator.
Fast full-context ADFLIP refresh steps	4	2–8	Cheap whole-object refresh after each accepted edit.
Terminal full repack steps	50	20–60	Matches ADFLIP's higher-fidelity inference setting.
Primary trusted SPURS epistasis order	2	1–2	Single and double mutants are the primary cheap-supervision setting.
Higher-order SPURS epistasis	down-weighted	optional	Reserve explicit $K \geq 5$ SPURS epistasis for ablations.

Hyperparameter	Default	Suggested range	Rationale
Baseline oracle scoring	on for every training example	on/off only in ablation	Cache starting-sequence or starting-complex properties before redesign.
Mean u_{pack}threshold for penalty	0.25	0.15–0.40	Screening for poorly packed intermediates or high-entropy terminal proposals.
BioEmu samples per candidate during training	2048	512–4096	Enough to estimate whole-chain ensemble features without turning BioEmu into the bottleneck [16].
BioEmu sample count for shortlisted candidates	10k	5k–20k	Matches the scale used in the BioEmu paper for more stable ensemble estimates [16].
BioEmu evaluation rate in Simul-MF	25%	15–50%	BioEmu is cheap enough to screen a sizable fraction of terminal candidates.
UMA evaluation rate in Simul-MF	3% overall	1–5% overall	Sensible default under moderate compute.
UMA-cat broad-screen evaluation rate in catalytic Simul-MF or OneShot-AL	5–15% of filtered pool	1–25%	Broad whole-enzyme equilibrium catalytic screen before narrower structural refinement.
UMA-cat sMD evaluation rate in catalytic Simul-MF or OneShot-AL	same as broad UMA-cat batch	task dependent	By default the repository runs forward and reverse steering on the same catalytic batch once endpoints are packed.
UMA-cat PMF finalist rate	0–10% of catalytic batch	0–20%	Optional path umbrella refinement for finalists; off by default because it is materially more expensive.
GraphKcat evaluation rate in catalytic Simul-MF or OneShot-AL	50–100% of candidate pool	10–100%	Packed-structure catalytic prefilter before the expensive whole-enzyme UMA stage.
GraphKcat pocket cutoff	8 Å	6–10 Å	Matches the current GraphKcat preprocessing setting in the repository.
Catalytic fusion weights $(w_{\text{UCAT}}, w_{\text{GK}}, w_{\text{agree}})$	$(0.90, 0.55, 0.20)$	task dependent	Matches the current repository defaults for UMA-cat plus GraphKcat fusion before further tuning.
Whole-protein UMA atom budget	8000 atoms	4000–10000	Matches the documented large-system MD example while keeping the primary protocol conservative [5].
BioEmu-dominant switch	$N_{\text{atom}} > 8000$ or weak whole-protein UMA calibration	7000–10000	Makes the large-protein setting explicit rather than implicit.

Hyperparameter	Default	Suggested range	Rationale
MD timestep for UMA molecular runs	0.1 fs	0.05–0.25 fs after validation	Conservative value taken from FAIR Chemistry examples [5].
Whole-protein UMA temperature ladder	300, 330, 360, 390, 420 K	tune around expected fold class	Supports thermostability curves rather than a single-temperature score.
Production time per temperature (screening)	25 ps	10–50 ps	Adequate for ranking during training; extend for shortlisted candidates.
Replicate trajectories per temperature	4	2–8	Needed for uncertainty estimation.
sMD images per replica	24	8–64	Repository default for the catalytic endpoint-to-endpoint steering schedule.
Path-PMF umbrella windows	16	8–32	Repository default for optional catalytic PMF refinement seeded from sMD path images.

17.2 Additional defaults for complex and goal-conditioned training

Table 5 lists the defaults that are specific to complexes, bound-versus-separated evaluation, and target-conditioned generation. They should be read together with Table 4; if a value appears in both tables, the complex-specific table takes precedence for multimeric and ligand tasks.

Table 5: Proposed defaults specific to complex-aware and target-conditioned training.

Quantity	Default	Range	Purpose
Monomer-only warmup updates U_0	25k	10k–100k	Lets the base generator stabilize before interface or pocket tasks are introduced.
Complex-mixture ramp end U_1	75k	30k–150k	Controls how quickly PPI and ligand tasks become standard parts of training.
Steady-state monomer/PPI/ligand mix	0.333 / 0.333 / 0.333	task dependent	Default mixed-task sampling distribution after warmup and ramp.
Residue-context radius r_{ctx}	10 Å	8–12 Å	Defines which partner or ligand atoms condition a mutable residue.
Maximum context atoms per mutable residue	256	128–512	Keeps pocket or interface neighborhoods bounded for memory efficiency.
Symmetry-tying mode for homo-oligomers	on when map available	on/off	Applies one mutation action to all symmetry-equivalent copies.
Target-conditioning dropout	0.20	0.05–0.40	Supports both conditional and unconditional generation within one model.
Target-attainment reward weight w_{target}	0.75	0.25–1.5	Balances explicit target matching against raw stability or affinity optimization.
Acceptance tolerance for T_{m}	user specified; default ±2 K	task dependent	Determines success at deployment for temperature-like targets.
Acceptance tolerance for ΔG_{bind} or $\Delta\Delta G_{\mathrm{bind}}$	user specified; default ±0.5 kcal/mol	task dependent	Determines success at deployment for binding targets.
Acceptance tolerance for $\log_{10} k_{\mathrm{cat}}$, $\log_{10} K_{\mathrm{M}}$, or $\log_{10}(k_{\mathrm{cat}}/K_{\mathrm{M}})$	user specified; default ±0.25	task dependent	Determines success at deployment for kinetic targets on the log scale.
Reaction-metadata dropout	0.10	0.0–0.30	Encourages the model to remain usable when assay metadata are partially missing.
Candidates per retry at inference	64	16–256	Controls how broad each target-conditioned search attempt is.
Maximum retries $R_{\max}$	16	4–64	Bounded deployment latency with graceful failure.

17.3 Additional defaults for OneShot-AL

Because OneShot-AL has a teacher, a surrogate, and a one-shot student, its hyperparameters are easier to understand by component. Table 6 gives the proposed defaults.

Table 6: Proposed defaults specific to OneShot-AL.

Parameter	Default	Range to tune	Comments
Active-learning rounds	8	5–15	Enough to refresh the surrogate and expensive sets without making orchestration unwieldy.
Teacher off-policy fraction γ_{off}	0.5 early, decay to 0.25	0.25–0.60	Directly motivated by the static-dataset updates in the 2022 biological-sequence GFlowNet setting [8].
Teacher steps per round	30k	20k–40k	Larger values only if the round dataset is changing quickly.
Student distillation steps per round	15k	10k–25k	Should be long enough that teacher-student KL stabilizes before proposal sampling.
Surrogate ensemble size	8	4–16	Increase when epistemic uncertainty is central to selection.
Student proposal pool per round	50k candidates	10^4–10^5	One-shot generation is cheap enough that oversampling is usually beneficial.
BioEmu batch size per round	512 candidates	128–2048	BioEmu should screen a much wider set than UMA in every round.
UMA batch size per round	64 candidates	16–128	Match to the available UMA budget and desired turnaround time.
UMA-cat broad-stage batch size per round	256 candidates	64–1024	Matches the default catalytic round budget in the repository and keeps the whole-enzyme dynamics stage tractable.
GraphKcat prefilter batch size per round	candidate pool or its largest feasible packed subset	task dependent	In the implemented catalytic specialization, GraphKcat runs before UMA-cat and should screen as much of the packed pool as budget allows.
PMF finalist quota	0 by default	task dependent	Optional catalytic path-PMF refine-

17.4 Compute budget planning

A realistic initial implementation should separate generator compute from dynamics-oracle compute. The generator branch requires batched ADFLIP denoising and SPURS inference and can be run on a small number of high-memory GPUs. A sensible starting point is 4–8 H100-class GPUs for policy training and batched SPURS inference. BioEmu screening can run on a separate but comparatively small pool because the BioEmu paper reports thousands of statistically independent samples per hour on a single GPU [16]. In OneShot-AL, the one-shot student makes proposal generation much cheaper than in the edit-trajectory methods, so the same hardware can cover far larger candidate pools per wall-clock hour; the extra cost appears at the teacher-surrogate refresh between active-learning rounds.

The highest-fidelity UMA branch can run asynchronously on a separate pool. The FAIR Chemistry repository documents multi-GPU and multi-node inference as well as LAMMPS compatibility, and reports that wall-clock MD throughput scales materially with multiple H100 GPUs for very large systems [5]. For the whole-protein UMA setting up to roughly 8000 atoms considered here, the actual throughput will be dominated jointly by atom count, trajectory length, and the number of temperatures and replicates. Above that setting, BioEmu usually remains the cheaper whole-chain screen and local or hybrid UMA is what should be costed.

For planning purposes, it is useful to think in terms of trajectory-equivalent units. One rollout of a length-K edit trajectory requires about K local ADFLIP evaluations, K SPURS evaluations, and one final reward fusion. A BioEmu screening bundle means generating and processing M_{B} ensemble samples for one candidate. A UMA bundle means all temperatures and replicates for a single candidate, with local or hybrid fallbacks counted separately. Because the exact cost depends heavily on sequence length, prepared-system atom count, number of temperatures, and the oracle-selection rates, we recommend logging six counters from the first day of experiments: mean rollout length, BioEmu screening rate, UMA screening rate, median prepared-system atom count, wall-clock per BioEmu bundle, and wall-clock per UMA bundle. These six numbers determine most of the eventual compute bill.

18 Validation methodology and evaluation plan

18.1 Oracle validation before design claims

Before comparing final design performance, one must validate the three oracle branches and the reward fusion. We recommend the following order.

Step 1: SPURS calibration on the target setting. Run SPURS on held-out experimental mutation-effect datasets and, separately, on ADFLIP-designed sequences for held-out monomers, complexes, and multistate targets. Quantify ranking quality (Spearman, Pearson), classification of stabilizing mutations (AUROC, AUPRC at thresholds such as $\Delta\Delta G < -0.5$ kcal/mol in the destabilization convention), and calibration error. If SPURS degrades strongly on designed sequences relative to natural proteins, that degradation must be modeled explicitly or the reward weight w_{S} should be reduced on those classes of backbones.

Step 2: BioEmu calibration on the intended use classes. Evaluate whether BioEmu-derived equilibrium features correlate with experimental stability on BioEmu-eligible proteins. Report

correlation, calibration error, and the sensitivity of the score to sample count M_{B}. Because BioEmu is most important in the large-protein setting, stratify results by prepared-system atom count or sequence length, and report separately the subset above roughly 8000 atoms where whole-protein UMA is not the default whole-chain dynamics signal.

Step 3: UMA whole-protein and fallback validation. Evaluate whether whole-protein UMA-derived signals are stable across replicate trajectories and whether they correlate with experimental stability on proteins whose prepared systems fit the default budget. Report not only average correlation but also confidence diagnostics: fraction of candidates whose uncertainty intervals cross zero improvement, agreement between screening and enhanced-sampling estimates, and disagreement between whole-protein and fallback local or hybrid estimates when both are available. For proteins above the whole-protein budget, evaluate the fallback settings separately rather than pooling them with the direct whole-protein setting.

Step 4: cross-oracle agreement and disagreement. On a calibration set, measure agreement between SPURS, BioEmu, and UMA. Low agreement is not necessarily a problem, but systematic disagreement on specific mutation classes (for example solvent-exposed charge flips, buried aromatic packing, or local unfolding around glycine or proline insertions) should inform reward weights, reliability gates, and acquisition priorities. In the large-protein setting, the critical comparison is usually SPURS versus BioEmu on the wide pool and BioEmu versus UMA on the small physically focused subset.

Step 5: catalytic-oracle calibration and disagreement. On reaction-conditioned enzyme datasets, evaluate the whole-enzyme UMA catalytic proxy, GraphKcat, and their fused catalytic score against experimental k_{cat}, K_{M}, and $k_{\mathrm{cat}}/K_{\mathrm{M}}$ when each quantity is available. The broad whole-enzyme UMA branch should be analyzed separately into its constituent parts: productive-pose occupancy, gating free energy, sMD barrier, PMF barrier when enabled, and the resulting log-rate proxy. GraphKcat should be stratified by whether the structure and ligand geometry were fully materialized without repair. Cross-oracle disagreement is especially informative here: large disagreement between the dynamical catalytic proxy and GraphKcat often signals that static pocket features and explicit endpoint-to-endpoint dynamics are pointing in different directions. KcatNet and MMKcat should still be evaluated as auxiliary predictive baselines or missing-modality controls, but they are no longer the primary implemented catalytic feedback loop.

18.2 Teacher-student validation for OneShot-AL

OneShot-AL introduces a new validation axis that Methods I and II do not have: the student must faithfully approximate the teacher on the parts of sequence space that matter. We therefore recommend four round-level diagnostics.

1. **Teacher-student KL on held-out pools.** Hold out a stratified candidate pool by backbone, mutation-order setting ($x^{(0)}$, $K = 1$, $K = 2$, $K \geq 3$), and estimate the cross-entropy or KL between teacher weights and student log-probabilities.

2. **Top-k overlap.** Compare the teacher and student top-k sets after rescoring with the same oracle bundle. The overlap should be reported both by exact sequence identity and by cluster identity.

3. **Tail coverage.** Measure whether the student under-samples high-Hamming-distance or higher-order ($K \geq 3$) teacher modes. Report this separately, because average KL can look acceptable while rare but important modes disappear.

4. **Oracle-stage preservation.** Check whether the student preserves the teacher's ordering after dense SPURS rescoring and again after BioEmu rescoring. A student that matches only the SPURS stage but not the dynamics stage is not good enough for deployment.

Only after these diagnostics are satisfactory should one compare OneShot-AL directly against the edit-trajectory methods on held-out backbone design tasks.

For the catalytic specialization, the same diagnostics should be repeated on reaction-conditioned pools. In particular, teacher-student KL should be stratified by substrate class and by catalytic-oracle eligibility, because a student may match the teacher on pure sequence-edit statistics while still underrepresenting rare but valuable catalytic modes.

18.3 Design metrics

For terminal samples generated on held-out backbones, report:

1. **Best-of-N fused score:** maximum fused reward among N samples.

2. **Mean top-k fused score:** average reward of the best k distinct samples, reported overall and stratified by mutation-order bins.

3. **Diversity:** mean pairwise Hamming distance among top-k samples and number of unique clusters after sequence identity clustering.

4. **Mutation-order profile:** histogram of $K(x)$ among accepted candidates, together with the fraction of accepted candidates in the explicit double-mutant setting and the higher-order setting.

5. **Structure plausibility:** u_{pack}, terminal purity or entropy, self-consistency pass rate, and backbone RMSD to the target after structure prediction with BioEmu where applicable.

6. **Cross-oracle agreement:** pairwise agreement between SPURS, BioEmu, and UMA on the shortlisted candidates.

7. **Catalytic quality:** best-of-N and top-k fused catalytic reward, together with the distributions of productive-pose occupancy, UMA-cat log-rate proxy, GraphKcat turnover, and, when enabled, PMF barrier estimates on catalytic tasks.

8. **Large-protein breakdown:** all metrics above repeated on the subset whose prepared systems exceed the whole-protein UMA setting.

18.4 Ablations

A method paper of this kind is not credible without ablations. The minimum required set is:

1. **No BioEmu branch.** Compare against the SPURS-plus-UMA version of the system.

2. **No UMA branch.** Especially important for the 8000–10 000 atom setting, where BioEmu may already supply much of the useful dynamics signal.

3. **No uncertainty terms.** Remove risk adjustment from one branch at a time.
4. **SPURS-guided proposals.** Reintroduce SPURS-based proposal shortlisting or feature augmentation and compare against the primary ADFLIP-only proposal policy.
5. **Higher-order SPURS epistasis.** Replace the primary single/double-mutant SPURS supervision with SPURS's own higher-order decoder on $K \geq 3$ candidates and measure whether this helps or destabilizes training.
6. **LoRA or adapters versus full fine-tuning.** Compare parameter-efficient updates against the primary full-fine-tuning baseline for the non-frozen ADFLIP components.
7. **MINT replacement of the SPURS ESM-2 branch.** For multimer and PPI-heavy tasks, replace the sequence branch of the rewired oracle with MINT and repeat calibration and design training [20].
8. **LigandMPNN replacement of the SPURS ProteinMPNN branch.** For ligand-rich tasks, and optionally also for monomers and PPIs, replace the structure branch with LigandMPNN and repeat calibration and training [**?**].
9. **Teacher-free one-shot baseline.** Train the same one-shot ADFLIP student used in OneShot-AL directly on reward-weighted replay data without a GFlowNet teacher.
10. **Large-protein setting switch.** Compare hard BioEmu-dominant routing versus softer reliability gating above 8000 atoms.
11. **No reverse sMD branch.** Measure the effect of dropping reverse pulls and the forward-reverse consistency diagnostics.
12. **No PMF branch.** Measure the effect of using only the sMD barrier surrogate without path umbrella refinement.
13. **No GraphKcat branch.** Measure the effect of removing pocket-aware structural catalytic refinement and relying only on the whole-enzyme UMA catalytic proxy.
14. **Auxiliary KcatNet/MMKcat reinstatement.** Compare the default UMA-cat plus GraphKcat path against variants that add KcatNet or MMKcat as extra predictive catalytic channels.
15. **Catalytic-only versus joint reward.** Compare a catalytic-only teacher against the joint stability-binding-catalysis reward on the same enzyme tasks.

At least one ablation table should be stratified by prepared-system atom count or sequence length, because that is the central setting where BioEmu is expected to change the behavior of the method.

18.5 Complex-task and target-conditioning validation

The validation suite should explicitly test whether the architecture changes introduced in Section 7 are actually being used. For complex tasks, this means checking whether mutations outside the interface or ligand shell receive lower marginal proposal probability or lower acceptance rate than mutations inside the relevant shell when all else is held equal, whether bound-versus-separated bookkeeping remains numerically stable across the three oracles, and whether the policy changes when partner or ligand context is ablated. For target-conditioned generation, the required metrics are

success rate at tolerance, average number of retries, best-failed normalized deviation, calibration of the predicted property vector returned to the user, and the difference between cheap-only acceptance and acceptance after higher-fidelity rescoring.

Whenever a final experimental paper is written, it should include at least one calibration figure comparing requested versus achieved properties for held-out target vectors. This prevents the target-conditioning interface from becoming a purely cosmetic conditioning channel with no operational meaning. The same requirement applies to catalytic targets: if users can request target $\log_{10} k_{\mathrm{cat}}$, $\log_{10} K_{\mathrm{M}}$, or $\log_{10}(k_{\mathrm{cat}}/K_{\mathrm{M}})$, the achieved distributions must be reported explicitly rather than only summarized with success flags.

19 Failure modes and mitigations

19.1 Oracle exploitation

The most obvious danger is reward hacking: the policy may discover sequences that look good to SPURS, to a fragile BioEmu calibration model, or to a narrow UMA protocol while actually violating foldability. Mitigations include (i) explicit structural plausibility terms, (ii) self-consistency filters on shortlisted candidates, (iii) cross-oracle disagreement audits, and (iv) preserving a diverse candidate distribution rather than committing to a single mode. Because GFlowNets already favor diversity, they are less vulnerable to this failure than purely maximizing RL, but they do not eliminate it.

19.2 Complex-specific reward hacking

Complex tasks introduce new reward-hacking routes beyond those already present for monomers. A policy may appear to improve binding by collapsing one isolated component, or by exploiting a narrow pocket score while worsening global foldability. The mitigation is to make the separated-component terms non-optional, to track thermodynamic consistency explicitly, and to report component collapse metrics alongside binding metrics.

19.3 BioEmu misuse on full complexes

BioEmu is not presented in this manuscript as a universal explicit-complex simulator. In multimeric and ligand settings it is primarily a fast chain-wise or component-wise dynamics oracle and a way to regularize whole-chain conformational plausibility at scale. If one treats it as if it were an exact bound-complex free-energy engine, the reward can become overconfident. The mitigation is simple: use BioEmu heavily for scalable screening, but let direct bound-versus-separated thermodynamic differences be anchored by cSPURS and especially by UMA wherever the scope and atom budget allow it. As an ablation, we include BioEmu run on complexes with a span of 200 mask tokens in between chains, similar to the way AlphaFold2 before AlphaFold-Multimer was trained, which will require some additional calibration, but could be viable in most well-packed and globular complex settings.

19.4 Target-conditioning failure modes

A target-conditioned generator can fail in two distinct ways. The first is *silent ignoring*: the policy emits plausible designs, but the requested targets have little effect on the output distribution. The

second is *over-conditioning*: the model chases target values so aggressively that foldability or diversity collapses. These failure modes should be monitored by conditional calibration curves, success-versus-diversity plots, and the ablation that drops the target-attainment term from the reward while keeping the same conditioning interface. The retry-aware deployment rule in Algorithm 1 is also a mitigation because it forces the system to expose when a target was not achieved instead of returning an unchecked claim.

19.5 BioEmu scope mismatch

BioEmu is trained as a single-chain equilibrium-ensemble emulator at a fixed thermodynamic condition and should not be treated as a universal biomolecular simulator [16]. The main overreach risks are membrane proteins, explicitly ligand-dependent stability mechanisms, strong and multimeric coupling. The mitigation is explicit eligibility tagging, reduced reliability gate ρ_{B} outside the intended scope, and separate reporting of BioEmu-eligible versus BioEmu-ineligible benchmarks.

19.6 UMA out-of-distribution use on proteins

The updated methodology intentionally uses whole-protein UMA-MD up to roughly 8000 atoms because public FAIR Chemistry documentation now demonstrates that system-size throughput is practical at that scale and beyond [5]. The main overreach risk is therefore no longer raw atom count alone; it is treating an MLIP trained on OMol25-scale chemistry as an automatically exact whole-protein folding-thermodynamics engine. The mitigation is explicit calibration, uncertainty decomposition, separate reporting for whole-protein and fallback settings, and a rule that whole-protein UMA signals are never presented without their calibration context and replicate uncertainty. Above the whole-protein setting, BioEmu should carry the whole-chain dynamics burden and UMA should only answer targeted local or hybrid questions, until further UMA evals are run on larger systems.

19.7 Collapse to trivial near-seed sequences

If the stop action becomes too attractive, the policy may learn to stop immediately or to apply only one obviously stabilizing mutation. This is easy to detect through the mutation-order histogram. The fix is either to weaken w_{rad} if active, widen the mutation-order curriculum, reduce the stop-action prior, or increase temperature diversity while keeping stop optional at inference.

19.8 Sidechain degradation under RL updates

Because thermostability reward is sequence-focused, the model may sacrifice packing quality unless constrained. This is why Eq. (25) includes packing regularization and why u_{pack} is tracked. Any RL run in which u_{pack} drifts sharply should be treated as invalid until corrected.

19.9 Epistasis beyond the cheap oracle's comfort zone

SPURS includes a higher-order decoder, but its reliability for higher-order candidates is still an empirical question. The updated protocol therefore allows large total Hamming distance while treating explicit cheap supervision as strongest for singles and doubles. A practical mitigation is to

increase the BioEmu and UMA acquisition rates for candidates with $K(x) \geq 5$, especially when the cheap score changes sharply relative to the best additive baseline or when oracle disagreement rises.

19.10 One-shot student mode dropping

A failure mode unique to OneShot-AL is that the one-shot student may become too concentrated even when the teacher remains diverse. This usually appears first in the tails: large-Hamming-distance candidates, unusual but stabilizing residue swaps, or rare explicit double-mutant or higher-order motifs vanish from the student proposal pool. The mitigation is straightforward: monitor teacher-student KL by mutation-order bin, raise the distillation temperature on under-covered bins, and allocate a fixed fraction of each BioEmu and UMA batch to teacher-only samples so that the dataset does not become self-confirming.

19.11 Catalytic metadata mismatch

Catalytic branches can fail for reasons that stability branches never see. A substrate SMILES may be wrong, a product annotation may refer to the wrong reaction direction, a pH label may be inconsistent with the underlying wet-lab assay, or a ligand conformer may be physically implausible. These failures can create apparently systematic model errors that are in fact data errors. The mitigation is explicit metadata validation, persistent storage of admissibility flags, and stratified reporting by metadata completeness.

19.12 GraphKcat pocket fragility

GraphKcat depends on protein structure and ligand geometry. If the protein structure is mis-packed, if the ligand conformer is badly embedded, or if the wrong pocket is extracted, the resulting prediction can be confidently wrong. This is not an argument against GraphKcat; it is an argument for routing discipline. GraphKcat should be used only when its structural prerequisites are satisfied, and its predictions should remain coupled to structure-quality diagnostics rather than being treated as a free extra label.

19.13 sMD coordinate mismatch and hysteresis

Steered catalytic dynamics can fail for a different reason: the pulling coordinate may be physically misaligned with the true reactive pathway. In that case the work distribution becomes broad, forward and reverse pulls separate strongly, and the harvested path images may reflect steering artifacts rather than the dominant reactive channel. The mitigation is to treat forward-reverse mismatch, dissipated work, and path hysteresis as first-class diagnostics, to harvest near-transition-state-like structures conservatively rather than declaring them exact transition states, and to prefer PMF refinement only when the steered path shows adequate continuity and overlap.

19.14 MMKcat mask collapse

MMKcat is valuable precisely because it exposes missing-modality sensitivity, but that also creates a new failure mode: mask collapse. If all admissible masks produce nearly identical predictions because the model has learned to ignore the optional channels, the method gives the appearance of

robustness without actually extracting information from the missing-modality training mechanism. The mitigation is to monitor mask disagreement, to report performance stratified by mask family, and to treat vanishing disagreement as a diagnostic that needs explanation rather than as an automatic positive.

19.15 Catalysis-stability tradeoff collapse

Catalytic optimization can create variants that appear beneficial on turnover but destroy foldability, binding, or expression. The fused reward is intended to reduce exactly this failure, but it will not do so if the catalytic branch is over-weighted or if the stability branch is under-calibrated. The mitigation is to report Pareto fronts rather than only scalarized scores, to keep explicit component-protection terms active, and to require that catalytic improvements be reported together with the stability or binding changes they induce.

20 Responsible use, and publication discipline

A second responsibility issue is scientific rather than biosafety-related: it is easy to overstate what an expensive oracle means. This manuscript deliberately avoids that. It specifies how BioEmu and UMA can be used as complementary dynamics oracles in a protein design pipeline while also making clear that neither branch automatically guarantees exact experimental folding free energies. Any eventual NeurIPS submission should preserve that distinction.

21 Conclusion

This paper has specified a complete methodology for GFlowNet fine-tuning of ADFLIP toward thermostable, affinity-aware, and catalysis-aware fixed-backbone protein design. The central idea is to turn ADFLIP from a full-atom inverse folding model into a reward-proportional sequence sampler over variable-order edit trajectories rooted at ADFLIP seed sequences, and then to complement those edit-trajectory methods with a one-shot deployment option. SPURS provides the dense cheap stability branch: baseline-aware mutation landscapes plus explicit single- and double-mutant supervision with conservative higher-order handling, now extended in complex settings into a bound-versus-separated cSPURS objective for protein–protein and protein–ligand tasks. BioEmu provides the fast whole-chain dynamics branch: sequence-conditioned equilibrium ensembles whose derived folded-basin statistics, global unfolding and folded-basin features can be calibrated into a scalable thermostability signal [16]. UMA provides the highest-fidelity MLIP-MD branch: calibrated whole-protein thermostability estimates up to roughly 8000 atoms, with local or hybrid fallback beyond that setting. The catalytic branch is now equally explicit. KcatNet, MMKcat, and GraphKcat remain important learned kinetic predictors [17, 19, 29], but the default catalytic RL specialization implemented in this repository uses whole-enzyme UMA equilibrium screening, forward and reverse steered endpoint-to-endpoint dynamics, optional path umbrella PMFs, and GraphKcat as an orthogonal structural catalytic refinement oracle. The updated manuscript also generalized the generator and reward to monomers, multimers, protein–protein interfaces, protein–ligand systems, and reaction-conditioned enzyme tasks by exploiting ADFLIP's native all-atom complex conditioning and by translating oracle outputs into component stability, binding free-energy, and catalytic-kinetic terms. We developed three concrete training protocols. Simul-MF interleaves all oracle families online using asynchronous dynamics and catalytic acquisition plus hierarchical reward correction.

Seq-Curr separates broad cheap-oracle exploration from dynamics refinement and then targeted high-fidelity rescoring before optional distillation. OneShot-AL follows the round-based biological-sequence active-learning style of prior GFlowNet work and is now instantiated in the repository as an actual trajectory-balance teacher over canonical edit trajectories plus a one-shot student for non-iterative proposal generation. In the catalytic specialization already implemented in this repository, the broad stage is a real whole-enzyme UMA catalytic screen, the first learned prefilter is packed-structure GraphKcat with MC-dropout uncertainty, and the narrower expensive stage remains the UMA catalytic dynamics stack.

Just as importantly, the manuscript has made the large-protein setting explicit. The FAIR Chemistry release now documents large-system MD execution including an 8000-atom worked example and reported throughput on 100k+-atom systems [5], so whole-protein UMA-MD is a realistic branch rather than a hypothetical one. But the public OMol25 evidence remains rooted in chemically rich systems up to about 350 atoms [15], so the scientifically conservative design is to combine UMA with a dedicated protein-equilibrium emulator rather than to rely on UMA alone. BioEmu is precisely that complement: a fast, protein-specific, dynamics-aware oracle trained from structural data, more than 200 milliseconds of MD, and large-scale experimental stabilities [16]. In the large-protein setting above roughly 8000–10000 atoms, the updated methodology therefore lets BioEmu carry the whole-chain dynamics burden while UMA contributes targeted local or hybrid corrections.

The methodology now also includes target-conditioned deployment: users may request target values or thresholds for quantities such as T_{m}, folding stability, binding free energy, turnover number, Michaelis constant, or catalytic efficiency, and the model will retry generation up to a bounded budget before returning either a successful design or the best failed design together with its predicted property vector. The next step is straightforward: implement the pipeline exactly as specified here, run the ablations and evaluation suite, and only then make performance claims. If the empirical study succeeds, the result would not merely be another thermostability optimizer. It would be a full-atom, diversity-preserving, multi-oracle protein design framework whose outputs are explicitly tuned for variable-order multi-mutant redesign, oracle robustness, catalytic competence, and experimental follow-up.

A Appendix A: detailed implementation notes

A.1 State caching and replay design

Because both ADFLIP and SPURS are called frequently, caching is essential. We recommend the following cache keys:

- backbone identifier,
- current sequence string,
- edit mask and last edited position,
- hash of current sidechain coordinates rounded to a coarse tolerance if local repacking is stochastic.

For each cached state, store ADFLIP residue embeddings, denoiser purity or entropy summaries, SPURS mutation matrix, mean u_{pack}, BioEmu-eligibility tags, and any cheap structure features. For terminal candidates that have already been screened, cache BioEmu summary features u_{B} and

the availability of direct UMA labels. The replay buffer should store both full trajectories and terminal candidates. When memory is limited, retain terminal candidates plus sufficient metadata to reconstruct the trajectory loss locally.

A.2 Reward normalization details

Running means and variances used to standardize reward components should be computed on a lagged buffer, not on the current minibatch. A simple exponential moving average with decay 0.99 is adequate. Maintain separate EMAs for the dense SPURS score, the BioEmu score, the directly measured UMA score, and the corrected high-fidelity score. During early training when the higher-fidelity branches are underpopulated, cap $|z_{\mathrm{B}}|$ and especially $|z_{\mathrm{U}}|$ more aggressively than $|z_{\mathrm{S}}|$.

A.3 Fold Prediction Checks

All Fold predictions needed for verifying backbone RMSD should be done with BioEmu, with the option of sampling *very* short sampling run, and choosing top N nearest frames for comparison, where N is set to some number that ensure the original conformation is sampled a high enough percentage of the time. As a more robust alternative, we cache an original dynamics ensemble from BioEmu for the original structure, then compare the dynamics of the designed to the original via Jensen-Shannon divergence (utilizing ENCORE inside of MDAnalysis), however, this could over-constrain the dynamics variability and may prevent the model from sampling higher thermostability designs than the original input structure.

B Appendix B: additional algorithms

Algorithm 6 Baseline-aware scoring of the starting object and the first ADFLIP full redesign

Require: Starting object $(B, s^{\mathrm{base}}, \mathcal{C})$, target-conditioning vector c, admissible oracles, pretrained ADFLIP

1: Evaluate admissible oracles on $(B, s^{\mathrm{base}}, \mathcal{C})$ to obtain baseline property vector y^{base}
2: Sample first full-redesign proposal $x^{(0)}$ from ADFLIP conditioned on $(B, \mathcal{C}, c, y^{\mathrm{base}})$
3: Repack with a full PIPPACK pass
4: Evaluate the admissible oracles on $x^{(0)}$
5: Translate oracle outputs into absolute properties and baseline-relative quantities such as $\Delta\Delta G$, ΔT_{m}, and $\Delta\Delta G_{\mathrm{bind}}$
6: Return $(y^{\mathrm{base}}, x^{(0)}, \widehat{y}(x^{(0)}), \widehat{y}(x^{(0)}) - y^{\mathrm{base}})$

Algorithm 7 Full-context ADFLIP refresh and repacking after a single accepted edit

Require: Current full-atom state $(B, s_t, X_t^{\text{sc}}, \mathcal{C})$, edit (j, a), fast refresh steps N_{refresh}

Apply mutation $s_{t+1}(j) \leftarrow a$
Warm-start the full sequence from (s_t, X_t^{sc})
Run ADFLIP for N_{refresh} full-context Euler steps on the entire current object (all designable residues plus all bound context)
Repack sidechains with a full PIPPACK pass
Compute global clash and packing-disagreement statistics and return updated full-atom state plus mean global u_{pack}

Algorithm 8 Selecting candidates for BioEmu and UMA branches

Require: Terminal candidate set $\mathcal{T}$, BioEmu budget B_{B}, UMA budget B_{U}

for $x \in \mathcal{T}$ **do**
    Compute BioEmu acquisition score $A_{\text{B}}(x)$ from Eq. (87)
end for
Cluster $\mathcal{T}$ by sequence identity or edit-set overlap
Select top candidates within each cluster using asynchonous cyclic runs by $A_{\text{B}}(x)$ until B_{B} is reached
Run BioEmu or retrieve cached BioEmu scores for the selected set $\mathcal{T}_{\text{B}}$
for $x \in \mathcal{T}_{\text{B}}$ **do**
    Compute UMA acquisition score $A_{\text{U}}(x)$ from Eq. (88)
end for
Cluster $\mathcal{T}_{\text{B}}$ again if needed and select B_{U} candidates by asynchronous cyclic runs on $A_{\text{U}}(x)$
Return selected sets for BioEmu and UMA

C Appendix C: recommended experimental table templates

The following tables should appear in the eventual empirical paper, even though no numbers are filled in here.

Table 7: Template for oracle validation on experimental stability datasets.

Dataset	SPURS Spearman	BioEmu Spearman	UMA Spearman	Fused Spearman	Notes on homology filtering and calibration setting
Megascale holdout					
FireProt(HF)					
S669					
Ssym					
BioEmu-eligible					
large-protein subset ($N_{\text{atom}} > 8000$)					
Whole-protein UMA subset ($N_{\text{atom}} \leq 8000$)					

Table 8: Template for design-sampler evaluation on held-out backbones.

Model	Best-of-64 reward	Top-8 mean reward	Diversity	Mean Hamming dist./setting	scTM/scRMSD pass rate	Notes
Base ADFLIP						
Greedy SPURS						
PPO editor						
Simul-MF						
Seq-Curr						
OneShot-AL						

D Appendix D: checklist for a first implementation

1. Cache ADFLIP seeds, denoiser purity summaries, SPURS mutation matrices, and BioEmu-eligibility tags.
2. Verify canonical edit ordering and deterministic parent reconstruction.
3. Confirm that the stop action is reachable and not trivially dominant.
4. Track mutation-order histograms for the bins $(x^{\text{base}}, x^{(0)}, K = 1, K = 2, K \geq 3)$ from the first training epoch.
5. Monitor u_{pack}, terminal purity, and seed-to-terminal sequence distance continuously.
6. Validate SPURS calibration on held-out natural proteins before running de novo design.
7. Validate BioEmu calibration separately on BioEmu-eligible proteins and on the large-protein setting where it is expected to dominate the whole-chain dynamics signal.

8. Use whole-protein UMA-MD up to roughly 8000 atoms when the prepared system fits the budget; switch to BioEmu-dominant whole-chain screening above that size or when whole-protein UMA calibration degrades, and reserve UMA for local or hybrid corrections.

9. Validate catalytic metadata explicitly: every catalytic record should carry substrate metadata, reactant-bound and product-bound endpoint complexes, protein chain IDs, and pocket positions before the default UMA-cat loop begins.

10. Calibrate the whole-enzyme UMA catalytic proxy and GraphKcat separately before fusing them, and log disagreement rather than averaging it away; treat KcatNet and MMKcat as auxiliary predictive branches rather than the default catalytic control loop.

11. Log both direct and corrected expensive rewards separately.

12. Preserve a frozen copy of the initial ADFLIP for anchor loss and rollback.

13. For OneShot-AL, log teacher-student KL, top-k overlap, and high-order tail coverage every active-learning round.

14. Pre-register candidate selection rules before any wet-lab testing.

E Appendix E: bibliography notes and source provenance

The methodology in this manuscript was derived from ten primary source clusters reviewed in detail: the ADFLIP ICML paper for all-atom inverse folding, multistate conditioning, and training-free guidance, the SPURS article for scalable thermostability prediction, the BioEmu preprint for scalable equilibrium-ensemble emulation and stability-relevant protein dynamics, the core GFlowNet theory papers, the 2021–2022 GFlowNet literature on non-iterative diverse candidate generation and biological sequence active learning, the UMA or OMol25 releases for MLIP-based molecular simulation and data provenance, the KcatNet, GraphKcat, and MMKcat papers for catalytic turnover and kinetic-parameter prediction, the ProTrek paper for joint sequence-structure embedding and similarity-aware clustering of RF3 endpoint pairs, and the nonequilibrium free-energy literature underlying Jarzynski, Crooks, and Hummer–Szabo style path reconstruction [1, 2, 8, 12–19, 21, 22, 29**? ?** , 30]. In addition, the updated large-system deployment scope in this paper is informed both by the FAIR Chemistry public documentation of large-system UMA-MD execution, including a worked 8000-atom example and reported throughput on larger systems, and by the BioEmu results on scalable equilibrium sampling and protein stability estimation [5, 16]. All proposed additions beyond those sources are explicitly framed as methodological proposals rather than established results.

References

[1] Emmanuel Bengio, Moksh Jain, Maksym Korablyov, Doina Precup, and Yoshua Bengio. Flow Network based Generative Models for Non-Iterative Diverse Candidate Generation. In *Advances in Neural Information Processing Systems*, 34, 2021.

[2] Yoshua Bengio, Salem Lahlou, Tristan Deleu, Edward J. Hu, Mo Tiwari, and Emmanuel Bengio. GFlowNet Foundations. *Journal of Machine Learning Research*, 24(210):1–55, 2023.

[3] Justas Dauparas, Ivan Anishchenko, Neil Bennett, et al. Robust deep learning based protein sequence design using ProteinMPNN. *Science*, 378(6615):49–56, 2022.

[4] Hendrik Dieckhaus, Marco Brocidiacono, Noah Z. Randolph, and Brian Kuhlman. Transfer learning to leverage larger datasets for improved prediction of protein stability changes. *Proceedings of the National Academy of Sciences*, 121(6):e2314853121, 2024.

[5] FAIR Chemistry. *fairchem: FAIR Chemistry's library of machine learning methods for chemistry*. GitHub repository, accessed 2026. https://github.com/facebookresearch/fairchem.

[6] P. M. Ghari, A. M. Tseng, and collaborators. GFlowNet Assisted Biological Sequence Editing. In *Advances in Neural Information Processing Systems*, 2024.

[7] Ching-Hua Hsu, Robert Verkuil, Jiaxuan Liu, et al. Learning inverse folding from millions of predicted structures. In *Proceedings of ICML*, pages 8946–8970, 2022.

[8] Moksh Jain, Emmanuel Bengio, Alex Hernández-García, et al. Biological Sequence Design with GFlowNets. In *Proceedings of ICML*, pages 9786–10002, 2022.

[9] Moksh Jain, Sharath Chandra Raparthy, Alex Hernández-García, et al. Multi-Objective GFlowNets. In *Proceedings of ICML*, 2023.

[10] Bowen Jing, Stephan Eismann, Patricia Suriana, Raphael Townshend, and Ron Dror. Learning from protein structure with geometric vector perceptrons. In *International Conference on Learning Representations*, 2021.

[11] Minsu Kim, Joohwan Ko, Dinghuai Zhang, Ling Pan, Taeyoung Yun, Woochang Kim, Jinkyoo Park, and Yoshua Bengio. Learning to Scale Logits for Temperature-Conditional GFlowNets. In *NeurIPS AI for Science Workshop*, 2023.

[12] Gavin E. Crooks. Entropy production fluctuation theorem and the nonequilibrium work relation for free energy differences. *Physical Review E*, 60(3):2721–2726, 1999.

[13] Gerhard Hummer and Attila Szabo. Free energy reconstruction from nonequilibrium single-molecule pulling experiments. *Proceedings of the National Academy of Sciences*, 98(7):3658–3661, 2001.

[14] Christopher Jarzynski. Nonequilibrium equality for free energy differences. *Physical Review Letters*, 78(14):2690–2693, 1997.

[15] Daniel S. Levine, Muhammed Shuaibi, Evan Walter Clark Spotte-Smith, et al. The Open Molecules 2025 (OMol25) Dataset, Evaluations, and Models. arXiv:2505.08762, 2025.

[16] Sarah Lewis, Tim Hempel, Jose Jimenez-Luna, Michael Gastegger, Yu Xie, Andrew Y. K. Foong, Victor Garcia Satorras, and collaborators. Scalable emulation of protein equilibrium ensembles with generative deep learning. bioRxiv, 2024. doi:10.1101/2024.12.05.626885.

[17] Ding Luo, Xiaoyang Qu, and Binju Wang. Catalytic pocket-informed augmentation of enzyme kinetic parameters prediction via hierarchical graph learning. bioRxiv, 2025. doi:10.1101/2025.05.18.654694.

[18] Ziang Li and Yunan Luo. Generalizable and scalable protein stability prediction with rewired protein generative models. *Nature Communications*, 17:891, 2026.

[19] Tong Pan, Xin Cui, Huan Yee Koh, Yue Bi, Xiaoyu Wang, Yumeng Zhang, Shantong Hu, Geoffrey I. Webb, Lukasz Kurgan, Guimin Zhang, and Jiangning Song. A geometric deep learning framework for genome-wide prediction of enzyme turnover number. *Genome Biology*, 2026.

[20] Varun Ullanat, Bowen Jing, Samuel Sledzieski, and Bonnie Berger. Learning the language of protein-protein interactions. bioRxiv, 2025. doi:10.1101/2025.03.09.642188.

[21] Kanika Madan, Emmanuel Bengio, Tristan Deleu, and collaborators. Learning GFlowNets from partial episodes for improved convergence and stability. In *Proceedings of ICML*, 2023.

[22] Nikolay Malkin, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: improved credit assignment in GFlowNets. In *Advances in Neural Information Processing Systems*, 2022.

[23] Pascal Notin, Andrew W. Kollasch, David Ritter, et al. ProteinGym: large-scale benchmarks for protein fitness prediction and design. In *Advances in Neural Information Processing Systems*, 2023.

[24] Ling Pan, Dinghuai Zhang, Moksh Jain, et al. Better Training of GFlowNets with Local Credit and Incomplete Trajectories. In *Proceedings of ICML*, 2023.

[25] Claudia Pancotti, Stefano Benevenuta, Giorgio Birolo, et al. Predicting protein stability changes upon single-point mutation: a thorough comparison of the available tools on a new dataset. *Briefings in Bioinformatics*, 23(2):bbab555, 2022.

[26] Sean Reeves and Subramanian Kalyaanamoorthy. Zero-shot transfer of protein sequence likelihood models to thermostability prediction. *Nature Machine Intelligence*, 6:1063–1076, 2024.

[27] Vikram R. Shanker, Thomas U. J. Bruun, Brian L. Hie, and Patrick S. Kim. Unsupervised evolution of protein and antibody complexes with a structure-informed language model. *Science*, 385(6704):46–53, 2024.

[28] Richard W. Shuai, Talal Widatalla, Po-Ssu Huang, and Brian L. Hie. Sidechain conditioning and modeling for full-atom protein sequence design with FAMPNN. bioRxiv, 2025. doi:10.1101/2025.02.13.637498.

[29] Xin Sun, Yu Guang Wang, and Yiqing Shen. A multimodal deep learning framework for enzyme turnover prediction with missing modality. *Computers in Biology and Medicine*, 193:110348, 2025.

[30] Jin Su, Yan He, Shiyang You, Shiyu Jiang, Xibin Zhou, Xuting Zhang, Yuxuan Wang, Xining Su, Igor Tolstoy, Xing Chang, and others. A trimodal protein language model enables advanced protein searches.